



University of Bahrain
College of Science
Department of Mathematics
Academic Year 2025–2026

Graph Data Science

Thesis submitted in partial fulfillment of the requirements for the degree of
B.Sc. in Mathematics

Done by Zainab Abdulwahab Hassan

ID Number: 202209593

Under the Supervision of Dr. Abdulla Eid

الإهداء

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ
وَأَخِرُ دَعْوَاهُمْ أَنْ الْحَمْدُ لِلَّهِ رَبِّ الْعَالَمِينَ

الحمد لله الذي ما نجحنا ولا علونا إلا برضاه،
الحمد لله الذي ما اجتزنا درباً ولا تخطينا جهداً إلا بفضلته
بالحمدِ نَحْنُ ما بدأناهُ بالدعاء، وبالشكرِ تُزهرُ ثمارُ صبرنا وعنائنا
ها قد انتهى الطريق، وما هو بانتهاه، إنما هو بدءٌ جديدٌ حُفَّتْ خطاهُ بالإمتنان

إلى من بهم قَوِيَّ عودي، وارتفعت هامتي، ومضيتُ نحو النور دون أن أَلْتَفَت خلفي خوفاً أو تعباً ..

إلى سندي الأول وضلعي الثابت، أَسِي العزيز .. إن كان في هذا اليوم فخرٌ يُقال، فهو أنك كنت السبب بعد الله
في كل ما أنا عليه، هذا التخرج ليس بشهادتي وحدي بل هو تعب سنيك، وحرصك، وحبك الصامت الذي كان
يكفيني عن ألف كلمة. أسأل الله أن أكون دائماً الابنة التي تفخر بها كما أفتخر بك أمام العالم كله

إلى ظل الجنة في الدنيا، ومصدر الطمأنينة في قلبي أُمِّي الحبيبة .. كل كلمات الدنيا لا تكفي لأعبر عن
امتناني لك، كنتِ نوراً أضاء طريقي وسط ظلمات الحياة، ودعاؤك كان مفتاح كل بابٍ مغلق. فما خاب دربٌ كانت
أُمِّي تمهّده بالدعاء، هذا اليوم هو يومك قبل أن يكون يومي، وفرحتي هي انعكاس لدموع الفرح في عينيك لأنك
من خفتِ وبكيتِ شوقاً لهذه اللحظة
أُهديكِ هذا النجاح يا جنتي ونعيمي

إلى حضرة الدكتور عبدالله عيد .. أتقدّم لك بخالص الشكر وعظيم الامتنان على كل ما قدّمته لي من دعمٍ
وتشجيع طوال هذه الرحلة .. ثقتك بي، وإيمانك بقدراتي، هم الدافع الأكبر للإصرار والتّميّز ..
شكراً لجهودك وتوجيهك الكريم سائلةً المولى لك دوام التوفيق ورفعته المكانة

أدركت الآن أن الصبرَ مر، لكن ثمره أبهى، وأن من استودع الله سعيه، نال ما تمنى .. وأرى السماء بعيني بعدما
كُنْتُ أرجوها بالدعاء

تمّ المرادُ بفضلِ الله وتوفيقه
انتهت رحلَةُ السعي، ولم تنتهِ خطى الطموح
فالحمدُ لله الذي بلغنا التمام، وجعلَ الختامَ مسكاً لا يزول

Acknowledgement

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ
وَأَخِرُ دَعْوَاهُمْ أَنِ الْحَمْدُ لِلَّهِ رَبِّ الْعَالَمِينَ

First and foremost, all praise and gratitude belong to Allah, the Most Merciful and Most Compassionate. By His grace, guidance, and countless blessings, I have been able to complete this journey. Every success begins with His will, and every achievement is made possible through His mercy.

To My Dearest Father,

To the man who has been my first source of strength and inspiration. Thank you for your endless sacrifices, unwavering support, and constant belief in me. Whatever I have achieved today is a reflection of your hard work, patience, and encouragement. I hope to always make you as proud of me as I am proud to be your daughter.

To My Beloved Mother,

To the light of my life and the source of my comfort and peace. Your prayers have guided me through every challenge, and your love has given me the strength to continue whenever the path seemed difficult. This achievement is as much yours as it is mine, for your support has been present in every step of this journey.

To My Respected Supervisor, Dr. Abdulla Eid,

I would like to express my sincere gratitude for your guidance, encouragement, and continuous support throughout the years. Your valuable guidance, patience, and confidence in my abilities have played a significant role in helping me achieve this milestone successfully.

In Conclusion,

I thank everyone who supported me, believed in me, and contributed to this journey in any way. This accomplishment marks not only the completion of this project but also the beginning of a new chapter filled with hope, aspirations, and opportunities.

Contents

Introduction	6
1 Graph-Based Data Representation	7
1.1 Graphs and Relational Structure	7
1.2 Graph-Shaped Problems and Structural Examples	8
2 Representing Network Structure	10
2.1 Understanding Graph Structures and Types	11
2.2 Designing Graph Models	14
2.3 Extracting Knowledge from Text	15
3 Social Network Analysis	18
3.1 The Four Questions of Graph Analysis	18
3.2 The Fundamentals of Social Network Analysis (SNA)	18
3.3 Follower Networks and Directional Influence	19
3.4 Degree Measures and Their Limitations	20
3.5 Structural Connectivity and Centrality	22
3.6 PageRank and Personalized PageRank in Social Networks	25
3.7 Personalized PageRank Algorithm	28
4 Projecting Monopartite Networks	31
4.1 Motivation for Monopartite Projection	31
4.2 Translating Indirect Patterns into Direct Relationships	31
4.3 Projected Network Analysis and Influence Interpretation	32
4.4 Handling Self-loops in Projected Networks	33
4.5 Weighted Influence Measures in Cleaned Projected Networks	33
5 Inferring Co-occurrence Networks from Bipartite Graphs	35
5.1 From Projection to Co-occurrence	35
5.2 Normalization with Jaccard Similarity	35
5.3 Community Detection and Representative Nodes	37
6 Similarity-Based Network Construction from Node Features	39
7 Node Embeddings and Node Classification	43
7.1 Foundations of Node Embeddings	43
7.2 Learning Embeddings Through Random Walks	46
7.3 From Embeddings to Node Classification	49
7.4 Considerations for Isolated Nodes	51
8 Link Prediction in Networks	52
8.1 Motivation and Concept of Link Prediction	52
8.2 Structural Evidence Used to Infer Missing Links	53
8.3 From Structural Signals to Predictive Interpretation	54

9	Knowledge Graph Completion	57
9.1	Motivation and Nature of Knowledge Graphs	57
9.2	Capturing Semantic Structure Through Knowledge Graph Embeddings .	58
9.3	Interpreting Predictions and Practical Considerations	60
10	Constructing Graphs from Text Using Natural Language Processing	62
10.1	Motivation for Building Graphs from Text	62
10.2	Inferring Structure Through Co-occurrence and Graph Analysis	63
10.3	From Text to Graph: Integration and Analysis	64
10.4	Challenges and Considerations	65
11	Practical	67
11.1	Implementation Overview	67
11.2	Dataset Description	69
11.3	Data Preprocessing	70
11.4	Graph Construction	71
11.5	Personalized PageRank Algorithm	71
11.6	Evaluation	75
11.7	Visualization	78
11.8	Extensions	83
11.9	Non-Graph Models	92
11.10	Final Model Comparison	102
11.11	Extended Analysis	107
	Conclusion	110

Introduction

Graph Data Science has become an important field for analyzing highly connected data and discovering hidden patterns within complex networks. Unlike traditional data analysis methods that focus mainly on isolated records or tabular structures, graph-based approaches emphasize relationships and interactions between entities. This relational perspective allows graphs to model real-world systems more naturally, including social networks, recommendation systems, communication networks, biological systems, and knowledge graphs.

In many modern applications, the relationships between entities are often more important than the entities themselves. For example, in recommendation systems, user preferences can be inferred not only from direct interactions but also from indirect relationships, shared interests, and structural similarity within the network. Graph algorithms therefore provide powerful techniques for understanding connectivity, influence, similarity, and information flow in such systems [5].

Among graph-based ranking algorithms, PageRank and Personalized PageRank have gained significant importance due to their ability to evaluate node importance based on network structure. While standard PageRank measures global importance within the graph, Personalized PageRank introduces user-specific bias through a personalized random walk process, enabling recommendations tailored to individual users and contexts [6, 7].

This project explores the theoretical foundations and practical implementation of graph-based recommendation systems, using Personalized PageRank as the core recommendation mechanism.

The report first introduces fundamental concepts including graph representation, social network analysis, centrality measures, graph projection, co-occurrence networks, node embeddings, link prediction, and knowledge graph completion.

The practical component is implemented using the MovieLens dataset, where user-movie interactions are modeled as a bipartite graph. The implementation includes graph construction, recommendation generation, evaluation using ranking metrics such as Precision@K, Recall@K, and NDCG@K, as well as comparative analysis with non-graph approaches including KNN, SVD, Random Forest, content-based recommendation, hybrid models, and Graph Neural Networks (LightGCN).

The objective of this project is to demonstrate how graph-based methods can model relational data and generate personalized recommendations, while comparing their strengths and limitations with traditional machine learning approaches.

1 Graph-Based Data Representation

1.1 Graphs and Relational Structure

Graphs are a data modeling approach that focuses on representing data through connections rather than isolated records. Instead of describing entities independently, graphs describe how entities relate to each other. Formally, a graph is defined as $G = (V, E)$ [3], where V is a finite set of vertices and E is a set of edges, with each edge being a subset of two-elements of V , as illustrated in Figure 1. This definition establishes the foundation for modeling interactions, dependencies, and paths within connected data.

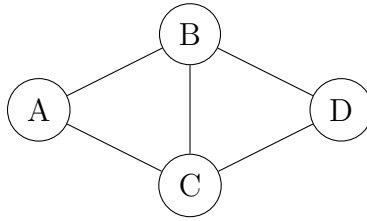


Figure 1: Graph $G = (V, E)$

In many real-world applications, data is traditionally stored in tabular form. Although tables are efficient for aggregation and numerical analysis, they struggle to capture complex relational structures. As relationships extend beyond direct connections, such as friends of friends or multi-step interactions, tabular representations require increasingly complex joins. This limitation motivates the use of graphs, where relationships are treated as first-class components of the model. For example, two movies may appear unrelated in a table, yet become connected in a graph through a shared actor, as shown in Figure 2.

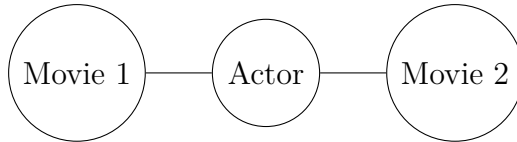


Figure 2: Indirect connection via actor

By explicitly modeling relationships, graphs allow both direct and indirect connections to be explored naturally. Vertices may represent people, movies, books, or products, while edges represent interactions such as friendships, collaborations, or shared participation.

Once relationships are made explicit, higher-level structures begin to emerge. One such structure is a **community**, defined as a group of vertices that are densely connected to each other but only loosely connected to the rest of the graph. In movies or books, characters who interact frequently within the same storyline have many connections between them, causing them to appear as a distinct community on the graph, as indicated in Figure 3.

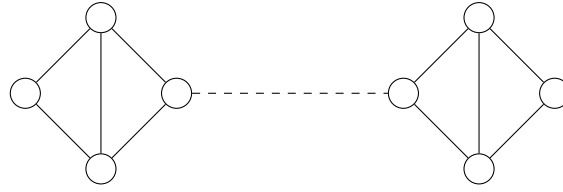


Figure 3: Two densely connected communities

Another important concept that arises from the relational structure is **influence**. Certain vertices play a disproportionate role in maintaining connectivity or enabling information flow. These influential vertices often act as bridges between communities. In storytelling terms, this corresponds to a central character who links multiple storylines and whose removal would fragment the narrative, as illustrated in [Figure 4](#).

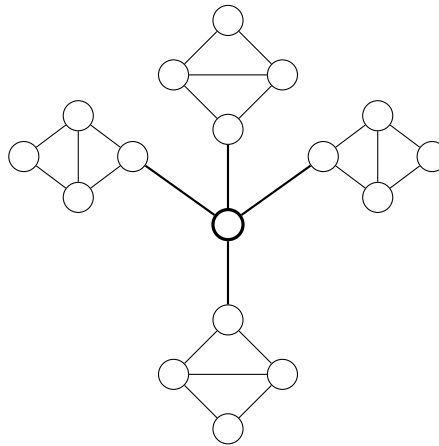


Figure 4: Influential vertex connecting communities

1.2 Graph-Shaped Problems and Structural Examples

Using graphs shifts the focus from individual entities to their connections. Graphs encourage a relational view of data, which leads to graph-shaped problems where connections between entities are essential. This occurs when data are highly connected, when paths matter, or when indirect relationships affect outcomes.

One direct consequence of this relational view is **pathfinding**. Many problems involve finding efficient paths in a network, such as the shortest route or the minimum cost connection.

This is similar to following a character's movement between connected locations, where the path taken is more important than each location on its own, as seen in [Figure 5](#).

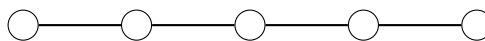


Figure 5: A simple path in a graph

In some applications, relationships exist only between different types of entities. This leads to **bipartite graphs**, where vertices are divided into two distinct sets and edges connect only vertices from different sets.

A common example is readers connected to the books they have read. Although readers

are not directly connected to each other, similarities between them can be identified through shared books, as depicted in [Figure 6](#).

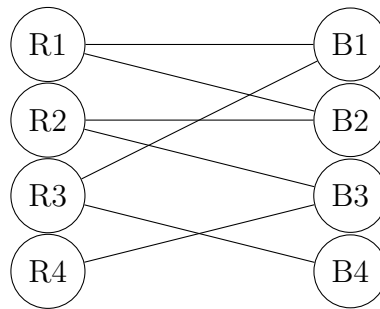


Figure 6: Bipartite graph of readers and books

As data systems grow in size, these properties often appear together, resulting in more complex networks that involve multiple types of vertices, different types of relationships, and high levels of connectivity. In such cases, graph-based modeling provides a clear and scalable way to represent structure, discover patterns, and support more advanced analytical methods introduced later.

2 Representing Network Structure

Before applying graph algorithms or machine learning techniques, it is important to understand how data is represented as a graph. Graph modeling provides the basic language for describing network structures and explains how different graph types influence analysis. Since many algorithms require specific graph structures as input, understanding these representations is a necessary first step.

The learning path in graph data science typically begins with **graph modeling**, followed by **graph query languages and graph algorithms**, and ultimately **graph-based machine learning** [1]. Among these stages, graph modeling forms the foundation upon which all subsequent analysis is built.

Graph Data Science provides a structured framework for extracting insight from connected data by combining **graph modeling**, **statistical analysis**, and **algorithmic techniques**. Rather than focusing on individual data points, it emphasizes relationships and network structure as primary sources of information. Within this framework, graph data science is commonly organized into three complementary layers.

The first layer consists of **graph statistics**, which provide descriptive measures of the network and support an initial understanding of its structure, such as degree distributions and connectivity patterns. The second layer involves **graph analytics**, where algorithms are applied to analyze structural properties of the graph, including centrality, community structure, and similarity relationships. The third layer is **graph-enhanced machine learning**, which integrates graph-derived insights into learning-based workflows. In this stage, information extracted from the graph, such as structural measures or learned representations, is used to support prediction, classification, and inference tasks. This layered organization highlights how graph modeling and analysis naturally progress toward more advanced learning and decision-making applications.

Understanding degree-based properties is essential in network analysis; therefore, we first state a fundamental theorem from graph theory.

Theorem 2.1 (Handshaking Lemma (The First Theorem of Graph Theory)). *If G is a graph of size m , then*

$$\sum_{v \in V(G)} \deg v = 2m.$$

[3]

Proof. Each edge in an undirected graph contributes exactly two to the total degree count, one for each of its incident vertices. Therefore, summing the degrees of all vertices counts each edge twice, which gives $2|E|$. ■

Example 2.2. In graph data science, networks are often used to model interactions between entities, such as users in a social network or devices in a communication system. Suppose a graph represents a social network where vertices correspond to users and edges represent interactions between them. If the network contains m interactions, the Handshaking Lemma ensures that the sum of all users interaction counts (degrees) is exactly

$2m$. This property is useful for validating network data and detecting inconsistencies in large-scale graph datasets.

An immediate consequence of the Handshaking Lemma is the following result:

Corollary 2.3. *Every graph has an even number of odd vertices.*

[3]

Proof. Suppose not. Assume there exists a graph with an odd number of odd vertices. By the First Theorem of Graph Theory,

$$\sum_{u \in V(G)} \deg(u) = 2m.$$

We can write

$$\sum_{\substack{u \in V(G) \\ u \text{ is odd}}} \deg(u) + \sum_{\substack{u \in V(G) \\ u \text{ is even}}} \deg(u) = 2m.$$

The sum of degrees of odd vertices is odd, while the sum of degrees of even vertices is even. Hence, the left-hand side is odd, whereas the right-hand side is even, which is a contradiction. Therefore, every graph has an even number of odd vertices. ■

This corollary is useful in network analysis for identifying anomalies in degree distributions.

2.1 Understanding Graph Structures and Types

Graphs can take different forms depending on how relationships are defined. One common distinction is between **directed** and **undirected** graphs. In directed graphs, relationships have a clear direction that governs how information flows through the network, as shown in Figure 7 (a). In contrast, undirected graphs represent mutual relationships where all connections are bidirectional, as shown in Figure 7 (b).



Figure 7: Directed and undirected graphs

When representing directed networks such as social networks and information networks, connectivity must take edge direction into account. This leads to the concept of strong connectivity, which captures whether entities in the network are mutually reachable through directed paths and directly affects the applicability of graph-based analysis and ranking algorithms.

Definition 2.4 (Strong Connectivity). A directed graph D is said to be strongly connected if, for every pair of distinct vertices u and v , there exists a directed path from u to v and also a directed path from v to u .

Definition 2.5 (Spanning Walk). A spanning walk in a graph G is a walk that visits every vertex of G at least once. If the walk begins and ends at the same vertex, it is called a closed spanning walk.

Theorem 2.6 (Strong Digraph Characterization). A digraph D is strong if and only if D contains a closed spanning walk.

[3]

Proof. Suppose first that D contains a closed spanning walk W . Since W visits every vertex of D , for any two distinct vertices u and v , there exists a directed walk from u to v and also from v to u . By standard results in digraph theory, the existence of such walks implies the existence of directed paths in both directions. Hence, D is strong.

Conversely, assume that D is a strong digraph with a set of vertex $V(D) = \{v_1, v_2, \dots, v_n\}$. Since D is strong, there exists a directed path from v_i to v_{i+1} for each $i = 1, \dots, n-1$, as well as a directed path from v_n back to v_1 . By concatenating these paths, we obtain a closed walk that visits every vertex of D . Therefore, D contains a closed spanning walk. ■

The relevance of strong connectivity to random-walk-based ranking methods, such as PageRank [6], will be clarified in later chapters.

Another important distinction is between **weighted** and **unweighted** graphs. In unweighted graphs, all relationships are treated equally, and only the existence of a connection matters. Weighted graphs, on the contrary, assign numerical values to relationships to represent factors such as distance, cost, or frequency, as presented in Figure 8.

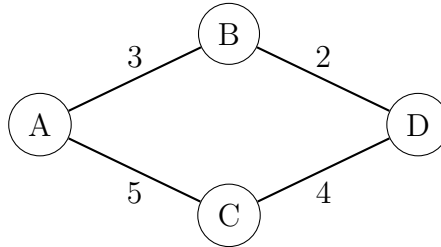


Figure 8: Weighted graph

Graphs may also differ based on how nodes are grouped. A **monopartite** graph contains a single type of node, where any node may connect to any other, as display in Figure 9 (a).

On the other hand a **bipartite** graph, by contrast, consists of two distinct node sets, with connections allowed only between the two sets, as shown in Figure 9 (b).



Figure 9: Monopartite and bipartite graphs

Bipartite graphs can be further transformed by **projection** into monopartite graphs. This process creates direct connections between nodes of the same type based on shared neighbors, often increasing the number of edges and introducing weights.

Bipartite graphs commonly appear in real-world applications that model interactions between two distinct types of entity, such as readers and books. In this setting, edges exist only between readers and the books they have read, and there are no direct reader-reader connections, as illustrated in Figure 10 (a) .

To uncover similarities among readers, this bipartite structure can be transformed through projection into a monopartite graph. Under this transformation, readers become directly connected when they share books, with edge weights reflecting the number of shared interactions Figure 10 (b).



Figure 10: Graph projection example

A graph can be classified based on the number of relationships allowed between nodes. A **simple graph** permits at most one edge between any pair of nodes and does not include self-loops , as seen in Figure 11. For example, a basic friendship network where two individuals are either connected or not can be modeled as a simple graph.

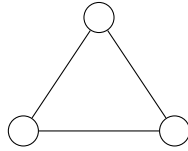


Figure 11: Simple graph

In contrast, a **multigraph** allows multiple edges between the same pair of nodes, making it more suitable for representing repeated or different types of interactions, as seen in Figure 12. For instance, multiple flights between two cities or several collaborations between two researchers can be modeled using a multigraph.

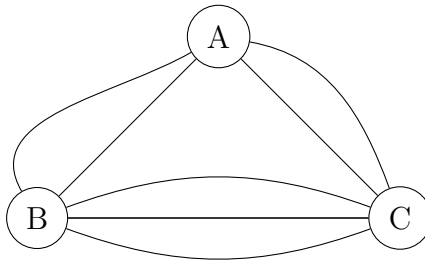


Figure 12: Multigraph with different numbers of edges between vertex pairs

A special case is the **complete graph**, where every pair of distinct nodes is connected

by exactly one edge, representing maximum connectivity, as seen in [Figure 13](#). A simple example is a small group where everyone knows everyone else.

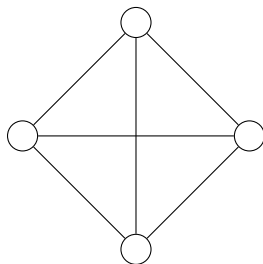


Figure 13: Complete graph

2.2 Designing Graph Models

Designing a graph model starts with understanding the data at a conceptual level before thinking about tools or algorithms. The goal is to decide what should be represented in the graph and how different elements are connected. Graph modeling is therefore a decision-making process guided by the analysis objectives rather than fixed rules.

The first step in graph modeling is identifying **entities**. Entities are elements that exist independently and can participate in multiple relationships, such as people, documents, and locations. These entities form the core of the graph and are represented as nodes, as observed in [Figure 14](#).

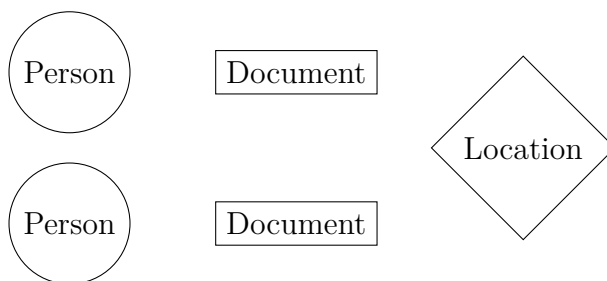


Figure 14: Nodes representing different entity types

Not all information should be modeled as nodes. Elements that are purely descriptive and do not need to form relationships are better represented as properties. The same information may be modeled differently depending on the analytical goal, which highlights the flexibility of graph models.

After identifying entities, the next step is defining **relationships**. Relationships describe how entities interact, depend on, or influence one another, and they cannot exist independently of the nodes they connect. The direction of a relationship is often critical, as it can change the meaning of the interaction, as displayed in [Figure 15](#).

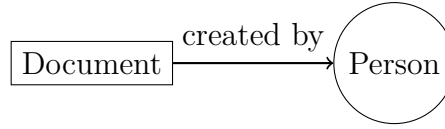


Figure 15: A directed relationship between entities

Another important modeling decision is the **level of granularity**. A graph does not need to capture all possible details at once. It is usually more effective to start with a simple model that captures key entities and relationships, then refine it as analytical needs evolve. To support consistency, a **graph schema** can be used to define the types and relationships of nodes allowed [4].

To maintain consistency and organization within a graph model, a graph schema can be used to define the permitted node types and relationships. The schema acts as a structural framework that specifies how entities are connected and what types of interactions are allowed between them. This helps prevent inconsistent or invalid relationships and ensures that the graph remains organized and suitable for analysis.

For example, in a movie recommendation system, the schema may define node types such as users, movies, actors, and genres. It also specifies the valid relationships between these nodes. A user may be connected to a movie through a *WATCHED* or *RATED* relationship, while a movie may be connected to a genre through a *BELONGS_TO* relationship or linked to an actor through an *ACTED_IN* relationship. The schema also prevents invalid relationships that do not match the system structure, such as a movie *WATCHING* a user or an actor *WATCHING* another actor. By defining these rules in advance, the graph model becomes more organized, consistent, and easier to analyze using graph algorithms and recommendation techniques.

2.3 Extracting Knowledge from Text

Once a graph model is defined, it can be used to structure information extracted from unstructured data sources such as text. Textual data often contains valuable knowledge, but this knowledge is usually implicit and not organized in a way that supports direct analysis.

A key step in knowledge extraction is to **identify entities within text**. These entities represent important elements with independent meaning and often form the main nodes of the resulting graph, such as people, organizations, locations, and concepts, as presented in [Figure 16](#).

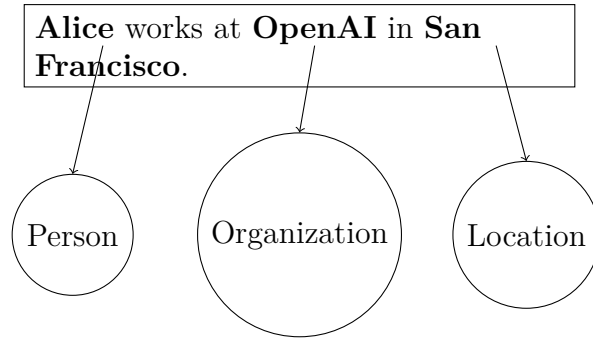


Figure 16: Extracting entities from text

In addition to entities, the text also contains **relationships** that describe how elements are connected. These relationships may be explicitly stated or inferred from context.

One common inference method in knowledge discovery is **co-occurrence** analysis, where entities that frequently appear together within the same documents are assumed to share a meaningful relationship [1].

This method allows us to model information as a graph: entities become nodes (vertices), and their co-occurrence frequencies define the edges (links) and their weights.

Illustrative example, consider a large corpus of text about world cities. By counting how often pairs of city names appear together in the same articles, we can construct a co-occurrence graph. For instance, "Paris" and "London" might appear together very frequently in travel guides, creating a strong edge (high weight) between them. Similarly, "London" and "New York" might co-occur often in business news, forming another strong connection. In contrast, "Paris" and "Rome" could appear together at a moderate rate in articles about European culture, resulting in an edge with a medium weight, while "Paris" and "New York" might appear together only occasionally, yielding a weak edge (low weight).

The resulting graph structure itself reveals the inferred relationships, with edge weights directly encoding the strength of connection between entities based on their co-occurrence patterns in the text.

The resulting graph structure itself reveals the inferred relationships. For instance, if we apply a **threshold** to remove weak connections, "Paris" and "New York" might become disconnected, highlighting that their relationship is not as direct or significant in the corpus. Figure 17. visualizes this co-occurrence graph.

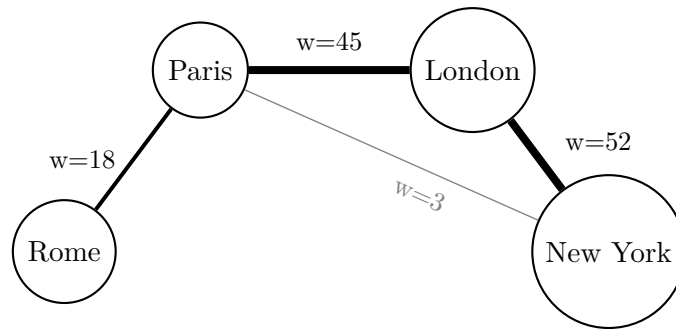


Figure 17: Co-occurrence graph with weighted edges indicating relationship strength.

More structure can be added using **hashtags**, **keywords**, or **topics**, which help group related content and link similar documents.

Mentions and references between entities or documents also provide contextual relationships that preserve meaning.

By representing extracted entities, relationships, and topics as nodes and edges, a knowledge graph can be constructed. This graph **enables structured analysis, supports the discovery of hidden patterns, and provides a connected view of information from multiple sources.**

3 Social Network Analysis

3.1 The Four Questions of Graph Analysis

Graph-based analysis is often guided by the types of questions that are asked about a network. In graph data science, these questions can be broadly categorized into four main areas:

- **Movement** questions focus on how things travel through a network. For example, how many steps does a post need to reach a friend-of-a-friend? What is the shortest route from home to work? Or how might a disease spread if an infected person enters a city?
- **Influence** questions aim to identify the most important nodes. Who are the real influencers whose retweets reach thousands? Which researchers are most cited? Or who connects different scientific fields together?
- **Group** questions seek to uncover communities with shared interests. On Twitter, we can group users into sports, politics, or technology communities. On Amazon, purchase patterns reveal which items are "frequently bought together" for recommendations.
- **Pattern** questions focus on structural similarities to predict missing relationships. Facebook's "People You May Know" works this way when you share ten mutual friends with someone, the platform suggests them as a potential connection based on that pattern.

Therefore, social network analysis primarily addresses influence and group-related questions by examining how the structure of the network shapes visibility, interaction, and connectivity among users.

3.2 The Fundamentals of Social Network Analysis (SNA)

After establishing these four broad categories of graph questions we can now focus on how they are applied in a specific and highly relevant field: **Social Network Analysis (SNA)**.

SNA takes the general tools of graph data science and focuses them squarely on networks of human interaction. Consequently, as we shift our focus to SNA, we will see that it is particularly concerned with the influence and group-related questions, as these are the keys to understanding how connections shape social behavior.

Social Network Analysis builds on graph-based data representation by focusing specifically on networks of human interaction [1, 2]. It doesn't introduce new graph structures, instead, it applies existing graph concepts to understand how relationships between people shape behavior, influence, and the flow of information.

In social networks, a person's importance isn't just about who they are, it is about where they are located in the network. Your position may matter more than your personal characteristics.

For example, imagine two people. One is a wealthy celebrity with millions of followers, but all his followers are similar and tightly connected. The other is an ordinary person who works as a translator between two different communities say, connecting Arabic-speaking and English-speaking groups. This ordinary person might actually be more influential when it comes to spreading ideas across cultures, simply because of their unique position as a bridge between otherwise separate worlds.

Social networks aren't static, they're dynamic systems where relationships evolve constantly. Friendships form, fade, or grow stronger over time. Think of a TV series like a soap opera. Two characters might be best friends in the first episode, have a conflict in the middle, and reconcile by the finale. Graph analysis can track these changes across episodes, measuring how the relationship strength fluctuates. This helps analysts understand how influence and social structure develop over time, capturing snapshots at different moments and comparing how the network transforms.

To better analyze and explore social networks, different graph-processing approaches are used depending on the analytical objective. Some methods focus on retrieving known information from the network, while others aim to discover hidden structures and patterns. This leads to the distinction between **graph queries** and **graph algorithms**.

Graph queries are used when the required information is already known and only needs to be retrieved from the network. They function like direct questions asked to the graph. For example, a query such as "Show me all of Ali's friends" simply retrieves existing connections associated with Ali.

Graph algorithms, on the other hand, are used when you want to uncover hidden patterns or structures that are not immediately obvious. They analyze the entire network to reveal insights you didn't know existed. For example, asking "Who are the hidden influencers in this group?" requires an algorithm, you don't know the answer beforehand, and the algorithm discovers it by analyzing everyone's position and connections.

In practice, analysts use both approaches together. They might start with queries to understand specific parts of the network, then run algorithms to discover hidden patterns, and finally use queries again to examine those discoveries in detail. This combination enables a complete and comprehensive understanding of how social networks function and evolve over time.

3.3 Follower Networks and Directional Influence

After understanding the foundations of Social Network Analysis, we can now examine one of its most common real-world applications: follower networks. On platforms like Instagram, Twitter, or TikTok, users connect through following relationships that create directed networks.

In these networks, each user is represented as a vertex, and a directed edge points from a follower to the user they follow. This direction is crucial because it reflects how information flows content always moves from the followed user to their followers.

As illustrated in Figure 18, this directional structure creates two distinct types of users based on their position in the network:

- **Broadcasters (High in-degree):** Users with many incoming edges are followed by a large number of others. They occupy structurally prominent positions and are well positioned to broadcast information widely. A content creator on Instagram followed by thousands of users is a perfect example, they can influence a wide audience through a single post.
- **Consumers (High out-degree):** Users with many outgoing edges follow many others but have few followers themselves. They primarily act as information consumers, receiving content from broadcasters. An ordinary user might follow hundreds of accounts, but unless reciprocal relationships exist, their ability to broadcast information remains limited.

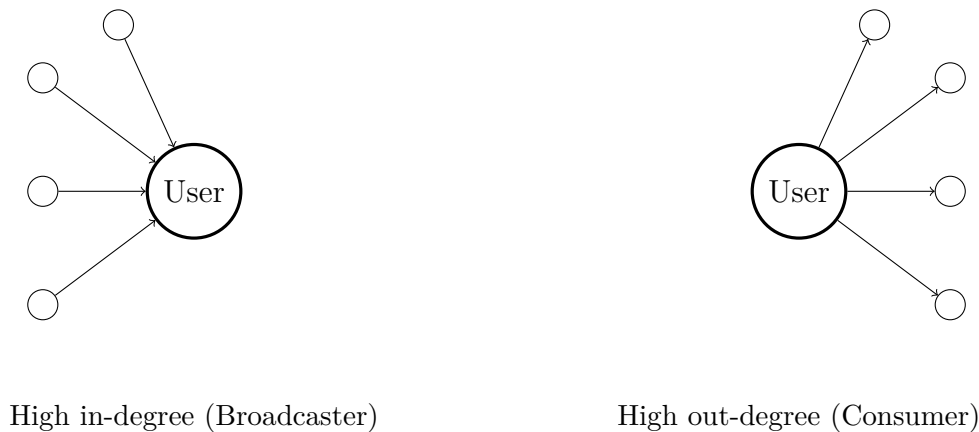


Figure 18: Incoming and outgoing links in a directed network

What makes these networks interesting is that influence isn't always one-way. When two users follow each other, creating a reciprocal relationship, the dynamic becomes more balanced. Both can broadcast to each other, and the relationship resembles a friendship more than a broadcast. However, in most social media platforms, the majority of relationships remain one-way, creating clear distinctions between content creators and content consumers.

3.4 Degree Measures and Their Limitations

After exploring follower networks and the distinction between broadcasters and consumers, we now turn to how influence is actually measured. The most basic way to measure a user's position in a network is through **degree**, simply counting their connections.

In directed networks like social media platforms, degree is divided into two types:

- **In-degree:** The number of incoming connections, or how many people follow a user.

- **Out-degree:** The number of outgoing connections, or how many accounts a user follows.

As shown in [Figure 19](#), nodes with high in-degree visually stand out as hubs in the network. A node like (A), with three followers but following no one, appears to be an important broadcaster at first glance.

However, degree measures have significant limitations, they only tell us about quantity, not quality. Two users may have identical in-degree values but completely different levels of actual influence.

Consider this example: Two authors each have exactly 1,000 followers. At first glance, they appear equally influential. But when we look closer, the first author is followed by influential critics, well-known writers, and active journalists who regularly share their work with their own large audiences. The second author is followed mostly by inactive readers who rarely engage or share content. Although their degree values are identical, their actual influence within the network is not. The first author's followers amplify their reach, while the second author's followers do not.

This limitation is illustrated in [Figure 19](#), where nodes A and B both have an in-degree of 3, but they differ in an important way. Node B follows one account (out-degree = 1), while node A follows no one. This small difference suggests that B might be more engaged in the network, potentially receiving information back from the account they follow, or having a different type of relationship with their followers.

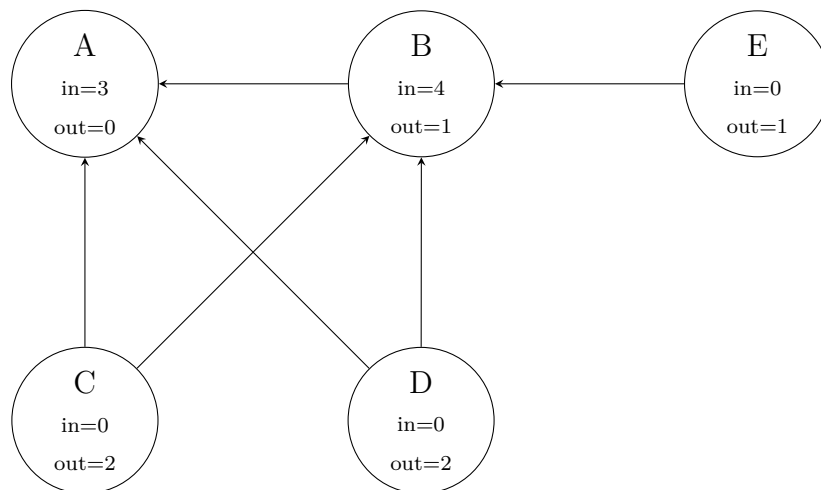


Figure 19: Directed graph illustrating in-degree and out-degree differences

What degree measures miss is the quality behind the numbers. A follower with high influence of their own can amplify your reach far more than an inactive one. Active followers engage and share your content, while inactive followers simply don't. Reciprocity also matters, mutual relationships create different dynamics than one-way broadcasts. And perhaps most importantly, being followed by someone who bridges different communities is far more valuable than being followed by isolated users with no reach of their

own.

Therefore, while degree measures provide a useful starting point for understanding network position, they are just the beginning. To capture the full complexity of social influence, we must move beyond simple counts and examine how users are structurally positioned within the network.

3.5 Structural Connectivity and Centrality

Understanding the connectivity of a social network is essential for describing its overall structure. Connectivity analysis focuses on how users are grouped and how connections are organized within the network, independently of influence or importance. By examining connectivity patterns, it becomes possible to identify **cohesive groups**, **isolated regions**, and **structural boundaries that shape interaction within the network**.

Before distinguishing different forms of connectivity, it is useful to establish a basic result relating walks and paths in a graph:

Theorem 3.1 (Walk Implies Path). *If a graph G contains a u - v walk of length l , then G contains a u - v path of length at most l .*

[3]

Proof. Among all u - v walks in G , let

$$W = (u = v_0, v_1, \dots, v_k = v)$$

be one of minimum length k , where $k \leq l$. We claim that W is a path. Suppose otherwise that some vertex is repeated in W , say $v_i = v_j$ for some $0 \leq i < j \leq k$. Removing the vertices $v_{i+1}, v_{i+2}, \dots, v_j$ from the walk produces a shorter u - v walk, contradicting the minimality of W . Therefore, W is a u - v path of length at most l . ■

This result ensures that whenever movement through a network is possible, it can be described using directed paths rather than arbitrary walks. Such path-based reachability provides a clear structural foundation for analyzing how information or influence can propagate through a network.

Types of Connectivity in Directed Networks

In directed social networks, connectivity is described using two important concepts:

- **Weakly connected components** identify groups of users that remain connected when edge directions are ignored. This reveals the underlying structure of the network regardless of information flow direction. For example, users connected through chains of following relationships, where information flows primarily in one direction form a weakly connected component. Although they are connected when direction is ignored, they cannot all reach each other through directed paths.
- **Strongly connected components** represent tightly interacting groups in which every user can reach every other user by following directed relationships. Within the same network, a smaller group of users may follow each other mutually, creating closed directed paths between them, as represented in [Figure 20](#).

This distinction is important because users may belong to the same general structure while differing significantly in their ability to exchange information.

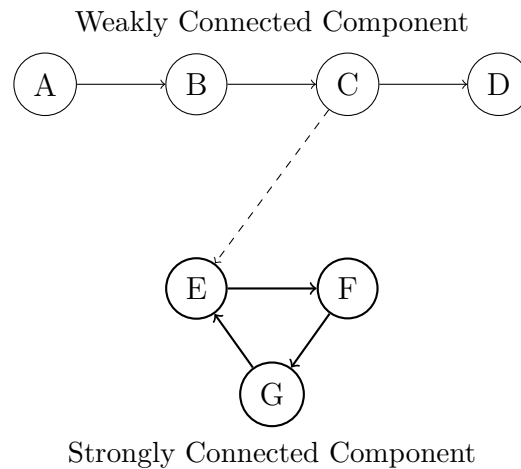


Figure 20: Weakly and strongly connected components in a directed social network

Local Clustering and Community Structure

Another important structural property is **local clustering**, which measures how interconnected a user's neighbors are. High clustering values often indicate the presence of tightly connected communities formed around shared interests or frequent interaction. When a user's followers are also connected to one another, this creates a dense local structure where information circulates primarily within the group rather than spreading to the broader network.

Central Nodes vs. Influential Nodes

Within this structural organization, not all users play the same role. The text distinguishes between two important types:

- **Central nodes** are users who occupy positions that are particularly important for maintaining connectivity across the network. Their importance arises from their position rather than from their individual attributes. A central node often connects multiple parts of the network and provides short paths between many other users.

As represented in [Figure 21](#), a central node may link several surrounding users and regions of the network, allowing interactions to occur efficiently, even if information does not necessarily flow through them in a dominant direction. Think of a central node as a major airport hub: many flights pass through it, but it may not be the final destination for most travelers.

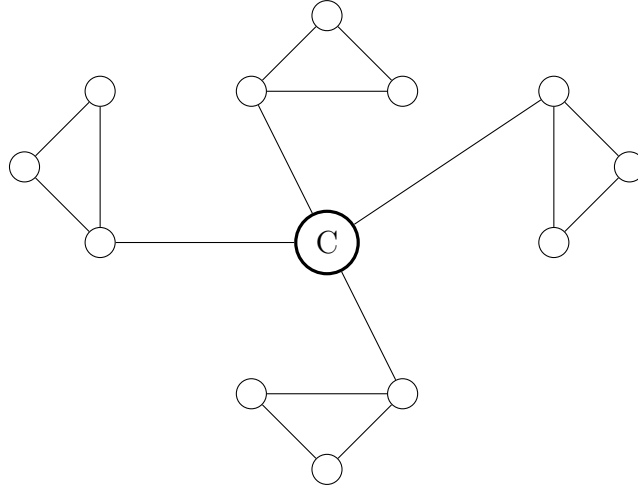


Figure 21: A central node connecting multiple communities within the network

- **Influential nodes** are a special type of central node that have a stronger impact on how information spreads through the network. They typically occupy strategic positions that control or mediate information flow between different groups or communities. Their influence is not determined by the number of connections alone, but by their ability to act as bridges between otherwise weakly connected regions.

This role is illustrated in [Figure 22](#), where an influential node connects two dense communities and directs information from one group to the other.

Removing such a node would significantly disrupt communication and reduce the reach of information across the network, like cutting the only bridge between two islands.

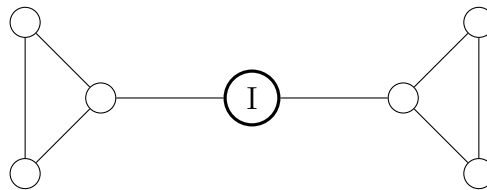


Figure 22: An influential node acting as a bridge between communities

The distinction between central and influential nodes highlights the importance of structural position in social networks. While central nodes contribute to overall connectivity, influential nodes shape how information propagates and how different communities interact. A user might be central without being influential (like a public figure many follow but few engage with), or influential without being central (like a connector who bridges two distinct communities with few followers overall).

This structural perspective motivates the use of formal centrality measures, such as **PageRank**, which quantify importance based on both connectivity and influence, a topic we will explore in the next section [\[6, 8\]](#).

3.6 PageRank and Personalized PageRank in Social Networks

While structural connectivity explains how nodes are positioned within a network, additional measures are needed to evaluate the relative importance of these nodes. Simple connectivity alone cannot fully capture influence, especially in large social networks where the quality of connections matters as much as their quantity. This motivates the use of advanced centrality measures such as PageRank and Personalized PageRank.

PageRank extends traditional centrality measures by evaluating not only the number of connections a node has, but also the importance of the nodes connected to it [6]. In social networks, influence depends on both direct and indirect relationships, making PageRank more effective than simple degree-based measures for identifying structurally important users.

PageRank assigns an importance score to each node based on a simple but powerful principle: connections from important nodes contribute more than connections from less important ones. This creates a recursive definition: a node is considered important if it is connected to other important nodes. This allows PageRank to distinguish between mere popularity (many followers) and true structural influence (being followed by influential users). As shown in Figure 23, PageRank prioritizes the influence of followers rather than their quantity. A user followed by a few highly influential accounts may receive a higher PageRank score than someone followed by thousands of inactive users.

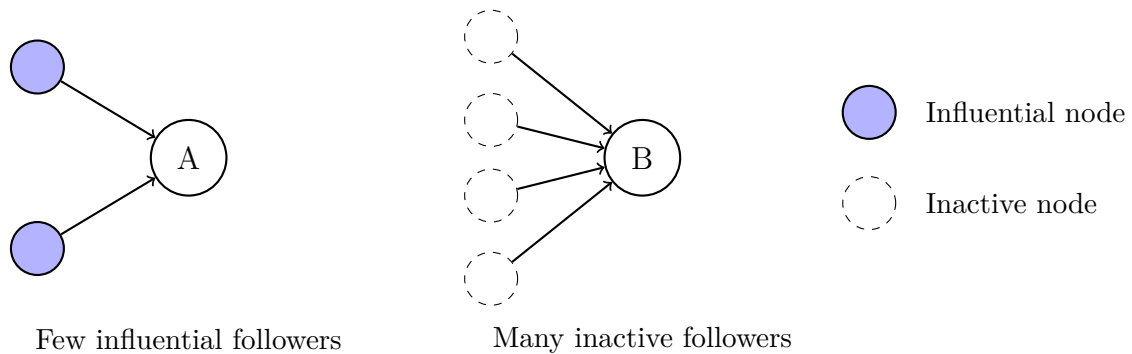


Figure 23: Difference between influential and inactive followers.

The Random Walk Interpretation

The PageRank algorithm can be intuitively understood through the concept of a **random walk**. Imagine a hypothetical user moving through the network by randomly following links from one node to another. The probability of moving to a neighboring node depends on the outgoing connections of the current node.

Over time, this random walker visits certain nodes more frequently than others. These visit frequencies converge to stable values that correspond to the PageRank scores of the nodes. As shown in Figure 24, nodes that are connected to highly connected neighbors are more likely to be visited during the random walk.

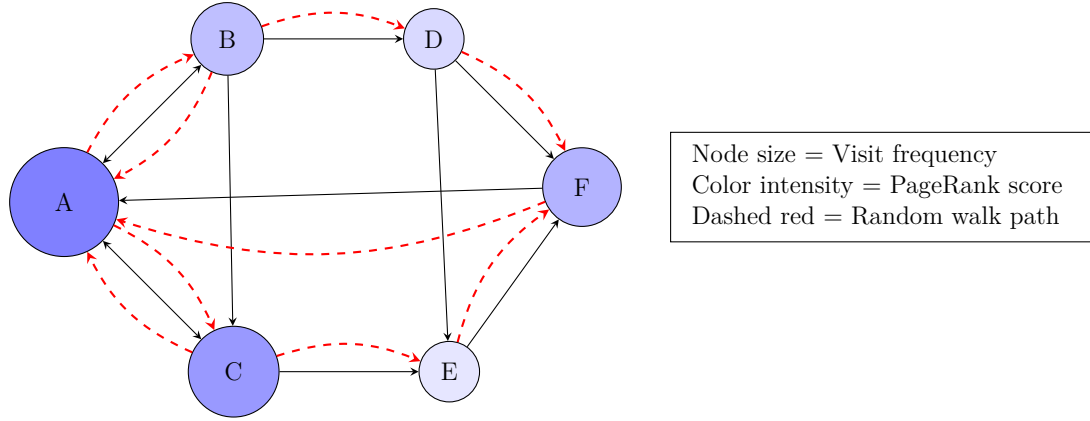


Figure 24: Random walk model of PageRank with size and color indicating importance.

This leads to an important insight a node may achieve a high PageRank not merely by having many incoming links, but by receiving links from nodes that are themselves frequently visited. This reflects the core principle of PageRank importance is propagated through the structure of the network via its links.

To prevent the random walk from becoming trapped within closed or highly connected regions, PageRank introduces a **damping factor**. This factor represents the probability that the random walker stops following links and instead jumps randomly to any other node in the network.

Consider a small group of users who only follow each other and have no connections to the rest of the network. Without a damping factor, a random walker entering this group would remain trapped within it indefinitely. The damping factor allows the walker to occasionally leave such closed groups and explore other parts of the network, ensuring that importance is distributed across the entire structure rather than confined to isolated regions.

In practice, the damping factor is commonly set to **0.85** [6], meaning that 85% of the time the walker follows a link, and 15% of the time they jump to a random node.

As illustrated in Figure 25, without damping (a) the random walker becomes confined within the closed group of nodes. When the damping factor is introduced (b), the dashed arrow represents the possibility of a random jump to an external node, allowing the walker to escape and ensuring global importance distribution.

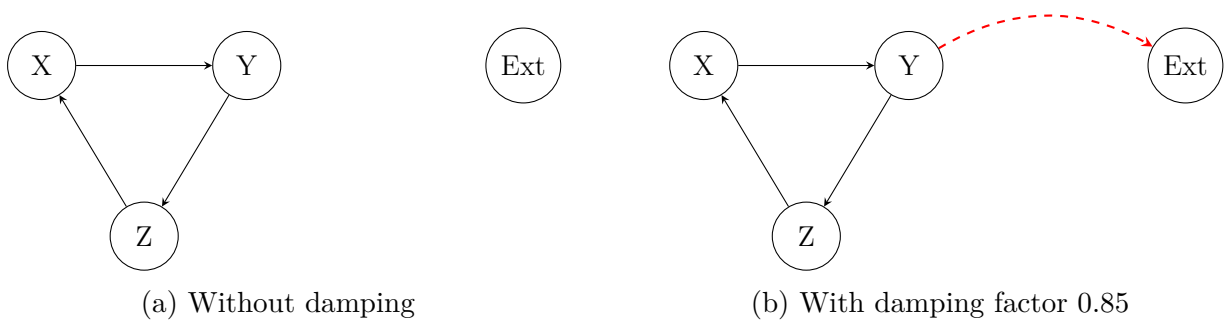


Figure 25: Damping prevents trapping in closed groups in PageRank.

Personalized PageRank

Standard PageRank treats all random jumps as equally likely, any node can be the destination of a jump. However, in many real-world applications, we want to measure importance relative to a specific user or context.

Personalized PageRank addresses this by modifying the random walk process. Instead of allowing random jumps to any node, the walk is biased toward a specific user or a predefined set of users. This causes the random walker to repeatedly return to a local region of the network, producing scores that reflect importance from that user’s perspective.

This distinction is illustrated in [Figure 26](#). In the standard PageRank model (a), random jumps are uniformly distributed across the network, leading to globally determined importance scores. In contrast, the personalized version (b) biases the random walk toward a specific user, causing importance to concentrate within the local neighborhood of that user and reducing the influence of distant nodes.

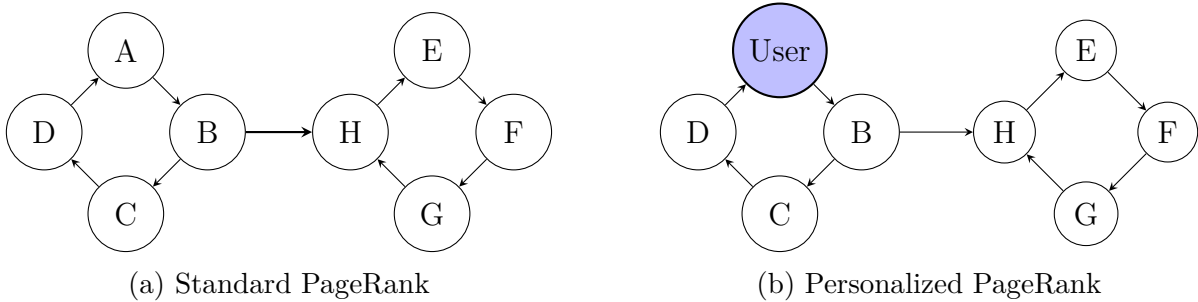


Figure 26: Comparison of Standard and Personalized PageRank.

For example, in a reader-book network, Personalized PageRank can be initialized from a specific reader’s preferences. The random walk starts at the target reader, moves to books they have read, then to other readers who read those books, and finally to new books those readers have enjoyed. The resulting scores will be higher for authors who are closely connected to that reader through shared books or reading patterns even if those authors are not globally popular. This makes Personalized PageRank particularly valuable for recommendation systems, as it captures both direct and indirect preferences through network structure.

Personalized PageRank follows the same iterative process as standard PageRank, with the only difference being the personalization vector that guides the random jump behavior. It enables recommendations to be generated by identifying authors and books that are structurally close and contextually relevant to a target user.

3.7 Personalized PageRank Algorithm

Standard PageRank

The standard PageRank algorithm was originally proposed to rank web pages based on their importance in a network [6]. The equation is defined as:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in B_u} \frac{PR(v)}{L(v)} \quad (1)$$

where:

- $PR(u)$ is the PageRank score of node u
- d is the damping factor, typically set to 0.85
- N is the total number of nodes in the graph
- B_u is the set of all nodes that link directly to node u
- $L(v)$ is the number of outgoing links from node v

Intuition

The standard PageRank algorithm models the behaviour of a random surfer browsing the internet. At each step, the surfer either jumps to a random node with probability $(1-d)$ or follows one of the outgoing edges from the current node with probability d . The PageRank score of a node therefore represents how frequently the random surfer visits that node after many iterations.

Equation Interpretation

Term 1: $\frac{1-d}{N}$

This term represents the random jump probability. The value $(1-d)$ is the probability that the surfer jumps to a random node, while N ensures that all nodes have an equal probability of being selected. This prevents the algorithm from becoming trapped inside disconnected regions and guarantees that every node receives a minimum score.

Term 2: $d \sum_{v \in B_u} \frac{PR(v)}{L(v)}$

This term represents score propagation through graph connections. Each node distributes its PageRank score equally across all outgoing edges. Nodes connected to highly important neighbours therefore receive higher PageRank scores.

Damping Factor $d = 0.85$

The damping factor controls the balance between random jumps and graph traversal. A value of 0.85 means that the surfer follows graph edges 85% of the time and jumps randomly 15% of the time. Higher values place greater importance on graph structure, while lower values encourage more random exploration.

Personalized PageRank

The Personalized PageRank (PPR) algorithm extends standard PageRank by introducing a personalization vector e_s that biases the random walk toward a specific target node [7]. The equation is defined as:

$$PR(u) = (1 - \alpha) \cdot e_s + \alpha \sum_{v \in B_u} \frac{PR(v)}{L(v)} \quad (2)$$

where:

- $PR(u)$ is the Personalized PageRank score of node u
- α is the probability of following graph edges
- e_s is the personalization vector
- B_u is the set of all nodes that link directly to node u
- $L(v)$ is the number of outgoing links from node v

The personalization vector is defined as:

$$e_s(v) = \begin{cases} 1 & \text{if } v = \text{target user} \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

Intuition

The key difference between standard PageRank and Personalized PageRank lies in the restart behaviour. Standard PageRank allows the random surfer to restart from any random node, whereas Personalized PageRank always restarts from the target user node. This causes the resulting scores to reflect personal relevance rather than global popularity.

Equation Interpretation

Term 1: $(1 - \alpha) \cdot e_s$

This term represents the personalized restart mechanism. The value $(1 - \alpha)$ determines the probability that the random surfer restarts the walk, while the personalization vector e_s ensures that the restart always occurs at the target user node. This keeps the recommendation scores anchored to the preferences of the target user.

Term 2: $\alpha \sum_{v \in B_u} \frac{PR(v)}{L(v)}$

This term represents graph propagation. The random walk moves from users to movies they liked, then to other users connected to those movies, and finally to additional movies. Each graph traversal step spreads relevance scores through the network, allowing structurally related movies to accumulate higher recommendation scores.

Restart Probability $(1 - \alpha)$

The restart probability determines how strongly the recommendations remain connected to the target user's preferences. Higher values of α encourage broader graph exploration, while lower values keep the recommendations closer to the user's direct interactions.

How PPR Produces Recommendations

In the implemented bipartite user–movie graph, the random walk follows the path shown below:

$$\text{User} \xrightarrow{\text{liked}} \text{Movie A} \xrightarrow{\text{liked by}} \text{Similar User} \xrightarrow{\text{liked}} \text{Movie B}$$

The recommendation process begins at the target user node and traverses through connected movies and neighbouring users. Movies that accumulate higher Personalized PageRank scores are considered more relevant recommendations. Movies already interacted with by the user are removed, and the remaining top-ranked unseen movies are returned as recommendations.

Key Difference Between Standard and Personalized PageRank

Property	Standard PageRank	Personalized PageRank
Restart destination	Any random node	Target user node
Measures	Global importance	Personal relevance
Use case	Web search ranking	Recommendation systems
Restart term	$\frac{1-d}{N}$	$(1 - \alpha) \cdot e_s$

Table 1: Comparison between Standard PageRank and Personalized PageRank

4 Projecting Monopartite Networks

4.1 Motivation for Monopartite Projection

Most graph algorithms are designed to operate on **monopartite networks**, where all nodes are of the same type and all relationships have a uniform meaning. However, many real-world datasets are heterogeneous, containing multiple node types connected through different kinds of interactions [8]. Applying standard graph algorithms directly to such networks can obscure the true relationships that the analysis aims to capture.

Monopartite projection addresses this issue by simplifying the network while preserving the interaction semantics relevant to the study. By transforming a complex network into a structure containing only one type of node, the projection retains meaningful interaction patterns while removing structural elements that are not essential for the chosen algorithm. This approach enables clearer interpretation of results and ensures that centrality measures reflect genuine relationships rather than artifacts of the data model.

For example, in a dataset containing readers and books, influence or similarity analysis becomes more transparent after projecting the graph into a reader–reader network, as seen in [Figure 10](#). In this projected network, connections represent shared reading preferences rather than having book nodes act as intermediaries.

4.2 Translating Indirect Patterns into Direct Relationships

In many heterogeneous networks, entities of the same type interact indirectly through intermediate nodes. While this modeling approach is accurate, it complicates the measurement of influence or similarity. Consequently, these indirect interaction patterns are translated into direct inferred relationships to enable clearer analysis.

These inferred relationships are not explicitly stored in the original dataset but are derived based on domain knowledge and reasonable assumptions. The goal is to allow influence, similarity, or importance to flow directly between entities of the same type, making the application of graph algorithms more meaningful.

For example, actors are indirectly connected through the movies in which they appear. Although the data model links actors to movies rather than to each other, appearing in the same film clearly implies a professional connection. This indirect actor–movie–actor pattern can be translated into a direct co-acting relationship between actors, enabling clearer analysis of collaboration intensity and prominence within the acting network, as shown in [Figure 27](#).

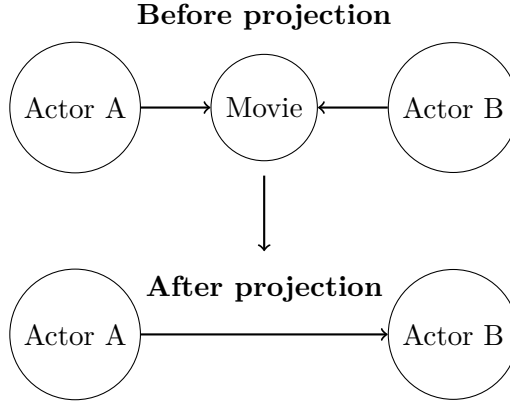


Figure 27: Transforming actor–movie–actor into a direct co-acting relation

When such interactions occur repeatedly, the inferred relationships can be weighted to reflect interaction strength. This preserves quantitative information such as how frequently two actors collaborate without reintroducing structural complexity.

4.3 Projected Network Analysis and Influence Interpretation

After transforming indirect interactions into direct inferred relationships, the network becomes easier to analyze structurally. This projected representation allows standard graph algorithms to operate on a simplified network where influence, similarity, and connectivity can be interpreted more clearly.

Once the network has been projected, it consists of a single node type connected by a single inferred relationship type, potentially enriched with weights. This simplified structure enables clearer structural analysis and more reliable interpretation of influence.

In a projected reader–reader network, for example, connections represent shared reading preferences rather than direct friendships. A reader who shares books with many others will occupy a central position in this network, indicating their role as a bridge between different reading communities. The weight of each connection reflects the number of shared books, capturing the strength of the similarity.

For example, if Ali and Sara have both read several science fiction novels, a connection is created between them in the projected reader–reader network. If they share many books, the edge between them receives a higher weight, indicating stronger similarity in reading preferences.

This structure allows standard graph algorithms to be applied meaningfully.

- **Degree centrality** identifies readers with many shared connections.
- **Weighted PageRank** assigns higher importance to readers connected to other important readers.
- **Community detection** reveals groups of readers with similar interests.

The projected network thus transforms an abstract bipartite structure into a form where influence and similarity can be directly measured.

4.4 Handling Self-loops in Projected Networks

Although projected networks simplify structural analysis, the projection process may also introduce artifacts that do not represent meaningful interactions.

Self-loops, where a node is connected to itself, may appear in projected networks depending on the underlying interaction patterns. While such structures can be valid representations of certain behaviors, they often distort influence measurements.

Consider a travel network where cities are connected based on joint appearances in travel itineraries. If a tourist visits Paris multiple times, the projection might create a self-loop from Paris back to itself. This self-loop represents self-endorsement Paris endorsing itself rather than recognition from other cities. When we calculate influence measures, this self-loop artificially inflates Paris’s importance without providing information about its relationships with other destinations.

The problem becomes even clearer in a reader–book network: if a reader reads multiple books by the same author, a projection might create a self-loop from that reader to themselves. This does not indicate influence or similarity with other readers.

For this reason, self-loops are commonly excluded before applying importance-ranking algorithms. By removing self-loops, as illustrated in [Figure 28](#), the analysis focuses only on meaningful relationships between different entities.

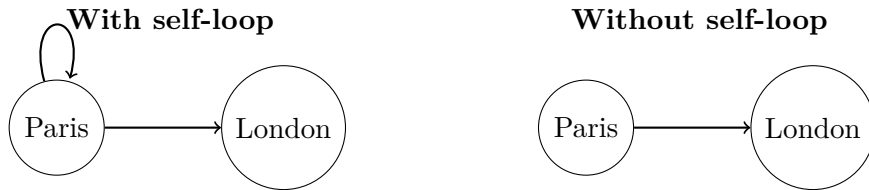


Figure 28: Effect of removing self-loops in a travel network

4.5 Weighted Influence Measures in Cleaned Projected Networks

After removing self-loops, the projected network contains only meaningful relationships between distinct entities. However, not all relationships are equal some connections represent stronger ties than others.

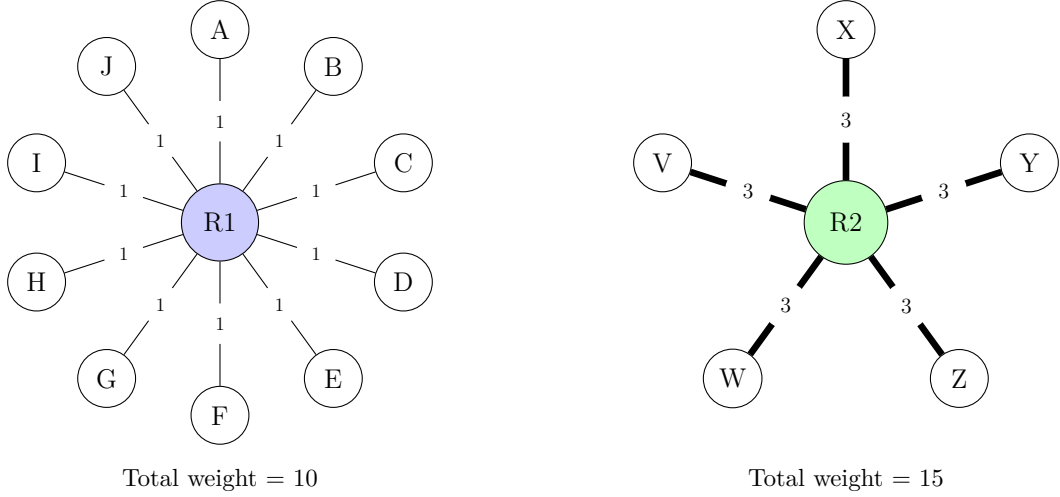
Weighted influence measures address this by accounting for both: number of connections (degree) and the strength of those connections (weight).

In a reader–reader network, the weight between two readers represents how many books they share. This distinction between connection quantity and connection strength is visually illustrated in [Figure 29](#).

As shown in part (a) of [Figure 29](#), Reader R1 is connected to ten other readers, but each connection has a weight of only 1 (represented by thin lines), giving R1 a total weight of 10. In contrast, part (b) shows Reader R2 with only five connections, but each

connection has a weight of 3 (represented by thick lines), resulting in a total weight of 15.

Although R2 has fewer connections than R1, the stronger relationships give R2 a higher total weight. This example demonstrates why weighted influence measures are essential: they account for both the number of connections and their strength. A recommendation system using weighted PageRank would prioritize authors connected to R2 through these strong relationships, even if R2 has fewer overall connections.



(a) Many weak connections

(b) Few strong connections

Figure 29: Readers with many weak versus fewer strong connections.

This distinction is crucial for accurate influence measurement. Standard PageRank treats all connections equally, while weighted PageRank propagates importance proportionally to connection strength. A recommendation system using weighted measures will therefore prioritize authors who are strongly connected to a target reader through multiple shared books, rather than those with many weak connections.

The combination of three key elements monopartite projection to simplify complex networks, self-loop removal to eliminate distorting artifacts, and weighted relationships to capture interaction strength creates a network structure that accurately reflects real-world influence dynamics while remaining suitable for standard graph algorithms. In recommendation systems, weighted PageRank can then prioritize items that are strongly connected to a target user through multiple shared interactions, rather than items with many weak connections.

5 Inferring Co-occurrence Networks from Bipartite Graphs

5.1 From Projection to Co-occurrence

After projecting bipartite graphs into monopartite structures, we need a principled way to infer the strength of relationships between nodes of the same type [2]. Co-occurrence provides a practical foundation for this task: two entities are considered related if they repeatedly appear together within a shared context. The underlying assumption is that frequent joint appearance reflects a meaningful association, even when no explicit relationship exists in the original data [1].

For example, in an actor–movie network, two actors who appear together in multiple films have a demonstrable professional connection. The more films they share, the stronger we expect their collaborative relationship to be.

The Problem with Raw Co-occurrence

However, relying solely on raw co-occurrence frequency can be misleading. Entities that appear very frequently such as highly popular actors tend to co-occur with many others simply due to their prevalence. This popularity effect can inflate the perceived strength of relationships.

Consider a famous actor who has appeared in 50 films. They will naturally co-occur with many co-stars, but this does not necessarily indicate a strong connection to each of them. Raw co-occurrence counts would incorrectly suggest that all these relationships are equally meaningful, when in reality many are incidental.

As a result, raw co-occurrence alone is not sufficient for accurately capturing similarity.

5.2 Normalization with Jaccard Similarity

To reduce the bias introduced by raw frequency, co-occurrence relationships are normalized using similarity measures such as the Jaccard similarity coefficient [16]:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

The numerator $|A \cap B|$ represents the number of shared contexts between the two entities for example, the number of films in which both actors have appeared together. This captures the absolute level of co-occurrence between them.

The denominator $|A \cup B|$ represents the total number of unique contexts in which either entity appears—the complete set of films involving actor A, actor B, or both. This accounts for the individual frequency of each entity.

By taking the ratio of shared contexts to total contexts, the Jaccard coefficient measures the proportion of overlap rather than absolute co-occurrence. A high value indicates that most of their appearances are shared, reflecting genuine similarity. A low

value, even when both entities appear frequently, indicates that their co-occurrence is incidental rather than meaningful.

This normalization ensures that inferred relationships reflect proportional similarity rather than general popularity, resulting in a more balanced and meaningful network structure.

Visualizing Jaccard Similarity

Figure 30 illustrates the difference between raw co-occurrence and normalized similarity using an actor–movie network.

High similarity example (top): Actors A and B share five films out of six possible. Their Jaccard similarity is calculated as:

$$J(A, B) = \frac{5}{6 + 6 - 5} = \frac{5}{7} \approx 0.71$$

This high value (0.71) reflects that most of their film appearances overlap, indicating a strong collaborative relationship.

Low similarity example (bottom): Actors C and D share only one film, even though each has appeared in multiple films. Their Jaccard similarity is:

$$J(C, D) = \frac{1}{6 + 6 - 1} = \frac{1}{11} \approx 0.09$$

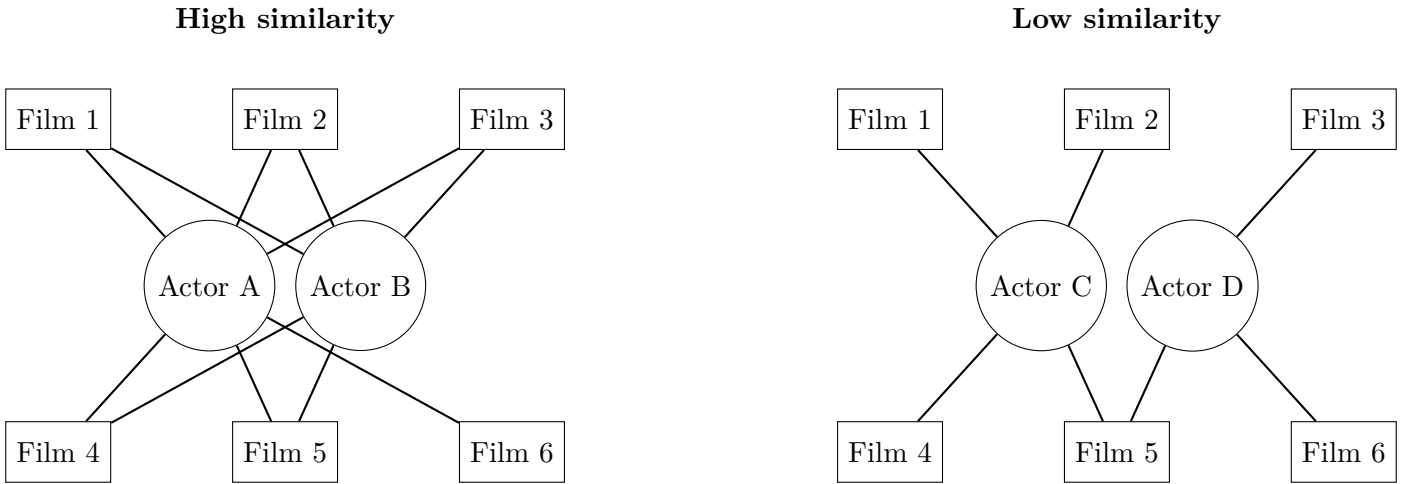


Figure 30: Illustration of Jaccard similarity using actor–movie relationships

Despite both actors being popular (many individual appearances), their proportional overlap is small, correctly indicating a weak connection.

This comparison demonstrates why normalized measures are essential: they distinguish between genuine similarity (high proportion of shared work) and co-occurrence driven by individual popularity (many appearances but little overlap).

Controlling Network Structure with Thresholds

Once the normalized co-occurrence network is constructed, its structure can be controlled by adjusting parameters such as similarity thresholds and connection limits.

Similarity thresholds allow us to filter out weak connections by setting a minimum similarity value. For instance, with a threshold of 0.3, only relationships achieving a Jaccard similarity of 0.3 or higher are retained in the network. This removes incidental connections such as the weak relationship between actors C and D with a similarity of only 0.09 resulting in a sparser but more meaningful network structure.

Connection limits provide another means of control by restricting each node to a maximum number of connections. For example, applying a "top-5 neighbors" policy means each actor connects only to their five most similar collaborators. This approach prevents highly popular nodes from dominating the network and ensures that the resulting structure remains interpretable and balanced.

The choice of threshold values significantly impacts the resulting network:

- **Low threshold (e.g., 0.1):** Produces a denser network that includes many weak ties and captures broader associations.
- **Medium threshold (e.g., 0.3):** Creates a balanced network that captures meaningful collaborations while filtering out incidental connections.
- **High threshold (e.g., 0.6):** Generates a very sparse network that highlights only the strongest relationships.

These parameter choices ultimately influence whether the network emphasizes only the strongest associations or captures broader contextual groupings, allowing analysts to tailor the network structure to their specific analytical goals.

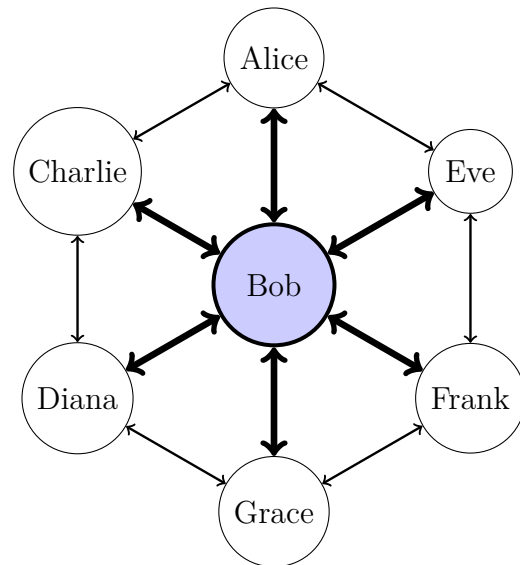
5.3 Community Detection and Representative Nodes

With a normalized co-occurrence network in place, community detection algorithms can identify clusters of closely related entities. These clusters often correspond to shared themes, genres, or contexts. For example, in an actor network, communities might represent action movie actors who frequently collaborate, comedic actors from the same era or studio, or character actors associated with specific directors.

Within each community, representative nodes can be identified using several approaches:

- **Degree centrality:** selects nodes with the largest number of connections within the community.
- **PageRank:** identifies nodes that are structurally important and connected to other influential members.
- **Community centroid similarity:** selects nodes with the highest average similarity to all other members in the community.

Figure 31 shows a character co-occurrence network where Bob emerges as a central representative. His position is characterized by stronger connections indicated by thicker edges and a central structural placement that makes him a hub within the group.



Node size: structural importance
Edge thickness: interaction strength

Figure 31: Character co-occurrence network with a central representative node.

Identifying such nodes provides valuable insight into the internal structure and dominant elements of each community. In a recommendation context, these representative nodes can serve as prototypical examples of the community’s interests, as starting points for generating personalized recommendations, or as category labels for understanding and labeling community themes.

This pipeline from bipartite projection through Jaccard normalization to community detection transforms raw relational data into a structured network ready for further analysis with standard graph algorithms, including the Personalized PageRank approach detailed in [subsection 3.6](#).

6 Similarity-Based Network Construction from Node Features

Traditional network analysis relies on explicit relationships such as friendships, purchases, ratings, or interactions between nodes. However, many meaningful relationships exist even when no direct interaction is observed. In many real-world systems, users may behave similarly despite never communicating or interacting explicitly. Similarity-based network construction addresses this limitation by building connections from behavioral resemblance rather than direct links [16].

Instead of relying on explicit links between nodes, similarity-based network construction represents each node using a set of features that capture its behavior and structural role within the network. These features describe how a node participates in the system rather than whom it is directly connected to. This representation makes it possible to identify meaningful relationships even when no direct interactions exist between nodes.

Consider two users on a content platform. The first user spends 80% of their time reading about machine learning and 20% about data visualization. The second user spends 75% of their time on machine learning and 25% on data visualization. Although these users have no direct interaction, their behavioral patterns are nearly identical. This similarity suggests a meaningful relationship that traditional interaction-based networks would fail to capture.

Comparison with Traditional Networks

The distinction between traditional interaction-based networks and similarity-based networks can be summarized through several structural and analytical differences. Table 2 compares the two approaches in terms of connection formation, required information, and the types of relationships they reveal.

Table 2: Comparison between Traditional and Similarity-based Networks

Aspect	Traditional Network	Similarity-based Network
Connection basis	Direct interaction	Behavioral similarity
Requires explicit links	Yes	No
Example	Follow, purchase, friendship	Similar interests or activity patterns
Hidden relationships	Limited	Captured through similarity

Traditional networks connect nodes only when there is explicit evidence of interaction such as a friendship, a purchase, or a rating. This approach captures observable relationships but often misses hidden behavioral similarities. For example, two customers who have never purchased the same product may still exhibit highly similar browsing behavior. A traditional network would show no connection between them, while a similarity-based network would reveal their behavioral resemblance and support recommendation systems based on shared interests rather than shared purchases.

Representing Nodes Using Feature Vectors

To measure behavioral similarity, each node is represented as a feature vector containing numerical values that describe its characteristics. These features may include content preferences, posting frequency, engagement behavior, or activity patterns over time. Representing nodes as vectors transforms behavioral information into a mathematical form suitable for similarity analysis.

Computing Similarity Between Nodes

Similarity between nodes is calculated by comparing their feature vectors. Different similarity measures capture different interpretations of similarity depending on the analysis objective.

Different similarity measures capture different aspects of node behavior and feature relationships. Some methods focus on proportional similarity, while others emphasize absolute distance or synchronized behavioral patterns. The following measures are among the most commonly used approaches for comparing feature vectors in similarity-based network construction [18].

- **Cosine Similarity** measures the angle between two vectors and focuses on similarity in behavioral proportions rather than activity magnitude. Users with similar interest distributions receive high similarity scores even if one user is more active overall.
- **Euclidean Distance** measures the straight-line distance between vectors in multi-dimensional space. Smaller distances indicate stronger similarity, while larger distances reflect greater behavioral differences.
- **Pearson Correlation** measures whether behavioral patterns rise and fall together over time. It captures synchronized trends such as users becoming active during similar periods or showing interest in the same topics simultaneously.

Each similarity measure provides a different perspective on node behavior, and the appropriate choice depends on the analytical objective and the type of relationships being studied.

Constructing the Similarity Network

Once similarity scores are computed, a new inferred network can be constructed in which edges represent behavioral similarity rather than direct interaction. The construction process consists of four main steps:

1. Extract feature vectors from node behavior.
2. Compute pairwise similarity scores between nodes.
3. Apply a similarity threshold to filter weak relationships.
4. Create edges between sufficiently similar nodes.

The choice of similarity threshold significantly influences the resulting network structure. A low threshold creates a dense network containing many weak similarities and broader behavioral associations. A high threshold produces a sparse network containing only the strongest and most meaningful relationships. In practice, multiple threshold values are often tested to identify the structure that best supports the analysis objectives.

Community Detection in Similarity Networks

Applying community detection to the inferred similarity network groups nodes that exhibit consistent behavioral or feature patterns. Unlike traditional communities formed through direct interaction, these communities represent functional or behavioral similarity. As illustrated in Figure 32, Users A, B, and C form one similarity-based community, while Users D and E form another.

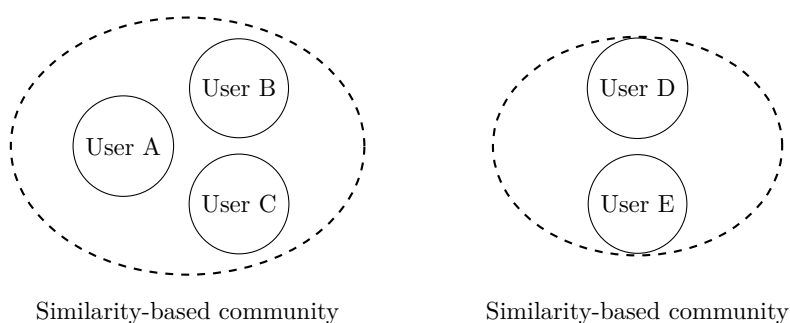


Figure 32: Feature-based similarity communities

On a content platform, several users may consistently engage with similar types of content such as machine learning articles, cooking videos, or sports highlights. Even without direct interaction, their behavioral similarity causes them to be grouped into the same community.

Representative Nodes within Communities

Within each similarity-based community, representative nodes can be identified to summarize the dominant characteristics of the group. Several approaches may be used for this purpose:

- **Degree centrality:** selects nodes with the largest number of similarity connections within the community.
- **PageRank:** identifies structurally important nodes connected to other influential members.
- **Centroid similarity:** selects nodes whose feature vectors are closest to the average behavior of the community.

These representative nodes provide a concise summary of the behavioral patterns shared by the community and support downstream analytical tasks such as recommendation and user profiling.

Practical Applications

Similarity-based networks support several important analytical applications:

- **Recommendation systems:** Users with highly similar behavioral patterns can be connected even when they have no direct interaction history. This allows recommendation systems to suggest content, products, or services based on inferred preferences and shared interests.
- **User segmentation:** Similarity networks can group users into behavioral communities based on activity patterns, interests, or engagement habits. These segments support targeted marketing and personalized user experiences.
- **Anomaly detection:** Users whose behavior differs significantly from their surrounding similarity-based community may represent unusual or suspicious activity. This approach is commonly used in fraud detection and spam identification.
- **Customer profiling:** Feature similarity helps construct detailed behavioral profiles that reflect genuine user preferences. These profiles improve predictive modeling, personalization, and long-term user analysis.

By shifting the focus from explicit interactions to behavioral similarity, similarity-based network construction reveals hidden relationships and provides a richer understanding of how entities within a system are related.

7 Node Embeddings and Node Classification

Graph algorithms are primarily designed to analyze the structural properties of a network by examining connectivity, paths, and node importance. These methods operate directly on the graph topology to reveal patterns such as central nodes, communities, or influential connections. Although such algorithms provide valuable descriptive information, their output is often limited to the current structure of the network.

Graph-native machine learning extends this by enabling prediction over graph-structured data. Instead of manually selecting structural features, these approaches learn representations directly from the graph. Node embeddings play a central role in this transition by mapping nodes into low-dimensional vectors that preserve structural and relational information, enabling tasks such as node classification, similarity analysis, and link prediction [13].

The evolution of machine learning on graph-structured data can be viewed as a progression from traditional tabular learning toward models that directly operate on graph topology. Figure 33 illustrates the main stages in this evolution.

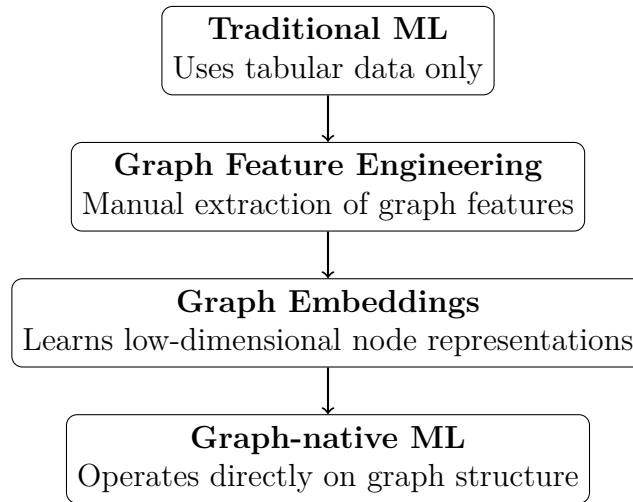


Figure 33: Evolution of approaches for learning from graph-structured data

Understanding where node embeddings fit in this landscape provides important context for why they are effective and what more advanced alternatives exist.

7.1 Foundations of Node Embeddings

Graphs provide a natural way to represent relational data, where nodes describe entities and edges capture interactions between them. Although this representation is expressive, it poses challenges when graph structure needs to be integrated into machine learning models. A direct encoding of a graph, such as an adjacency matrix, results in very high-dimensional representations that grow with the number of nodes. As networks become large, these representations become inefficient, sensitive to changes in graph size, and prone to overfitting.

The Adjacency Matrix and Its Limitations

To construct the adjacency matrix, the graph is first represented in a numerical form where each row and column corresponds to a node in the network. An entry in the matrix indicates whether a direct connection exists between a pair of nodes.

If two nodes are connected by an edge, the corresponding matrix entry is set to one; otherwise, it is set to zero. In undirected graphs, this representation is symmetric because connections are mutual.

Although the adjacency matrix provides a complete description of the network structure, its dimensionality grows rapidly with the number of nodes because each node requires a separate row and column in the matrix. For a network with n nodes, the adjacency matrix has size $n \times n$. As networks become large, this representation becomes memory-intensive, computationally inefficient, and prone to overfitting, making it impractical for large-scale learning tasks.

How Node Embeddings Solve This Problem

Node embeddings address this limitation by transforming graph structure into compact numerical representations. Each node is represented as a low-dimensional vector that preserves essential structural information about its position within the network. Rather than manually designing features, these representations are learned automatically in an unsupervised manner, allowing the model to capture patterns that emerge from connectivity alone.

The transformation from graph structure to node embeddings typically follows several stages:

1. The graph is first represented using an adjacency matrix that encodes node connectivity.
2. This matrix provides a high-dimensional numerical description of the network structure.
3. A node embedding algorithm then learns a mapping from this high-dimensional space into a compact low-dimensional vector space.
4. Each resulting vector summarizes the structural context of a node within the network.

As illustrated in [Figure 34](#), a simple graph with four nodes is first encoded as a four-by-four adjacency matrix representing pairwise connections. The embedding process then transforms this representation into a low-dimensional matrix with four rows and two columns, where each row corresponds to a node and each column represents a learned latent feature. Nodes that are directly connected or structurally similar tend to obtain similar embedding vectors despite the higher dimensionality of the original representation.

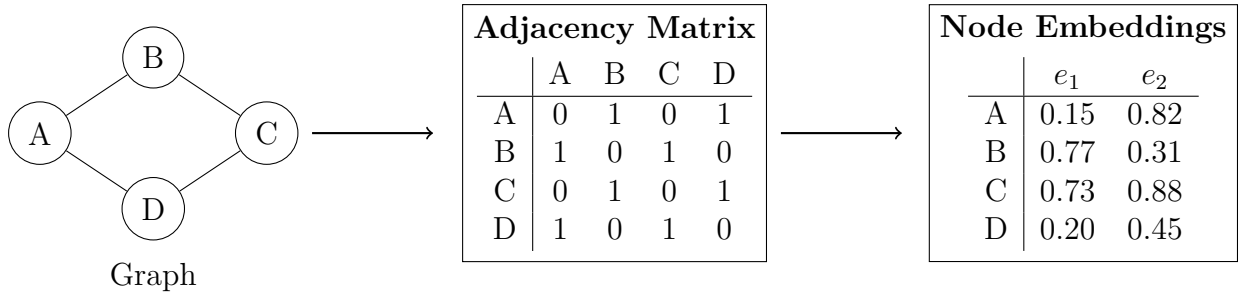


Figure 34: From graph structure to adjacency matrix and node embeddings.

Similarity Perspectives in Node Embeddings

The information captured by node embeddings depends on how similarity is defined within the network. In many real-world graphs, connected nodes tend to share similar properties. This assumption, often referred to as homophily, implies that nodes which interact frequently or belong to the same region of the network are likely to behave alike. Embedding models designed under this assumption position neighboring nodes close together in the embedding space, naturally revealing communities and clusters.

For example, in a social network, people who are friends tend to have similar interests. If we embed this network, friends would end up with similar vector representations. A user who follows many sports accounts and their friend who follows similar accounts would have nearly identical embedding vectors, even if they never interact directly.

Similarity in networks is not always determined by direct proximity between nodes. In some cases, nodes may occupy similar structural roles even when they are located in different parts of the graph. For example, certain nodes may consistently act as bridges between groups or serve as intermediaries for information flow.

For example, two employees in different departments may both act as bridges between their teams and upper management. Although they are not directly connected, they perform similar structural roles within the organization.

Although both approaches measure node similarity, they capture different structural perspectives. [Table 3](#) summarizes the key differences between homophily and structural role similarity.

Table 3: Homophily vs. structural role similarity

Aspect	Homophily	Structural Roles
Similarity basis	Nodes connect to similar neighbors	Nodes perform similar functions in the network
Distance in graph	Usually nearby	Can be far apart
Example	Friends with shared interests	Bridge nodes connecting communities
Focus	Local similarity	Functional equivalence
Applications	Recommendation systems	Infrastructure or role analysis

Figure 35 illustrates the difference between homophily-based similarity, where neighboring nodes obtain similar embeddings, and structural role similarity, where distant nodes with comparable functions receive similar representations.



Figure 35: Homophily vs. structural roles.

The choice between these perspectives depends on the analytical objective. Tasks such as preference prediction or recommendation benefit from embeddings that emphasize local similarity, whereas role analysis and infrastructure modeling require embeddings that capture functional equivalence.

7.2 Learning Embeddings Through Random Walks

A common strategy for learning node embeddings relies on the idea of random walks [12]. A random walk simulates a traversal through the network by moving from one node to a neighboring node at each step. Over time, this process generates sequences of nodes that reflect the local and global structure of the graph.

How Random Walks Work

As shown in Figure 36, a random walk begins at a starting node and proceeds by randomly selecting one of its neighbors to move to. From that neighbor, it again randomly selects one of its neighbors, and so on. This process creates a sequence of nodes that captures the local neighborhood around the starting point.

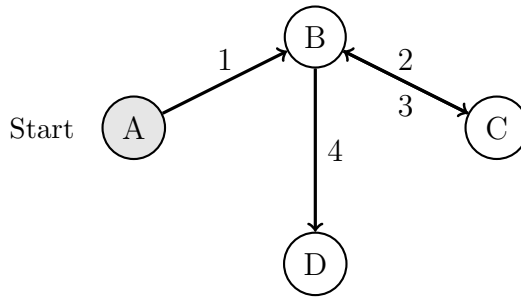


Figure 36: Random walk capturing local graph structure.

For example, imagine a small social network. Starting from a user named Zainab, a random walk might go: Zainab $\rightarrow$ her friend Ali $\rightarrow$ Ali's colleague Ahmed $\rightarrow$ Ahmed's roommate Abdullah. This sequence suggests that Zainab and Abdullah might be connected through their mutual acquaintances, even if they have never met.

By generating thousands of such walks from different starting points, we collect a large corpus of node sequences. These sequences are then fed into a model similar to

Word2Vec, which learns to predict a node given its surrounding nodes in the walk. The result is a set of embeddings where nodes that frequently appear together in walks have similar vector representations.

These node sequences can be interpreted in a way analogous to sentences in natural language. Nodes that frequently appear within similar walk contexts are treated as structurally related, allowing techniques originally developed for language modeling to be applied to graphs. By adjusting how the random walk explores the network, the embedding process can emphasize either local neighborhoods or broader structural patterns.

From Words to Nodes: The Word2Vec Inspiration

Before understanding node2vec, it helps to understand where its core idea came from. Word2Vec is a technique for learning word embeddings from text. It works on a simple principle: a word is known by the company it keeps. Words that appear surrounded by similar words in sentences tend to have similar meanings and thus end up with similar vector representations.

For example, the words “phone” and “laptop” appear in similar contexts, so Word2Vec assigns them similar embeddings. This concept is illustrated in [Figure 37](#).

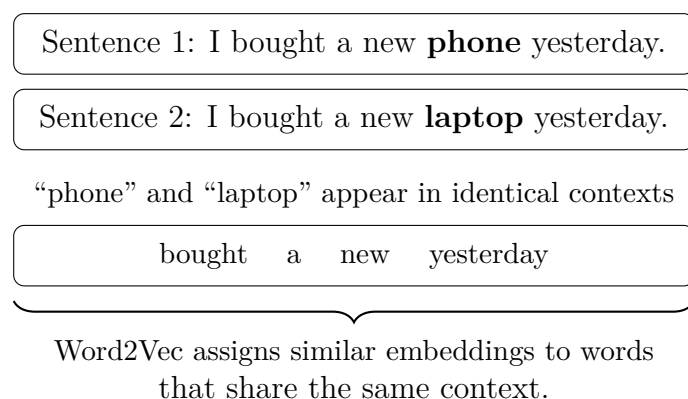


Figure 37: Word2Vec intuition: similar contexts lead to similar embeddings.

Node2Vec adapts this idea to graphs. Instead of sentences, it uses random walks. Instead of words, it uses nodes. The principle becomes: a node is known by the nodes it visits with during random walks. Nodes that appear together in many random walks are considered structurally related and receive similar embeddings.

Controlling the Walk with Node2Vec Parameters

Although random walks already capture useful structural information, different exploration patterns can reveal different types of relationships between nodes. In some applications, we may want the walk to remain close to local neighborhoods and communities, while in others we may prefer the walk to explore distant regions of the graph and capture broader structural patterns. To provide this flexibility, **node2vec** introduces two parameters that control how the random walk navigates through the network [13].

The node2vec algorithm introduces two parameters, p and q , to control how the random walk explores the graph. These parameters influence which nodes are likely to appear together in the generated walks, and therefore determine what kind of similarity the learned embeddings will capture.

The **return parameter** (p) controls the likelihood of immediately revisiting the previous node.

- **Low p :** The walk is more likely to return to recently visited nodes, causing it to remain within the same local neighborhood. This emphasizes tightly connected communities and local similarity.
- **High p :** The walk is less likely to backtrack and instead continues exploring new nodes. This encourages broader exploration of the graph.

The **in-out parameter** (q) controls whether the walk stays near the starting node or explores more distant regions of the graph.

- **Low q :** The walk explores outward to more distant nodes, helping capture structural role similarity between nodes that may not be directly connected.
- **High q :** The walk stays closer to the local neighborhood, emphasizing homophily and community-based similarity.

Together, these parameters allow node2vec to balance between local community exploration and broader structural exploration [13].

The two exploration strategies used in node2vec are often compared to Breadth-First Search (BFS) and Depth-First Search (DFS), which are common graph traversal methods.

- **Breadth-First Search (BFS):** explores nodes that are close to the starting node before moving farther away. This keeps the walk within the same local neighborhood and emphasizes community structure and homophily-based similarity.
- **Depth-First Search (DFS):** moves deeper into the graph by exploring distant nodes before returning. This allows the walk to discover nodes that occupy similar structural roles even when they are far apart in the network.

In node2vec, different combinations of the parameters p and q make the random walk behave more like either BFS or DFS, depending on whether the objective is to capture local community similarity or broader structural role similarity.

Figure 38 compares two different random walk behaviors produced by node2vec parameter settings.

In Figure 38 (a), the walk remains close to the starting node S and repeatedly visits nearby nodes such as A and B . This behavior resembles Breadth-First Search (BFS), where exploration stays within the local neighborhood. As a result, nodes belonging to the same community frequently appear together in walks, emphasizing homophily and local similarity.

In Figure 38 (b), the walk moves progressively farther away from the starting node, visiting nodes such as C , E , and F . This behavior resembles Depth-First Search (DFS), where the walk explores more distant regions of the graph. Such exploration helps capture structural role similarity between nodes that may be far apart but perform similar functions within the network.

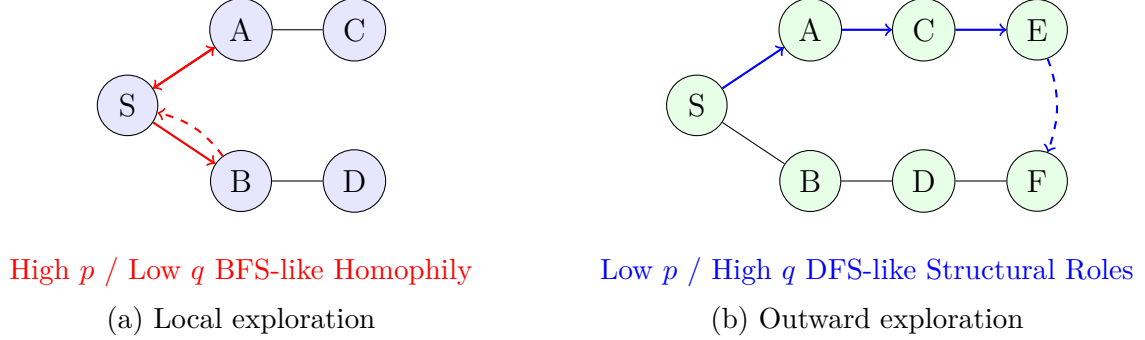


Figure 38: Effect of node2vec parameters p and q on random walk behavior.

By adjusting the parameters p and q , node2vec can balance between local neighborhood exploration and broader structural exploration. This flexibility allows the learned embeddings to capture either homophily-based similarity between nearby nodes or structural role similarity between nodes that occupy comparable functions within different parts of the graph.

7.3 From Embeddings to Node Classification

Once node embeddings are obtained, they can be used as input features for predictive tasks. Node classification aims to infer a node's label based on its structural position in the network rather than relying solely on intrinsic attributes. By encoding each node as a numerical vector, standard machine learning models can be applied without requiring direct access to the graph.

Imagine we have a social network of users on a streaming platform. Some users are labeled as "sports fans" and others as "movie buffs." We want to predict the interest of new users based on who they connect with.

After generating embeddings for all users, each user is represented by a vector of numbers say, 128 numbers per user. These vectors capture the structural position of each user in the network. Users who follow similar accounts and are followed by similar people will have similar vectors.

We can now take the labeled users, split them into training and test sets, and train a classifier such as logistic regression or random forest using the embedding vectors as features. The classifier learns patterns that associate certain embedding structures with specific labels. For example, users whose embeddings are similar to known sports fans

are more likely to be classified as sports fans themselves.

When a new user joins the network, we first generate an embedding based on their connections. The trained classifier then uses this embedding to predict the user's interests or category.

The Complete Workflow

A typical workflow for node classification involves generating node embeddings, training a classifier, and evaluating predictive performance. Figure 39 summarizes the main stages of this process.

1. **Input Graph:** The process begins with a graph where nodes represent entities and edges represent relationships between them.
2. **Generate Embeddings:** A method such as node2vec is applied to transform each node into a low-dimensional embedding vector that captures structural information from the graph.
3. **Train/Test Split:** The generated embeddings are divided into training and test sets. This separation helps evaluate the model on unseen data and prevents data leakage.
4. **Train Classifier:** Machine learning models such as logistic regression, random forest, or neural networks are trained using the embedding vectors as input features.
5. **Evaluate Performance:** The trained model is evaluated using metrics such as accuracy, precision, recall, and F1-score. For imbalanced datasets, precision and recall are often more informative than accuracy alone.
6. **Predict New Nodes:** After evaluation, the trained classifier can predict the labels or categories of previously unseen nodes based on their embeddings.
7. **Feedback Loop:** The workflow can be refined by adjusting embedding parameters, classifier settings, or training strategies to improve prediction performance.

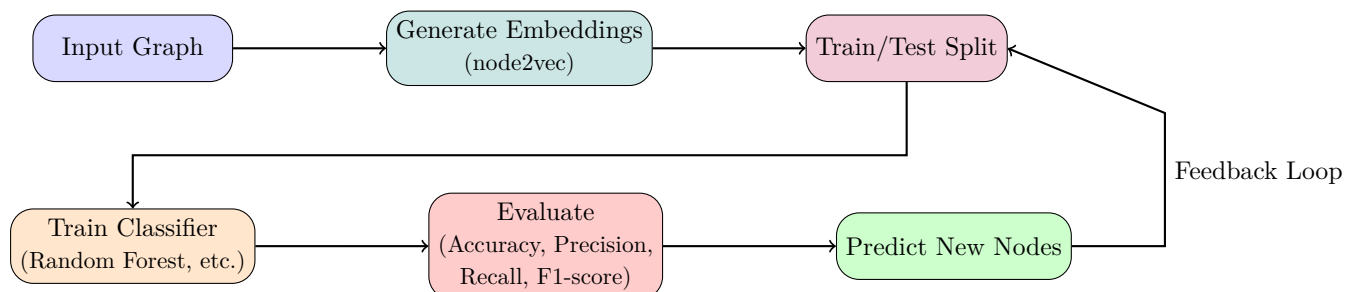


Figure 39: Complete node classification workflow using graph embeddings.

This approach highlights the strength of node embeddings: they translate relational information into a form that can be directly exploited by predictive models. Instead of

manually engineering features like "number of sports-related connections" or "density of local neighborhood," the embedding learns these patterns automatically from the network structure.

If the network structure is predictive of the labels we care about and in many real-world networks, then node embeddings provide a powerful mechanism for leveraging structural information in predictive tasks. A user's connections reveal their interests. A paper's citations reveal its field. A protein's interactions reveal its function.

7.4 Considerations for Isolated Nodes

While node embeddings are effective for capturing structural information, some challenges arise when certain nodes have very few or no connections. These isolated nodes require special consideration because embedding methods rely heavily on graph structure.

Not all nodes contribute equally to the embedding process. Nodes without connections lack structural context and are often encoded identically by embedding models. In `node2vec`, for example, an isolated node never appears in any random walk, so its embedding may remain uninformative, either staying close to its initial random value or collapsing into a near-zero representation.

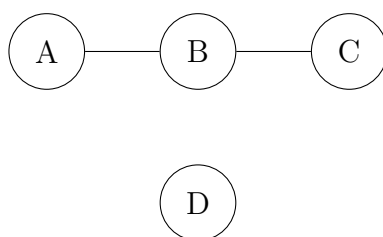


Figure 40: Example of an isolated node in a graph. Node D has no connections, so embedding methods cannot learn meaningful structural information for it.

Whether such nodes should be included depends on the task at hand. If isolation itself carries meaningful information, it may serve as a useful signal. For example, in a fraud detection network, completely isolated accounts might be suspicious and worth flagging. In this case, including them in the classification task could improve results.

However, in most recommendation scenarios, isolated users provide no signal about their preferences. Including them can introduce noise and reduce classification performance. A user who has rated no movies gives us no information about their tastes, and any prediction we make for them is essentially random. It is often better to exclude such users from training and only make predictions once they have accumulated some interaction history.

Careful preprocessing and interpretation are therefore essential when applying node embeddings to real-world networks. Understanding what information embeddings capture and what they miss helps in designing effective predictive systems.

8 Link Prediction in Networks

In graph data science, link prediction is commonly treated as a topological inference task, where the goal is to estimate the likelihood of a relationship forming based on the structural properties of the network [11]. Rather than relying on node attributes alone, topological link prediction exploits patterns such as shared neighbors, structural proximity, and connectivity structure to infer missing or potential links.

This perspective highlights network topology as a key predictive signal. By analyzing structural relationships between nodes, link prediction supports applications such as recommendation systems, network completion, and the discovery of hidden relationships.

After learning meaningful representations of nodes from network structure, the next natural step is to use this structural information to infer missing or potential relationships. Link prediction focuses on estimating the likelihood of connections forming between pairs of nodes based on their proximity, shared neighborhoods, or learned similarity within the graph.

In graph-based systems, the existence or absence of links carries important semantic meaning. Predicting such links enables the discovery of hidden interactions, the anticipation of future connections, and the completion of partially observed networks. As a result, link prediction plays a key role in recommendation systems and network analysis, where relevance often depends on inferred rather than explicit relationships.

For example, in a social network, two users who share many mutual friends but are not directly connected may have a high probability of forming a future connection. Similarly, in recommendation systems, a user and a movie may be linked if users with similar preferences interacted with that movie previously.

8.1 Motivation and Concept of Link Prediction

Network representations are often incomplete reflections of real-world systems. Relationships may be missing because they have not yet occurred, because data collection is limited, or because meaningful interactions exist but have not been explicitly recorded in the dataset.

Link prediction addresses this challenge by estimating whether a relationship exists or is likely to form between two nodes based on the current structure of the network.

The fundamental assumption behind link prediction is that network topology encodes relational intent. Nodes that occupy similar structural positions or appear within overlapping regions of the graph tend to be related, even if no direct connection is currently present. This assumption allows the network itself to serve as a predictive signal.

For example, nodes that share many neighbors or belong to the same community are often more likely to form future connections.

For instance in an academic collaboration network, two researchers may not have co-

authored a paper together. However, both collaborate with the same group of authors, publish in similar venues, and belong to overlapping research communities. Even without a direct edge, the network suggests a latent collaboration. Link prediction attempts to uncover such hidden relationships by analyzing existing connectivity patterns.

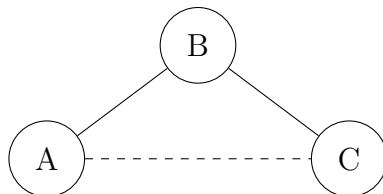


Figure 41: Example of link prediction. Nodes A and C are not directly connected, but their shared neighbor B suggests a potential future link.

By shifting the focus from observed links to potential ones, link prediction transforms networks from static structures into dynamic systems capable of anticipating future interactions.

8.2 Structural Evidence Used to Infer Missing Links

The likelihood of a link between two nodes is largely determined by their structural proximity within the graph. Nodes that are close in the network either through shared neighbors or through multiple short paths tend to have a higher probability of forming direct connections.

One of the most intuitive structural signals is **neighborhood overlap**, where two nodes share a significant number of common neighbors. As illustrated in Figure 42, nodes A and E are not directly connected; however, they both connect to the same intermediate nodes. This shared neighborhood places them within a similar local context, providing structural evidence that a link between them is plausible, even though it is not currently observed.

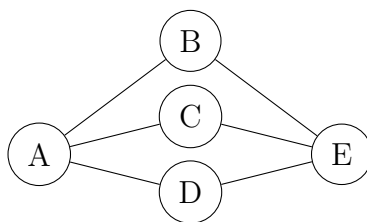


Figure 42: Shared neighbors provide structural evidence for a potential missing link.

In this example, nodes A and E are not directly connected, but they share the common neighbors B, C, and D. This overlap increases the likelihood that a future link may form between them.

Beyond immediate neighborhoods, longer **paths** also provide evidence. If many short paths connect two nodes, information can flow efficiently between them, suggesting compatibility within the network structure. These path-based signals capture both local and global organization and allow link prediction methods to operate without relying on node

attributes.

These two types of evidence capture different aspects of structural proximity. Neighborhood overlap reflects local similarity, where nodes sharing many common neighbors are likely to belong to the same community.

In contrast, path-based signals capture broader structural proximity. Nodes connected through multiple short paths may still be likely to interact even when they share few direct neighbors.

For example in an online marketplace, two products may never have been purchased together. However, if both are frequently bought alongside the same set of items, the network reveals a structural similarity. This similarity can be used to predict a potential association between the two products, even in the absence of direct co-purchase data.

8.3 From Structural Signals to Predictive Interpretation

To enable automated inference, link prediction is commonly framed as a learning problem over pairs of nodes. Each pair is evaluated based on how strongly the network structure supports a relationship between them. Existing links provide evidence of what connected node pairs look like structurally, while unconnected pairs illustrate the absence of such patterns.

To train a link prediction model, we need both positive examples (existing links) and negative examples (randomly selected non-links). The model learns to distinguish between pairs that are connected and pairs that are not, based on their structural features.

Consider a collaboration network where nodes represent individuals and edges represent past collaborations. If two individuals are not directly connected but repeatedly collaborate with the same set of people, the network places them in a similar structural position. A link prediction model identifies this overlap and assigns a high probability to a future collaboration, even though the connection has not yet been observed.

When evaluating link prediction, ranking metrics are commonly used because users typically interact with only the top recommendations [17].

- **Precision@K**: measures the proportion of predicted links in the top K that are actually correct [19].

$$\text{Precision@K} = \frac{\text{number of correct links in top } K}{K}$$

For example, if 4 out of the top 10 predicted links are correct:

$$\text{Precision@10} = \frac{4}{10} = 0.4 = 40\%$$

Precision@K focuses on recommendation accuracy. A high Precision@K means that most of the top-ranked predicted links are relevant and correct.

- **Recall@K**: measures how many of all true links are successfully retrieved within the top K predictions.

$$\text{Recall@K} = \frac{\text{number of correct links in top K}}{\text{total number of true links in the test set}}$$

If there are 20 true links overall and 4 appear in the top 10 predictions:

$$\text{Recall@10} = \frac{4}{20} = 0.2 = 20\%$$

Recall@K focuses on coverage. A high Recall@K indicates that the model successfully retrieves a large proportion of all true links.

- **NDCG@K (Normalized Discounted Cumulative Gain)**: evaluates not only whether relevant links appear in the top K , but also how highly they are ranked. Correct predictions appearing earlier in the ranking receive higher importance.

A simplified form of DCG@K is defined as:

$$\text{DCG@K} = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}$$

where rel_i indicates whether the item at position i is relevant.

NDCG@K is obtained by normalizing DCG@K using the ideal ranking:

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

[20]

For example, if relevant predicted links appear near the top of the recommendation list, the NDCG score becomes higher. This makes NDCG@K particularly useful in recommendation systems where ranking quality matters.

Unlike Precision@K and Recall@K, NDCG@K additionally considers the ranking positions of predicted links. Correct predictions appearing earlier in the recommendation list contribute more to the final score.

These metrics are particularly important in recommendation systems because ranking quality often matters more than evaluating every possible prediction.

The workflow illustrated in [Figure 43](#) consists of several main stages:

1. **Input Graph**: The process begins with the original graph, where nodes represent entities and edges represent observed relationships.
2. **Generate Node Pairs**: Pairs of nodes are generated to create candidate relationships for prediction.

3. **Positive Examples:** Existing links in the graph are treated as positive examples because they represent observed relationships.
4. **Negative Examples:** Unconnected node pairs are selected as negative examples to help the model distinguish non-links from true links.
5. **Extract Structural Features:** Structural properties such as shared neighbors, similarity, or path-based features are extracted from node pairs.
6. **Train Classifier:** A machine learning model is trained using the extracted structural features from both positive and negative examples.
7. **Predict Links:** The trained model estimates the likelihood of future or missing links between node pairs.
8. **Evaluate:** Prediction performance is evaluated using ranking metrics such as Precision@K, Recall@K, and NDCG@K.

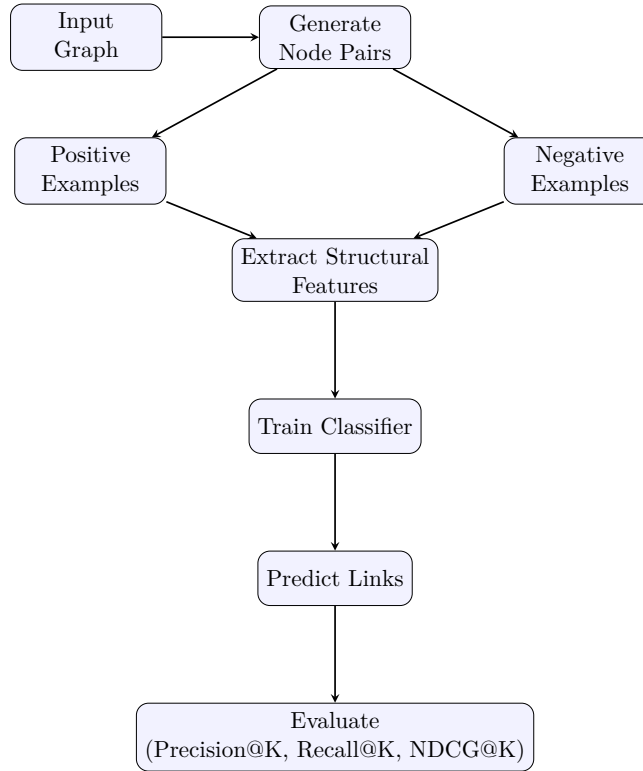


Figure 43: Link prediction workflow.

Predicted links should be interpreted as structural likelihoods, not guarantees. External factors such as time, preferences, or organizational boundaries may still prevent the relationship from forming. Additionally, large networks contain far more non-links than links, making careful interpretation essential when evaluating prediction outcomes.

Despite these challenges, link prediction plays a vital role in graph-based systems. It supports recommendation engines, social platforms, and scientific discovery by revealing hidden structure and guiding decisions about where new relationships are most likely to emerge.

9 Knowledge Graph Completion

In graph data science, knowledge graphs represent an essential foundation for organizing and integrating complex relational information. A knowledge graph models real-world entities and their relationships in a structured and semantically meaningful way, enabling data from diverse sources to be connected within a unified representation [4]. Unlike simple graph structures, knowledge graphs explicitly encode the types of entities and relationships, which supports richer interpretation and reasoning.

While link prediction discussed in [Section 8](#) focuses on inferring missing connections in homogeneous networks, knowledge graph completion extends this idea to heterogeneous graphs where both entities and relationships have semantic types. The core challenge shifts from "is there a link?" to "is there a link of this specific type between these entities?"

For example, in a movie knowledge graph, entities may include actors, directors, and films. Relationships can represent semantic connections such as *acted in*, *directed*, or *produced*. Knowledge graph completion attempts to infer missing relationships of specific types between entities.

Once a knowledge graph has been constructed, it can be further analyzed and enhanced through inference tasks. However, real-world knowledge graphs are often incomplete, containing missing entities or relationships due to limitations in data collection or extraction processes. This motivates the need for **knowledge graph completion**, which aims to infer missing information by leveraging both structural patterns and semantic consistency within the graph.

This task is particularly important in heterogeneous and semantic graphs, where entities and relationships carry explicit meaning beyond simple connectivity. By combining structural patterns with semantic consistency, knowledge graph completion enables more accurate reasoning and improves the quality of recommendation and inference systems.

9.1 Motivation and Nature of Knowledge Graphs

Knowledge graphs are structured representations of real-world facts, where nodes represent entities and edges represent semantic relationships between them. Unlike simple networks, knowledge graphs are typically **heterogeneous**, meaning they contain multiple types of entities and relationships.

To understand why knowledge graph completion differs from standard link prediction, it is important to distinguish between homogeneous and heterogeneous graphs.

A homogeneous graph contains only one type of node and one type of relationship. For example, in a social network, nodes may represent users and edges may represent friendships. In this setting, link prediction asks questions such as: "Will these two users become friends?"

In contrast, a heterogeneous graph contains multiple types of entities and relationships. For example, a scholarly knowledge graph may include authors, papers, and topics connected through relations such as *writes*, *cites*, and *about*. Here, the task becomes more semantically rich. Instead of simply predicting whether a link exists, knowledge graph completion asks whether a specific type of relationship should exist between two entities, such as: “Does this author have expertise in this topic?”

In summary, homogeneous graphs focus mainly on whether a connection exists between similar types of nodes, whereas heterogeneous knowledge graphs additionally consider the semantic type of both entities and relationships. This richer structure enables more expressive inference and reasoning tasks.

For instance, an academic knowledge graph may connect authors to the papers they write and papers to the topics they discuss. If an author has published extensively on a topic but is not explicitly connected to it, this missing relationship represents incomplete knowledge. Knowledge graph completion aims to infer such missing facts by leveraging existing relational patterns.

As illustrated in Figure 44, knowledge graphs represent entities through semantically meaningful relationships rather than simple connections. The graph shows how an *Author* is linked to a *Paper* through the relation *writes*, and how the *Paper* is connected to a *Topic* through the relation *about*. Such typed relationships provide richer contextual information and enable more precise inference tasks in knowledge graph completion.



Figure 44: Semantic relationships in a knowledge graph.

9.2 Capturing Semantic Structure Through Knowledge Graph Embeddings

After understanding the role of semantic relationships in knowledge graphs, the next step is to represent these structures in a form suitable for machine learning models. Knowledge graph embeddings provide a way to transform entities and relations into continuous vector representations while preserving semantic meaning.

To enable prediction in heterogeneous graphs, entities and relationships must be represented in a machine-readable form. Knowledge graphs commonly organize information as semantic triples of the form (head entity, relation, tail entity), which provide the foundation for knowledge graph embedding methods.

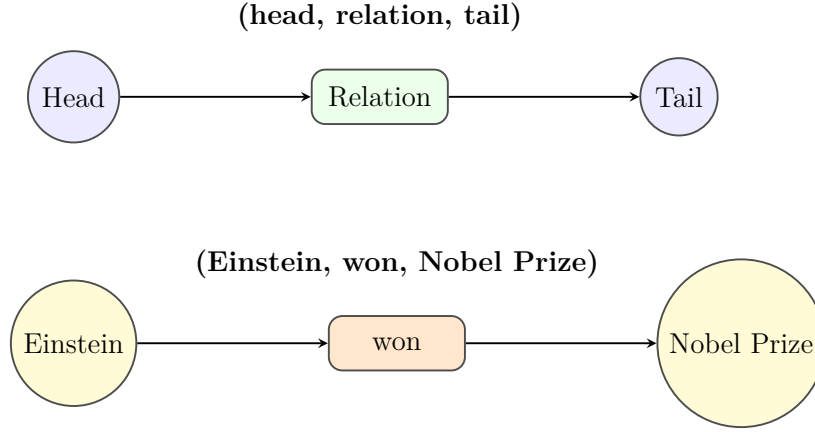


Figure 45: Structure of a knowledge graph triple.

As illustrated in Figure 45, each triple encodes a semantic fact by linking a head entity to a tail entity through a specific relation. The top half shows the abstract structure, while the bottom half provides a concrete example: (Einstein, won, Nobel Prize). This structured representation forms the basis for knowledge graph embeddings, which will be discussed in the following section.

Knowledge graph embedding models map entities and relations into a shared vector space while preserving the semantic structure of the graph.

Unlike embeddings in homogeneous networks, knowledge graph embeddings must represent both relation types and directionality. This means that different relations carry different meanings, even when they involve similar entities.

For example, the relation *author-writes-paper* expresses a different semantic meaning than *paper-cites-paper*. Therefore, embedding models learn not only which entities are connected, but also the specific type of relationship between them.

A simple way to understand knowledge graph embeddings is to view them as vector representations that capture semantic relationships between entities.

In translation-based models such as **TransE**, a relationship is interpreted as a transformation from the head entity to the tail entity. The model learns embeddings such that adding the relation vector to the head entity vector results in a vector close to the tail entity [25].

This idea is represented as:

$$\mathbf{h} + \mathbf{r} \approx \mathbf{t}$$

where $\mathbf{h}$ represents the head entity embedding, $\mathbf{r}$ represents the relation embedding, and $\mathbf{t}$ represents the tail entity embedding.

The approximation symbol indicates that the embeddings are learned so that valid triples are positioned close together in the vector space rather than matching exactly.

For example, if the triple (Einstein, won, Nobel Prize) is valid, the model learns embeddings satisfying:

$$\mathbf{e}_{Einstein} + \mathbf{e}_{won} \approx \mathbf{e}_{Nobel\ Prize}$$

This geometric relationship allows the model to identify missing or plausible triples by finding entities that best satisfy the translation pattern.

The underlying principle is that semantic regularities in the graph should correspond to geometric regularities in the embedding space. If two entities frequently appear in similar relational contexts, their embeddings will be positioned in compatible regions of the vector space. Likewise, relationships that connect similar pairs of entities will exhibit consistent geometric patterns. This structure allows the model to assess whether a proposed triple is semantically plausible.

Returning to the scholarly graph example in [Figure 44](#), the triples (Author, writes, Paper) and (Paper, about, Topic) define semantic relationships within the graph. Through embedding, these relationships are transformed into geometric patterns in vector space, enabling the model to infer missing facts, such as linking an author to a topic they frequently publish about even when the relationship is not explicitly recorded.

9.3 Interpreting Predictions and Practical Considerations

Knowledge graph completion focuses on predicting whether a specific relationship between two entities is likely to be valid given the existing graph structure.

Rather than predicting arbitrary links, the task evaluates whether a candidate triple is semantically plausible based on existing relational patterns.

[Figure 46](#) illustrates this concept in a corporate knowledge graph. Consider an employee who is not explicitly linked to a department. However, if the employee collaborates primarily with members of a particular department and works on projects associated with that department, the graph provides strong semantic evidence for a missing affiliation relationship. Knowledge graph completion uses such patterns to infer likely facts that are absent from the graph.

In this example, the shared project context provides indirect semantic evidence that the employee may belong to the department.

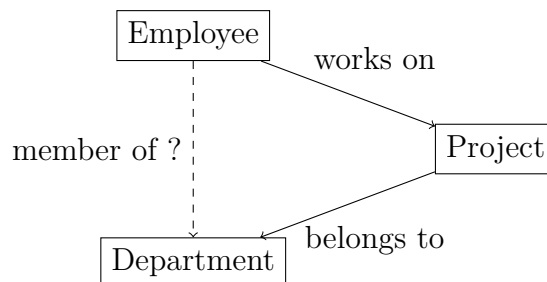


Figure 46: Inferring a missing relationship.

The dashed edge in [Figure 46](#) represents a candidate relationship that is not explicitly present but is semantically consistent with existing connections. Completion models assign plausibility scores to such candidates rather than asserting absolute truth. These scores indicate how well the candidate triple aligns with the learned semantic structure of the graph.

A key challenge in this task lies in the imbalance and diversity of relationship types. Some relations occur frequently, while others are rare, making evaluation non-trivial. Additionally, predictions must be interpreted carefully, as semantic plausibility does not guarantee factual correctness. External validation through domain expertise or additional data sources remains essential before acting on inferred relationships.

Despite these challenges, knowledge graph completion is essential for maintaining large-scale knowledge bases. It supports applications such as question answering, semantic search, and recommendation systems by enabling knowledge graphs to evolve beyond manually curated facts.

10 Constructing Graphs from Text Using Natural Language Processing

Many real-world graph-based systems rely on unstructured text as a primary source of information. Documents, articles, and user-generated content often contain rich relational knowledge that is not explicitly represented in structured form. To enable graph-based analysis in such settings, this implicit information must first be transformed into a graph representation.

Constructing graphs from text provides the foundation for tasks such as co-occurrence analysis, similarity-based relationships, and knowledge graph construction. By extracting entities and inferring relationships from textual data, unstructured sources can be integrated into graph-based workflows, enabling subsequent analysis, inference, and completion techniques discussed in earlier chapters.

Once constructed, text-derived graphs can be analyzed using graph learning techniques discussed in earlier sections. For example, community detection [subsection 5.3](#) can reveal thematic clusters, and link prediction [Section 8](#) can infer missing relationships.

10.1 Motivation for Building Graphs from Text

Large volumes of information are stored in unstructured text such as articles, reports, and scientific publications. Although these documents contain rich semantic information, relationships between entities are often implicit and scattered across multiple texts.

Constructing graphs from text aims to transform this unstructured information into a structured graph representation where entities become nodes and relationships become edges. This allows textual knowledge to be analyzed using graph-based methods.

For example, consider a collection of news articles discussing companies, executives, locations, and events. The relationships between these entities may exist implicitly within the text but are not explicitly organized. After constructing a graph, entities can be connected through meaningful relationships such as:

- company $\leftrightarrow$ executive
- company $\leftrightarrow$ location
- person $\leftrightarrow$ event

This structured representation enables collective analysis across documents and forms the basis for graph learning tasks such as community detection, link prediction, semantic search, and knowledge graph completion.

Entity and Relationship Extraction from Text

The first step in constructing a graph from text is **entity identification**. Entities represent meaningful elements such as people, organizations, locations, or concepts that

can exist independently and participate in relationships. **Natural language processing (NLP)** techniques are used to detect these entities and assign them semantic types. Common NLP techniques for entity extraction include:

- **Named Entity Recognition (NER):** Identifies entities like people, organizations, and locations [1]
- **Part-of-speech tagging:** Helps identify potential relationship indicators such as verbs and prepositions
- **Dependency parsing:** Analyzes grammatical connections between words to help identify relationships between entities in a sentence.

Once entities are identified, **relationships** between them are extracted. Some relationships are explicitly stated in text, while others must be inferred from context. Explicit relationships typically carry strong semantic meaning, whereas inferred relationships capture softer associations that emerge across multiple texts.

To illustrate explicit extraction, consider a sentence where both entities and relationships are clearly stated. For example, in the sentence "Apple acquired Beats in 2014," the entities "Apple" and "Beats" are connected through the explicit relationship "acquired." This direct extraction process forms the foundation for constructing graphs from textual data.

10.2 Inferring Structure Through Co-occurrence and Graph Analysis

While some relationships are explicitly stated in text such as "Apple acquired Beats" many meaningful connections are never directly expressed. Instead, they emerge through patterns of co-occurrence across multiple documents. This distinction motivates different approaches for constructing graphs from textual data.

When entities repeatedly appear together within documents or contexts, their co-occurrence can indicate a meaningful association. Although such relationships may lack explicit semantic labels, they often reveal strong conceptual or contextual connections that are valuable for analysis.

Figure 47 illustrates this concept using research topics. In a corpus of scientific articles, Topic A, Topic B, and Topic C may frequently appear in the same papers even if no direct relationship is explicitly stated. Over time, repeated co-occurrence suggests a conceptual link between these topics, allowing them to be connected in a graph. This form of inference is particularly useful for discovering emerging themes and hidden structures in large text collections.

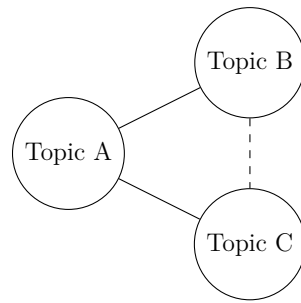


Figure 47: Relationships inferred from repeated co-occurrence of entities in text.

The strength of inferred relationships can be estimated using several co-occurrence measures:

- **Raw co-occurrence count:** Number of documents where both entities appear.
- **Jaccard similarity:** Co-occurrence count divided by the total unique occurrences of either entity.
- **Pointwise Mutual Information (PMI):** A statistical measure that compares the observed co-occurrence with what would be expected if the entities appeared independently. Higher PMI values indicate stronger associations that are unlikely to occur by chance.

Topics that appear together in many documents receive stronger connections, while those that co-occur only occasionally receive weaker links. This weighted approach preserves the nuance of the underlying textual evidence.

10.3 From Text to Graph: Integration and Analysis

After construction, the resulting graph enables graph-based analytical techniques to be applied to textual knowledge. Central entities may indicate key concepts, densely connected regions may reveal thematic communities, and inferred links may suggest previously unnoticed relationships.

The process of building graphs from text follows a general pipeline:

1. **Entity extraction:** Identify meaningful entities (people, organizations, concepts) in text using NER and related techniques
2. **Relationship extraction:** Identify explicit relationships (e.g., "acquired," "works for") or implicit associations through co-occurrence analysis
3. **Graph construction:** Create nodes for entities and edges for relationships, with weights reflecting frequency, similarity scores, or confidence levels
4. **Graph analysis:** Apply techniques from previous sections centrality, community detection, link prediction to extract insights from the constructed graph

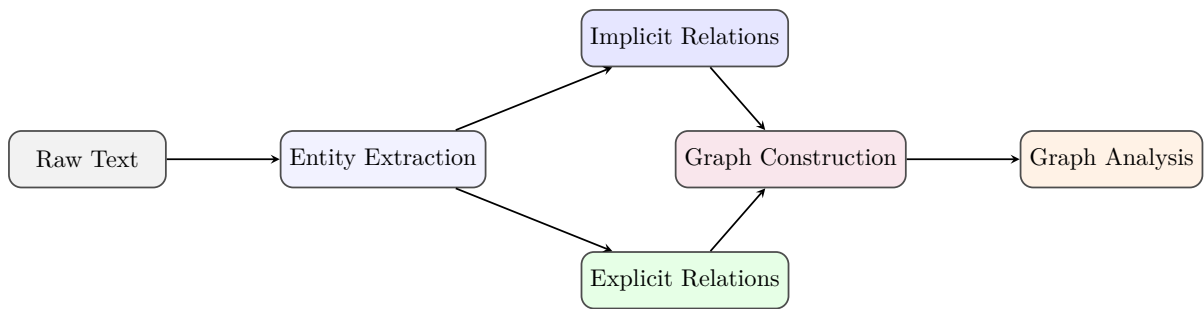


Figure 48: Pipeline from text to graph analysis.

Figure 48 illustrates the complete pipeline from raw text to graph analysis. The process begins with entity extraction using NLP techniques, followed by two parallel paths for relationship extraction: explicit relationships from direct statements and implicit relationships inferred from co-occurrence. These are combined to construct a graph, which then enables the analytical techniques discussed throughout this book.

10.4 Challenges and Considerations

Text-derived graphs are inherently noisy due to ambiguity in natural language. Key challenges include:

- **Entity disambiguation:** The same word or name may refer to different entities depending on context. For example, “Apple” may refer to the technology company or the fruit. Graph construction systems must use contextual information to determine the correct meaning. Additionally, coreference resolution is required to identify when different mentions such as “Einstein” and “he” refer to the same entity.
- **Relationship ambiguity:** Relationships expressed in text may have different meanings depending on linguistic or domain context. For example, the verb “works with” may describe collaboration in one context and employment in another. This ambiguity can reduce extraction accuracy, making domain-specific NLP models and training data important.
- **Noise and false positives:** Entities may frequently appear together in documents even when no meaningful relationship exists between them. As a result, co-occurrence alone can produce incorrect or weak graph connections. Statistical measures such as PMI and similarity metrics help distinguish significant associations from random co-occurrence patterns.
- **Scale:** Large-scale text collections may contain millions of documents and entities. Processing such datasets requires efficient NLP pipelines, scalable graph construction methods, and sometimes distributed computing frameworks to handle computational complexity.

Despite these challenges, when constructed at scale, text-derived graphs provide a powerful abstraction that transforms unstructured text into analyzable network structures. This process allows knowledge graphs to expand dynamically as new textual data

become available, complementing manually curated knowledge bases.

The integration of NLP and graph analytics from entity extraction to co-occurrence analysis to graph-based inference creates a complete workflow for turning raw text into actionable insights, bridging the gap between unstructured language and the structured graph representations explored throughout this report.

11 Practical

Graph-Based Recommendation System Using Personalized PageRank

This chapter presents the practical implementation of the graph-based recommendation system using the Personalized PageRank algorithm. The implementation focuses on constructing a bipartite graph between users and movies, processing the MovieLens dataset, and generating personalized recommendations based on user interactions. This chapter also explains the datasets, tools, algorithms, and major implementation steps used in the system.

11.1 Implementation Overview

Objective

Build a recommendation engine that uses graph algorithms to suggest items (movies, books, products) based on user-item relationships rather than traditional matrix factorization.

Implementation Plan

1. Define Scope

- Domain: Movie recommendation (or books/products).
- Graph Structure:
 - Nodes: Users and Items (bipartite graph).
 - Edges: Interactions (ratings, purchases, likes).
- Goal: Recommend top-N items for a given user using graph-based techniques.

2. Dataset

Options:

- MovieLens Dataset (movies & ratings).
- Amazon Product Dataset (for e-commerce).

3. Tools & Technologies

- Programming Language: Python
- Libraries:
 - NumPy and Pandas (data preprocessing and handling)
 - NetworkX (graph construction and PageRank algorithm)
 - Matplotlib and Seaborn (data visualization)

- Scikit-learn (KNN, Random Forest, evaluation metrics) [24]
- SciPy (sparse matrix representation)
- Surprise Library (SVD matrix factorization)

4. Steps to Implement

Step 1: Data Preprocessing

- Load dataset (users, items, ratings).
- Filter for active users and popular items to reduce graph size.
- Normalize ratings or convert to binary (liked/disliked).

Step 2: Graph Construction

- Create a bipartite graph:
 - Add user nodes and item nodes.
 - Add edges between users and items they interacted with (weighted by rating).

Step 3: Apply Graph Algorithms

- Personalized PageRank:
 - Run PageRank starting from a specific user node.
 - Higher scores indicate stronger recommendation.

Step 4: Generate Recommendations

- Sort items by PageRank score for the user.
- Filter out items the user already interacted with.
- Return top-N recommendations.

Step 5: Evaluation

- Split data into train/test.
- Use metrics like Precision@K, Recall@K, or NDCG.

Step 6: Visualization

- Show:
 - Graph structure (users and items).
 - Recommended items for a sample user.

5. Extensions

- Implement Graph Neural Networks (GNN) for better recommendations.
- Add content-based features (e.g., movie genres).

6. Comparison with Non-Graph Approach

Build the same model as above but use non-graph approaches such as matrix factorization or KNN or random forest (wherever applicable). Make a performance comparison for the two models.

11.2 Dataset Description

This project uses the MovieLens dataset, which is a widely used dataset for building and evaluating recommendation systems [21]. The dataset contains information about movies and user interactions in the form of ratings. It is suitable for graph-based recommendation systems because it provides relationships between users and movies that can be represented as a bipartite graph.

Two main files were used in this project:

- **ratings.csv**

This dataset contains the ratings provided by users for different movies. Each row represents an interaction between a user and a movie. The dataset was used to build the connections between user nodes and movie nodes in the graph.

The file contains the following columns:

- **userId**: A unique identifier for each user.
- **movieId**: A unique identifier for each movie.
- **rating**: The score given by the user to the movie, usually ranging from 0.5 to 5.0.
- **timestamp**: The time when the rating was submitted.

The ratings dataset is the core component of the recommendation system because it reflects user preferences and interaction behavior.

- **movies.csv**

This dataset contains information about the movies available in the system. It was used to display movie names and genres in the recommendation results.

The file contains the following columns:

- **movieId**: A unique identifier for each movie.
- **title**: The name and release year of the movie.
- **genres**: The categories associated with the movie, such as Action, Comedy, Drama, or Romance.

The movies dataset helps improve the interpretability of the recommendations by providing descriptive information about each movie.

The two datasets were combined using the **movieId** column in order to connect user ratings with movie information. This integration allowed the system to analyze user preferences and generate personalized movie recommendations using graph-based algorithms such as Personalized PageRank.

11.3 Data Preprocessing

Before constructing the recommendation graph, the dataset was preprocessed to improve recommendation quality and reduce sparsity. The preprocessing stage included filtering inactive users and unpopular movies, converting ratings into binary interactions, and splitting the data into training and testing sets.

To reduce data sparsity and improve recommendation quality, inactive users and unpopular movies were filtered out. Only users with a minimum number of ratings and movies with sufficient interactions were retained.

```
MIN_USER_RATINGS = 50 # a user must have rated at least 50 movies
MIN_MOVIE_RATINGS = 20 # a movie must have received at least 20 ratings

# Count how many ratings each user and movie has
user_counts = ratings['userId'].value_counts()
movie_counts = ratings['movieId'].value_counts()

# Keep only the active users and popular movies
active_users = user_counts[user_counts >= MIN_USER_RATINGS].index
popular_movies = movie_counts[movie_counts >= MIN_MOVIE_RATINGS].index

# Filter the ratings dataframe to only include those users/movies
ratings_filtered = ratings[
    ratings['userId'].isin(active_users) &
    ratings['movieId'].isin(popular_movies)
].copy()
```

Converting Ratings into Binary Interactions

Ratings were converted into binary values to simplify user preferences. Ratings greater than or equal to 3.5 were considered positive interactions (liked = 1), while lower ratings were considered negative interactions.

```
LIKE_THRESHOLD = 3.5

ratings_filtered['liked'] = (
    ratings_filtered['rating'] >= LIKE_THRESHOLD
).astype(int)
```

Keeping Positive User Interactions

Only positive user interactions were retained in order to construct the recommendation graph using meaningful user preferences.

```
liked_data = ratings_filtered[ratings_filtered['liked'] == 1].copy()
```

Train-Test Split

The filtered positive interactions were divided into training and testing sets using an

80/20 ratio. The training set was used to build the recommendation graph, while the testing set was used to evaluate recommendation performance.

```
train_data, test_data = train_test_split(
    liked_data,
    test_size=0.2,
    random_state=42 # for the same value every time
)
```

11.4 Graph Construction

A bipartite directed graph was constructed using the NetworkX library to represent relationships between users and movies [22]. User nodes and movie nodes were added separately to distinguish the two entity types within the graph structure. Each movie node also stores the movie title as metadata to improve the readability and interpretability of the recommendation results. The directed graph structure enables the Personalized PageRank algorithm to propagate recommendation scores through user–movie interactions effectively.

```
G = nx.DiGraph()

# Add User Nodes
user_ids = train_data['userId'].unique()

for uid in user_ids:
    G.add_node(f"u_{uid}", type='user')

# Add Movie Nodes
for _, row in train_data[['movieId', 'title_x']].drop_duplicates().iterrows():
    G.add_node(
        f"m_{row['movieId']}",
        type='movie',
        title=row['title_x']
    )
```

Directed weighted edges were added between user nodes and movie nodes to represent user interactions. Each edge weight corresponds to the original movie rating, allowing stronger preferences to have greater influence during Personalized PageRank propagation.

```
# Add Weighted Edges
for _, row in train_data.iterrows():
    u_node = f"u_{row['userId']}"
    m_node = f"m_{row['movieId']}"
    w      = row['rating']

    G.add_edge(u_node, m_node, weight=w)
```

11.5 Personalized PageRank Algorithm

Personalized PageRank (PPR) was applied to rank movies according to their relevance to a target user. The algorithm performs a biased random walk over the graph while

repeatedly returning to the selected user node through a personalization vector. Higher-ranked movie nodes represent stronger recommendations for that user.

```
def personalized_pagerank(graph, user_node, alpha=0.85, top_n=10):

    personalization = {node: 0 for node in graph.nodes()}
    personalization[user_node] = 1.0

    ppr_scores = nx.pagerank(
        graph,
        alpha=alpha,
        personalization=personalization,
        weight='weight'
    )

    movie_scores = {
        node: score
        for node, score in ppr_scores.items()
        if graph.nodes[node].get('type') == 'movie'
    }

    top_movies = sorted(
        movie_scores.items(),
        key=lambda x: x[1],
        reverse=True
    )[:top_n]

    return top_movies
```

Recommendation Generation

After computing Personalized PageRank scores, previously interacted movies were filtered out to avoid recommending already seen items. The remaining movies were ranked according to their relevance scores, and the top-N unseen movies were returned as personalized recommendations.

```
def get_recommendations(graph, user_node, train_df, alpha=0.85, top_n=10):

    raw_scores = personalized_pagerank(
        graph,
        user_node,
        alpha=alpha,
        top_n=len(graph.nodes())
    )

    uid = int(user_node.split('_')[1])

    seen_movies = set(
        f"m_{mid}"
        for mid in train_df[train_df['userId'] == uid]['movieId'].values
    )

    recommendations = []

    for movie_node, score in raw_scores:
```



```

if movie_node not in seen_movies:

    title = graph.nodes[movie_node].get('title', 'Unknown')

    recommendations.append({
        'title': title,
        'ppr_score': round(score, 8)
    })

if len(recommendations) >= top_n:
    break

return pd.DataFrame(recommendations)

```

The recommendation system was tested on a sample active user to demonstrate the generated personalized recommendations. The system successfully produced a ranked list of unseen movies based on Personalized PageRank scores.

Table 4: Top 10 Movies for User 414

Movie Title	Rating
Bound (1996)	5.0
Unforgiven (1992)	5.0
Clerks (1994)	5.0
Run Lola Run (Lola rennt) (1998)	5.0
Animal House (1978)	5.0
Midnight in the Garden of Good and Evil (1997)	5.0
Stripes (1981)	5.0
Sting, The (1973)	5.0
Excalibur (1981)	5.0
The Revenant (2015)	5.0

The recommendation pipeline was applied to all users in the test set. For each user, Personalized PageRank recommendations were generated and stored for evaluation purposes.

```

TOP_N = 10

all_recommendations = {}

for uid in test_users_in_graph:

    user_node = f"u_{uid}"

    recs_df = get_recommendations(
        G,
        user_node,
        train_data,
        alpha=0.85,
        top_n=TOP_N
    )

    if recs_df.empty:

```

```

    all_recommendations[uid] = []

else:
    rec_movie_ids = [
        int(node.split('_')[1])
        for node in recs_df['movie_node']
    ]

    all_recommendations[uid] = rec_movie_ids

```

Preview Recommendations vs Ground Truth

Recommended movies were compared with the actual movies liked by users in the test set to evaluate recommendation quality. The overlap between recommended and actual movies was used as an indicator of recommendation accuracy.

User 438

Recommended (10):

- Four Rooms (1995)
- City of Lost Children, The (Cite des enfants perdus, La) (1995)
- Vertigo (1958)
- Monty Python and the Holy Grail (1975)
- Grifters, The (1990)

Actually liked in test (65):

- Mr. Hollands Opus (1995)
- Broken Arrow (1996)
- Birdcage, The (1996)
- Hackers (1995)
- Star Trek: Generations (1994)

Hits (overlap): 0

User 249

Recommended (10):

- Four Rooms (1995)
- City of Lost Children, The (Cite des enfants perdus, La) (1995)
- Vertigo (1958)
- Monty Python and the Holy Grail (1975)
- Grifters, The (1990)

Actually liked in test (83):

- Ace Ventura: When Nature Calls (1995)
- Interview with the Vampire: The Vampire Chronicles (1994)
- Star Wars: Episode IV - A New Hope (1977)
- Lion King, The (1994)
- Beverly Hills Cop III (1994)

Hits (overlap): 2

A ground truth dictionary was constructed from the test set to store the actual movies liked by each user. This ground truth was later used to compare the recommended movies against the real user preferences during evaluation.

```

ground_truth = {}

for uid in test_users_in_graph:

    liked_in_test = test_data[
        test_data['userId'] == uid
    ]['movieId'].tolist()

    ground_truth[uid] = liked_in_test

Total users : 385
Avg liked per user : 21.1
Avg recs per user : 10.0

```

11.6 Evaluation

Evaluation Metrics

- **Precision@K** measures the proportion of recommended movies in the top-K list that are actually relevant to the user.
- **Recall@K** measures how many relevant movies from the ground truth were successfully retrieved in the recommendation list.
- **NDCG@K** evaluates the ranking quality of the recommendations by giving higher importance to relevant movies appearing earlier in the recommendation list.

```

def precision_at_k(recommended, actual, k):

    recommended_k = recommended[:k]
    hits = len(set(recommended_k) & set(actual))

    return hits / k

def recall_at_k(recommended, actual, k):

    recommended_k = recommended[:k]
    hits = len(set(recommended_k) & set(actual))

    return hits / len(actual)

```

NDCG@K was used to evaluate the ranking quality of the recommendation list. Higher scores are assigned when relevant movies appear earlier in the ranked recommendations.

Experimental Results

Table 5: Graph PPR Model Evaluation at $K = 10$

Metric	Value
Precision@10	0.0327
Recall@10	0.0133
NDCG@10	0.0315
Evaluated Users	385

The Personalized PageRank recommendation model was evaluated on 385 users using Precision@10, Recall@10, and NDCG@10.

The results indicate that the model was able to retrieve a limited number of relevant movies, while the low NDCG score suggests that relevant items were not consistently ranked near the top of the recommendation list.

Score Distribution Analysis

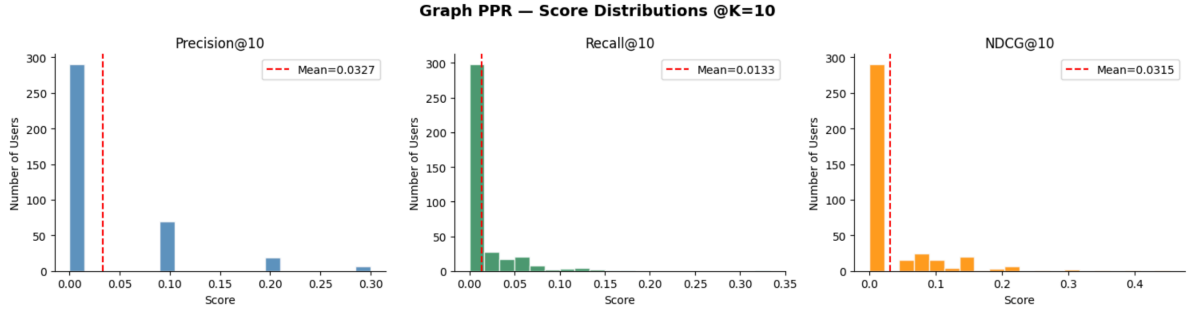


Figure 49: Distribution of Precision@10, Recall@10, and NDCG@10 scores for the Graph Personalized PageRank model.

The score distribution plots provide a more detailed analysis of the recommendation performance across all evaluated users.

Most users achieved very low Precision@10 and Recall@10 scores, as shown by the concentration of values near zero in the histograms.

This indicates that only a small portion of the recommended movies matched the movies actually liked by users in the test set.

The NDCG@10 distribution also shows relatively low values for most users, suggesting that relevant movies were not consistently ranked at the top positions of the recommendation list.

However, a small number of users achieved noticeably higher scores, indicating that the Personalized PageRank model was able to generate more accurate recommendations for users with stronger graph connectivity or richer interaction histories.

The red dashed lines in each histogram represent the average score across all evaluated users.

The distributions highlight the sparsity and variability of user preferences in the dataset, which makes accurate recommendation generation more challenging for graph-based approaches.

Evaluation at Different K Values

To further analyze the behavior of the Graph Personalized PageRank model, the evaluation process was repeated using different values of K.

This experiment helps examine how the recommendation quality changes as the number of recommended movies increases.

The model was evaluated using the following cutoff values:

```
k_values = [1, 3, 5, 10, 15, 20]
```

For each value of K, the average Precision@K, Recall@K, and NDCG@K scores were computed across all users in the test set.

```
k_results.append({
    'K' : k,
    'Precision' : np.mean(p_scores),
    'Recall' : np.mean(r_scores),
    'NDCG' : np.mean(n_scores)
})
```

Table 6 presents the evaluation results at different values of K.

Table 6: Evaluation metrics at different K values

K	Precision	Recall	NDCG
1	0.015584	0.000664	0.015584
3	0.029437	0.003140	0.027091
5	0.035844	0.007814	0.032256
10	0.032727	0.013272	0.031486
15	0.021818	0.013272	0.025237
20	0.016364	0.013272	0.022070

To better visualize the performance trends, the evaluation metrics were plotted against the different values of K.

```
ax.plot(k_results_df['K'], k_results_df['Precision'], label='Precision@K')
ax.plot(k_results_df['K'], k_results_df['Recall'], label='Recall@K')
ax.plot(k_results_df['K'], k_results_df['NDCG'], label='NDCG@K')
```

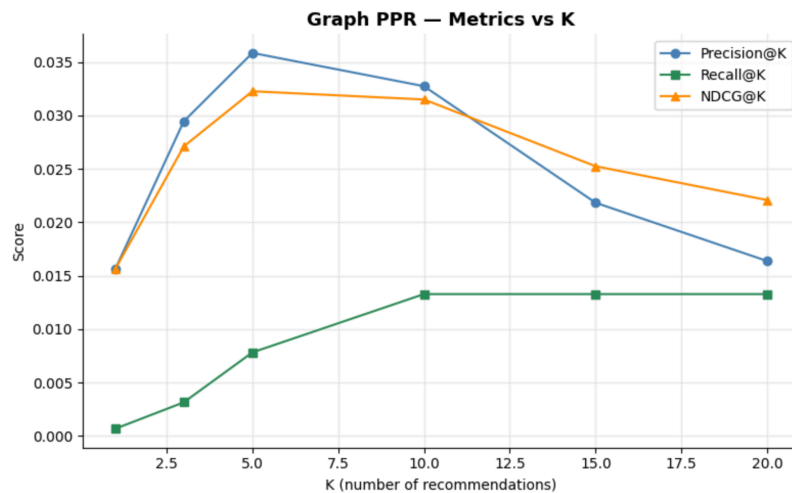


Figure 50: Graph PPR Evaluation Metrics at Different K Values

Figure 50 illustrates how the evaluation metrics change as the number of recommended items (K) increases.

The Precision@ K score gradually increases from $K = 1$ and reaches its peak value at $K = 5$ with a score of approximately 0.0358.

This peak indicates that the recommendation model achieves its highest recommendation accuracy when suggesting only a small number of movies.

In other words, the top-ranked recommendations at $K = 5$ are more likely to be relevant to users.

After $K = 5$, the Precision values begin to decline because increasing the number of recommended movies introduces more less-relevant items into the recommendation list.

Similarly, the NDCG@ K metric also reaches its highest value near $K = 5$, which indicates that relevant movies are ranked more effectively near the top positions of shorter recommendation lists.

This suggests that the model performs better at ranking highly relevant movies earlier in the recommendation order when fewer recommendations are generated.

On the other hand, Recall@ K continuously increases as K becomes larger and stabilizes around $K = 10$.

This behavior indicates that recommending more movies improves the probability of retrieving relevant items from the ground truth.

However, the improvement in Recall comes at the cost of lower Precision and weaker ranking quality.

Overall, the results suggest that the Graph Personalized PageRank model performs best at smaller K values, particularly around $K = 5$, where the balance between recommendation relevance and ranking quality is strongest.

11.7 Visualization

In this section, several visualization techniques are presented to provide a clearer understanding of the graph-based recommendation system and the relationships between users and movies. These visualizations help interpret the structure of the bipartite graph, analyze user–movie interactions, and better understand how the Personalized PageRank model generates recommendations. The following subsections include graph structure visualization, recommendation visualization, and graph statistics analysis.

Subgraph Visualization

The visualization illustrates a small bipartite subgraph extracted from the full recommendation graph. User nodes are displayed on the left, while movie nodes are displayed on the right. Edges represent interactions between users and movies in the training dataset.

```
# Select top active users
top_users = train_data['userId'].value_counts().head(5).index.tolist()
```

```
# Build subgraph
sub_nodes = [f"u_{uid}" for uid in top_users]

for u_node in sub_nodes.copy():
    movie_neighbors = list(G.neighbors(u_node))
    sub_nodes.extend(movie_neighbors[:8])

subgraph = G.subgraph(list(set(sub_nodes)))
```

```
# Draw bipartite graph
nx.draw_networkx_nodes(subgraph, pos,
                        nodelist=user_nodes_sub,
                        node_color='steelblue')

nx.draw_networkx_nodes(subgraph, pos,
                        nodelist=movie_nodes_sub,
                        node_color='darkorange')

nx.draw_networkx_edges(subgraph, pos)
```

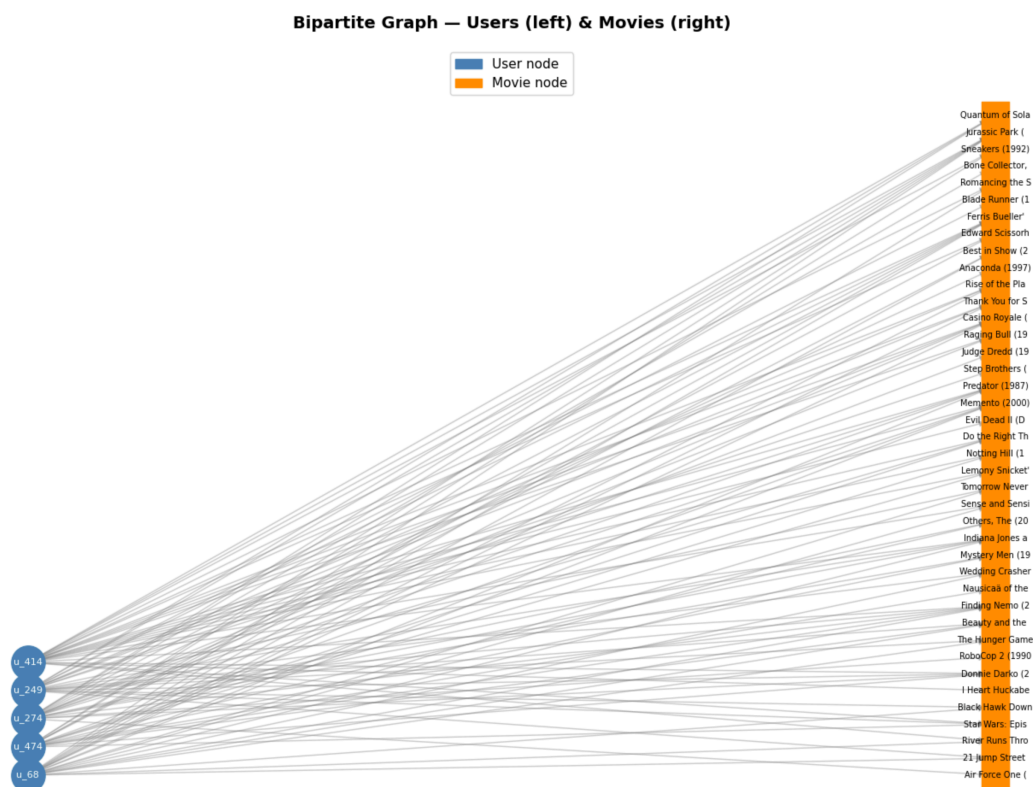


Figure 51: Subgraph visualization of the bipartite recommendation graph showing user–movie interactions.

The figure shows that some movies are connected to multiple users, indicating shared interests and relationships within the graph. It also demonstrates that certain users have denser interaction patterns, meaning they interacted with a larger number of movies.

Overall, the visualization provides an intuitive understanding of the bipartite graph structure used by the Personalized PageRank recommendation model.

Recommendation Visualization for a Sample User

To visually analyze recommendation quality, a sample user was selected. Three categories of movies were extracted: movies liked in the training set, movies recommended by the Personalized PageRank model, and movies actually liked in the test set.

```
viz_uid = train_data['userId'].value_counts().index[0]

train_movies = train_data[
    train_data['userId'] == viz_uid
]['movieId'].tolist()[:10]

rec_movies = all_recommendations.get(viz_uid, [])[:10]

test_movies = test_data[
    test_data['userId'] == viz_uid
]['movieId'].tolist()[:10]
```

The following code constructs a small recommendation graph centered around the selected user. Different node categories are added to represent training movies, recommended movies, and ground truth movies.

```
# Create recommendation graph
viz_G = nx.DiGraph()

for mid in train_movies:
    viz_G.add_node(f"m_{mid}", type='train')

for mid in rec_movies:
    viz_G.add_node(f"m_{mid}", type='rec')

for mid in test_movies:
    viz_G.add_node(f"m_{mid}", type='test')
```

Different colors were assigned to node categories to improve visualization clarity and allow easy comparison between recommended items and actual user preferences.

```
if ntype == 'train':
    color_map.append('seagreen')
elif ntype == 'rec':
    color_map.append('steelblue')
elif ntype == 'test':
    color_map.append('tomato')
```

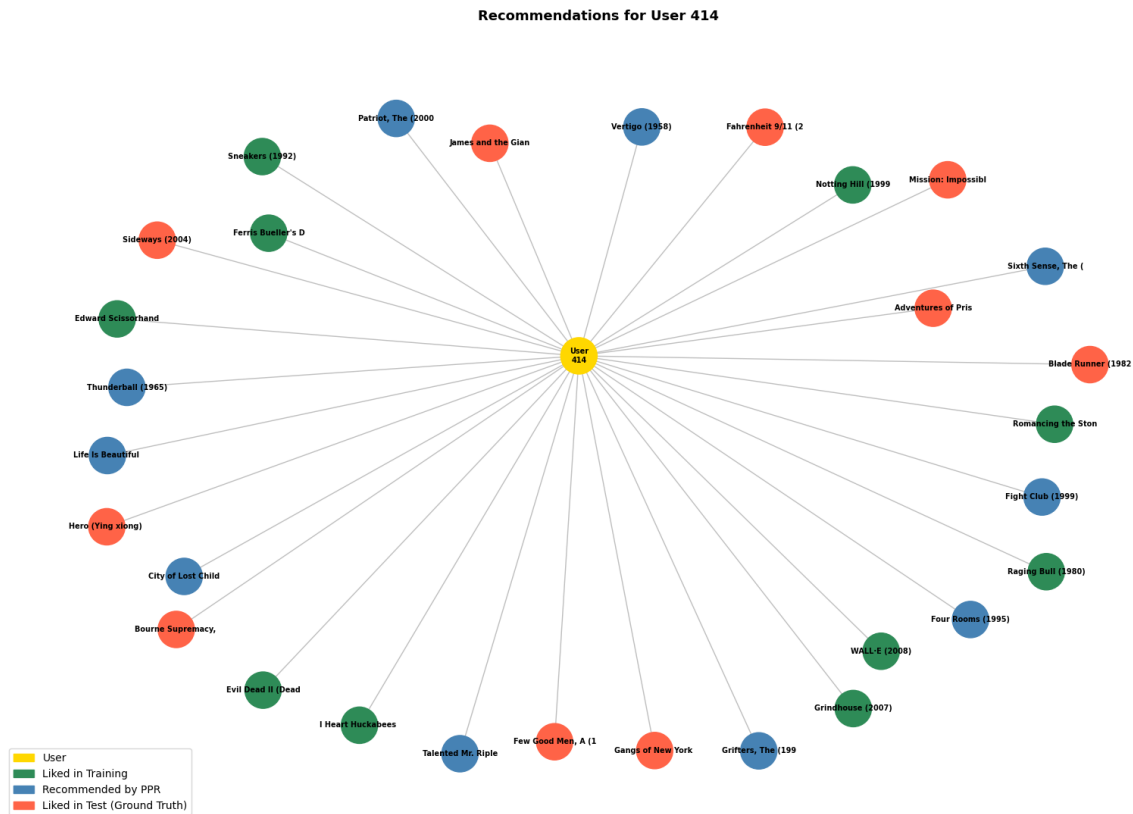



Figure 52: Recommendation visualization for a sample user showing training movies, recommended movies, and ground truth movies.

Figure 52 presents a recommendation visualization for a selected sample user using a star-shaped graph layout. The visualization compares three categories of movies: movies previously liked by the user in the training data, movies recommended by the Personalized PageRank (PPR) model, and movies actually liked by the user in the test set (ground truth).

In the graph, the central yellow node represents the selected user. Green nodes correspond to movies liked in the training set, blue nodes represent movies recommended by the PPR model, and red nodes indicate movies liked in the test set. The overlap between recommended movies and ground truth movies indicates successful recommendations generated by the model. Overall, the visualization provides an intuitive understanding of how graph relationships are used to generate personalized recommendations.

Graph Degree Distribution

The following code computes the degree values for both user nodes and movie nodes. User degree represents the number of movies interacted with by each user, while movie degree represents the number of users connected to each movie.

```
user_degrees = [
    G.out_degree(f"u_{uid}")
    for uid in train_data['userId'].unique()
    if f"u_{uid}" in G
]
```

```

movie_degrees = [
    G.in_degree(f"m_{mid}")
    for mid in train_data['movieId'].unique()
    if f"m_{mid}" in G
]

```

Histograms were generated to visualize the degree distributions of users and movies. These distributions help analyze graph connectivity, user activity levels, and movie popularity patterns within the dataset.

```

ax1.hist(user_degrees, bins=30)
ax2.hist(movie_degrees, bins=30)

```

To further analyze the structure of the bipartite graph, degree distribution analysis was performed for both users and movies. In graph-based recommendation systems, the degree of a node represents the number of connections associated with that node. User degree corresponds to the number of movies interacted with by each user, while movie degree represents the number of users connected to each movie.

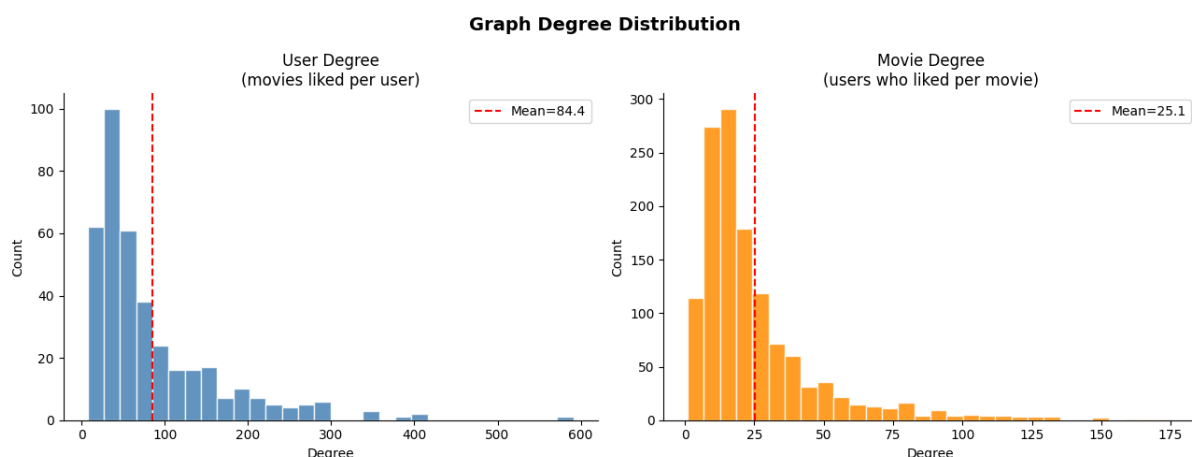


Figure 53: Degree distributions for user nodes and movie nodes in the bipartite recommendation graph

The left histogram illustrates the distribution of user degrees. Most users have relatively low to moderate interaction counts, while a smaller number of users exhibit very high degrees, indicating highly active users within the dataset. The red dashed line represents the average user degree, which is approximately 84 interactions per user.

The right histogram presents the distribution of movie degrees. Most movies were connected to a limited number of users, whereas a smaller subset of movies received significantly higher interaction counts, indicating popular movies with high connectivity. The average movie degree is approximately 25 users per movie, as indicated by the red dashed line.

Overall, the degree distributions reveal that the graph is highly imbalanced, with a few highly connected nodes and many sparsely connected nodes. Such patterns are

common in real-world recommendation datasets and influence the behavior of graph-based recommendation algorithms such as Personalized PageRank.

11.8 Extensions

Extensions 1 - Graph Neural Networks (LightGCN)

To further improve recommendation quality, a Graph Neural Network (GNN) extension was implemented using the LightGCN model. Unlike Personalized PageRank (PPR), which relies on fixed graph traversal heuristics, LightGCN learns user and movie embeddings directly from the interaction graph through neighborhood aggregation [10, 9]. This allows the model to capture collaborative filtering patterns and higher-order relationships between users and movies.

Preparing Data for GNN

Before training the GNN model, users and movies were converted into numerical indices. This encoding process allows the neural network to efficiently learn embedding representations for graph nodes.

```
from sklearn.preprocessing import LabelEncoder

user_encoder = LabelEncoder()
movie_encoder = LabelEncoder()

train_data['user_idx'] = user_encoder.fit_transform(train_data['userId'])
train_data['movie_idx'] = movie_encoder.fit_transform(train_data['movieId'])

NUM_USERS = train_data['user_idx'].nunique()
NUM_MOVIES = train_data['movie_idx'].nunique()
```

The encoding process transformed raw user and movie identifiers into continuous integer indices, enabling efficient embedding lookup and graph construction.

Building the Bipartite Graph

After encoding users and movies, the interaction graph was constructed using an edge index representation compatible with PyTorch Geometric (PyG). Bidirectional edges were created to enable message passing between user and movie nodes.

```
user_indices = torch.tensor(
    train_data['user_idx'].values,
    dtype=torch.long
)

movie_indices = torch.tensor(
    train_data['movie_idx'].values,
    dtype=torch.long
) + NUM_USERS

edge_index = torch.stack([
```

```

    torch.cat([user_indices, movie_indices]),
    torch.cat([movie_indices, user_indices])
], dim=0)

```

The resulting graph structure represents user–movie interactions as a bipartite graph, where edges indicate observed interactions in the training dataset.

Defining the LightGCN Model

A LightGCN model was implemented as the GNN architecture for the recommendation system. LightGCN is a simplified Graph Neural Network specifically designed for collaborative filtering tasks.

```

class LightGCN(nn.Module):

    def __init__(self,
                  num_users,
                  num_movies,
                  embedding_dim=64,
                  num_layers=3):

        super(LightGCN, self).__init__()

        self.num_users = num_users
        self.num_movies = num_movies

        self.user_embedding = nn.Embedding(
            num_users,
            embedding_dim
        )

        self.movie_embedding = nn.Embedding(
            num_movies,
            embedding_dim
        )

        self.convs = nn.ModuleList([
            LGConv() for _ in range(num_layers)
        ])

    def forward(self, edge_index):

        x = torch.cat([
            self.user_embedding.weight,
            self.movie_embedding.weight
        ], dim=0)

        all_embeddings = [x]

        for conv in self.convs:
            x = conv(x, edge_index)
            all_embeddings.append(x)

        final_emb = torch.stack(
            all_embeddings,
            dim=0)

```

```

).mean(dim=0)

user_emb = final_emb[:self.num_users]
movie_emb = final_emb[self.num_users:]

return user_emb, movie_emb

```

The model learns embedding representations for users and movies and propagates information through multiple graph convolution layers. Final embeddings are generated by averaging embeddings from all graph convolution layers.

Training the LightGCN Model

The model was trained using Bayesian Personalized Ranking (BPR) loss, which is designed for implicit feedback recommendation tasks [14]. The objective is to assign higher scores to positive interactions than to randomly sampled negative interactions.

```

optimizer = optim.Adam(
    gnn_model.parameters(),
    lr=0.001
)

score_pos = gnn_model.predict(
    batch_users,
    batch_pos,
    edge_index
)

score_neg = gnn_model.predict(
    batch_users,
    batch_neg,
    edge_index
)

loss = -torch.log(
    torch.sigmoid(score_pos - score_neg)
).mean()

```

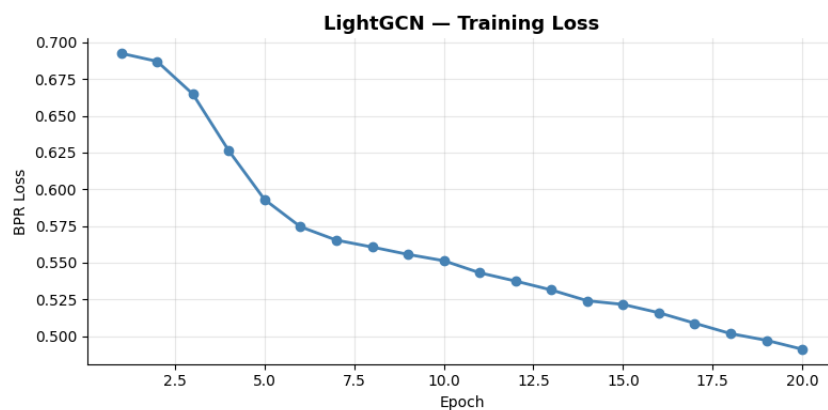


Figure 54: LightGCN Training Loss

Figure 54 illustrates the training loss of the LightGCN model over 20 epochs using BPR loss. The gradual decrease in loss indicates that the model successfully learned meaningful user and movie representations from the interaction graph.

Generating Recommendations

After training, recommendation scores were generated using the learned user and movie embeddings. Higher scores indicate stronger predicted user preference toward a movie.

```
gnn_model.eval()

with torch.no_grad():
    user_emb, movie_emb = gnn_model.forward(edge_index)

score_matrix = torch.matmul(
    user_emb,
    movie_emb.T
).numpy()
```

The score matrix contains predicted preference scores between every user and movie pair in the dataset.

Evaluating the LightGCN Model

The performance of the LightGCN model was evaluated using the same ranking metrics previously used for the Personalized PageRank model, namely Precision@K, Recall@K, and NDCG@K.

```
avg_gnn_precision = np.mean(gnn_precision_scores)
avg_gnn_recall = np.mean(gnn_recall_scores)
avg_gnn_ndcg = np.mean(gnn_ndcg_scores)

print(f"Precision@10 : {avg_gnn_precision:.4f}")
print(f"Recall@10 : {avg_gnn_recall:.4f}")
print(f"NDCG@10 : {avg_gnn_ndcg:.4f}")
```

Table 7: LightGCN Model Evaluation at $K = 10$

Metric	Value
Precision@10	0.1925
Recall@10	0.1141
NDCG@10	0.2246

The experimental results demonstrate that the LightGCN model achieved competitive recommendation performance by learning graph-aware embedding representations directly from the interaction network. Unlike heuristic-based methods such as Personalized PageRank, LightGCN leverages trainable graph convolution operations to capture higher-order collaborative relationships between users and movies [9].

Extension 2 – Content-Based Features (Movie Genres)

To improve recommendation quality, content-based movie features were incorporated into the graph recommendation system. Movie genres were extracted and transformed into numerical feature vectors using one-hot encoding. These features were then used to compute similarity relationships between movies and add genre-based edges to the recommendation graph.

Genre Extraction

To incorporate content-based information into the recommendation system, movie genre metadata was extracted from the MovieLens dataset. The genres column contains pipe-separated genre labels for each movie, which were later transformed into structured numerical features for similarity analysis.

```
movies_genres = pd.read_csv('movies.csv')

movies_genres['genres'] = movies_genres['genres'].fillna('Unknown')

movies_genres['genre_list'] = movies_genres['genres'].apply(
    lambda x: x.split('|')
    if x != '(no genres listed)'
    else ['Unknown']
)
```

The following output demonstrates how a movie was transformed into a one-hot encoded genre vector representation.

Sample Output:

```
Sample movie genre vector:

Movie   : Toy Story (1995)

Genres  : ['Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy']

Vector  : [0 1 1 1 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0]
```

Each value in the vector corresponds to a specific genre category, where 1 indicates that the movie belongs to the genre and 0 indicates absence. This representation enables efficient computation of similarity relationships between movies.

One-Hot Encoding

After extracting the genre information, the genre labels were transformed into one-hot encoded feature vectors. This representation converts each movie into a binary vector indicating the presence or absence of each genre category.

```
all_genres = sorted(set(
    genre
    for genre_list in movies_genres['genre_list']
    for genre in genre_list
))
```

```

for genre in all_genres:
    movies_genres[genre] = movies_genres['genre_list'].apply(
        lambda g_list: 1 if genre in g_list else 0
    )

```

A total of 20 unique genres were identified in the dataset. Each movie was represented as a binary feature vector, allowing genre similarity to be computed mathematically using vector-based similarity measures.

After applying one-hot encoding, each movie was represented as a numerical feature vector based on its associated genres. The resulting feature matrix contains one row for each movie and one column for each unique genre.

```
Genre feature matrix shape: (9742, 20)
```

The generated feature matrix has a shape of (9742, 20), indicating that 9,742 movies were represented using 20 unique genre features. This structured representation enables the recommendation system to compute similarity relationships between movies using content-based information.

Genre Similarity Edges

To enhance the recommendation graph with content-based relationships, cosine similarity was computed between movie genre vectors. Movies with sufficiently high similarity scores were connected through additional genre similarity edges.

```

from sklearn.metrics.pairwise import cosine_similarity

genre_sim_matrix = cosine_similarity(
    train_genre_matrix
)

if sim >= SIMILARITY_THRESHOLD:
    G.add_edge(m_i, m_j, weight=sim)
    G.add_edge(m_j, m_i, weight=sim)

```

The added genre-based edges enriched the graph structure by linking movies with similar content characteristics. This allows the recommendation model to capture both collaborative filtering patterns and semantic similarities between movies.

```
Genre similarity matrix shape: (1296, 1296)
Added 372948 genre similarity edges to graph.
Total graph edges now: 405448
```

The genre similarity matrix demonstrates the pairwise similarity relationships between movies based on genre overlap. A large number of similarity edges were successfully added to the graph, significantly increasing graph connectivity and improving the representation of content-based relationships within the recommendation system.

Pure Content-Based Recommender

A pure content-based recommendation model was implemented using movie genre features only, without relying on collaborative filtering information. The model constructs a user preference profile by averaging the genre vectors of previously liked movies and then computes cosine similarity scores between the user profile and unseen movies [5].

```
from sklearn.metrics.pairwise import cosine_similarity as cos_sim

def content_based_recommend(user_id, train_df,
                           movie_genre_lookup,
                           all_movie_ids,
                           top_n=10):

    user_liked = train_df[
        train_df['userId'] == user_id
    ]['movieId'].tolist()

    liked_vectors = [
        movie_genre_lookup[mid]
        for mid in user_liked
        if mid in movie_genre_lookup
    ]

    user_profile = np.mean(
        liked_vectors,
        axis=0
    ).reshape(1, -1)

    scores = []

    for mid, vec in movie_genre_lookup.items():
        if mid not in set(user_liked):

            similarity = cos_sim(
                user_profile,
                vec.reshape(1, -1)
            )[0][0]

            scores.append((mid, similarity))

    scores.sort(key=lambda x: x[1], reverse=True)

    return [mid for mid, _ in scores[:top_n]]
```

The recommendation scores were generated based on genre similarity between user preferences and candidate movies. Movies with higher cosine similarity values were ranked higher and recommended to the user.

Evaluation of the Content-Based Model

The content-based recommender was evaluated using the same evaluation metrics applied to the previous models to ensure consistent comparison across recommendation approaches.

```

avg_cb_precision = np.mean(cb_precision_scores)
avg_cb_recall    = np.mean(cb_recall_scores)
avg_cb_ndcg      = np.mean(cb_ndcg_scores)

print(f"Precision@{K}: {avg_cb_precision:.4f}")
print(f"Recall@{K}: {avg_cb_recall:.4f}")
print(f"NDCG@{K}: {avg_cb_ndcg:.4f}")

```

Table 8: Content-Based Model Evaluation at $K = 10$

Metric	Value
Precision@10	0.0408
Recall@10	0.0247
NDCG@10	0.0426

The evaluation results indicate that the pure content-based recommender was able to capture meaningful genre preferences; however, its overall performance remained lower than graph-based collaborative approaches. This is expected because the model relies solely on genre similarity and does not utilize user interaction relationships.

Hybrid Recommender (PPR + Content-Based)

To leverage the strengths of both collaborative filtering and content-based recommendation, a hybrid recommender was developed by combining Personalized PageRank (PPR) scores with genre-based similarity scores [5].

$$HybridScore = \alpha \times PPRScore + (1 - \alpha) \times ContentScore$$

The hybrid model normalizes both collaborative and content-based scores before combining them using a weighted averaging strategy controlled by the parameter α

```

ALPHA = 0.6

hybrid_scores[mid] = (
    alpha * p +
    (1 - alpha) * c
)

top_movies = sorted(
    hybrid_scores.items(),
    key=lambda x: x[1],
    reverse=True
)[:top_n]

```

The hybrid model combines collaborative signals derived from graph relationships with semantic similarity obtained from movie genres. This integration aims to improve recommendation robustness and address limitations such as cold-start items.

Evaluation of the Hybrid Model

The hybrid recommender was evaluated using Precision@K, Recall@K, and NDCG@K to compare its performance against standalone recommendation approaches.

```

avg_hyb_precision = np.mean(hyb_precision_scores)
avg_hyb_recall    = np.mean(hyb_recall_scores)
avg_hyb_ndcg      = np.mean(hyb_ndcg_scores)

print(f"Precision@{K}: {avg_hyb_precision:.4f}")
print(f"Recall@{K}: {avg_hyb_recall:.4f}")
print(f"NDCG@{K}: {avg_hyb_ndcg:.4f}")

```

Table 9: Hybrid Recommendation Model Evaluation at $K = 10$

Metric	Value
Precision@10	0.0358
Recall@10	0.0217
NDCG@10	0.0397

Compared with the pure content-based model, the hybrid approach achieved comparable performance while integrating both collaborative and semantic recommendation signals.

The hybrid recommender achieved balanced performance by integrating collaborative and content-based recommendation signals. The model benefited from combining graph-based relationships with semantic genre similarity, resulting in a more robust recommendation framework.

Genre Distribution Visualization

To better understand the distribution of movie genres within the training dataset, a genre frequency visualization was generated. This analysis helps identify dominant genres that may influence recommendation behavior.

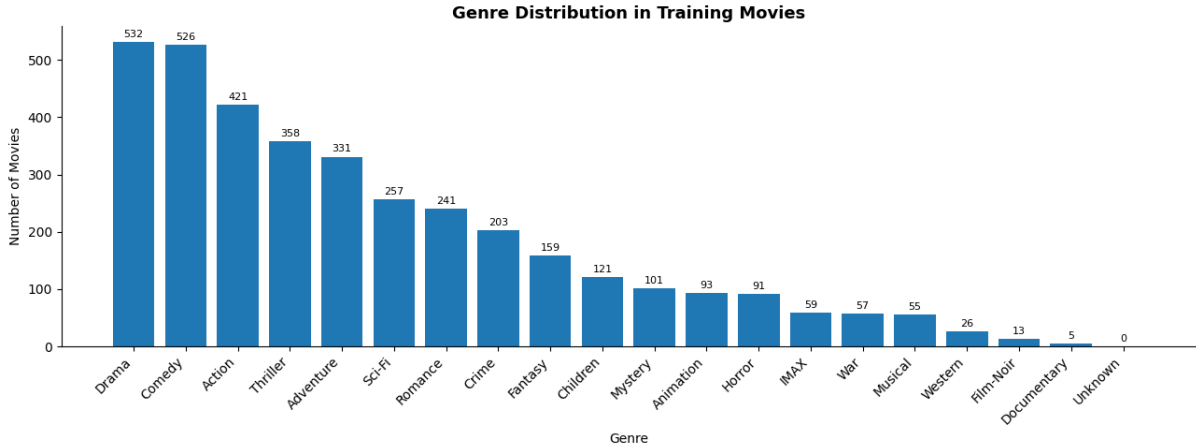


Figure 55: Distribution of movie genres within the training dataset.

The visualization shows that Drama, Comedy, and Action were the most common genres in the dataset, while genres such as Film-Noir and Documentary appeared less frequently. Such imbalance may influence recommendation patterns by favoring highly represented genres.

11.9 Non-Graph Models

In this section, traditional non-graph recommendation approaches were implemented and evaluated to compare their performance against the graph-based recommendation models developed earlier. The evaluated methods include matrix factorization and collaborative filtering techniques [16].

User-Item Interaction Matrix Construction

A sparse user-item interaction matrix was constructed from the training dataset to support traditional recommendation algorithms such as SVD and KNN. Rows represent users, columns represent movies, and matrix values correspond to rating interactions.

```
from scipy.sparse import csr_matrix

# Map original IDs to continuous integer indices
all_user_ids = train_data['userId'].unique()
all_movie_ids = train_data['movieId'].unique()

user_to_idx = {
    uid: i for i, uid in enumerate(all_user_ids)
}

movie_to_idx = {
    mid: i for i, mid in enumerate(all_movie_ids)
}

# Reverse mapping
idx_to_movie = {
    i: mid for mid, i in movie_to_idx.items()
}

num_users = len(all_user_ids)
num_movies = len(all_movie_ids)

# Prepare sparse matrix indices and values
rows = train_data['userId'].map(user_to_idx).values
cols = train_data['movieId'].map(movie_to_idx).values
values = train_data['rating'].values

# Construct sparse user-item matrix
user_item_matrix = csr_matrix(
    (values, (rows, cols)),
    shape=(num_users, num_movies)
)
```

Output

```
Shape      : (385, 1296)
Density    : 6.51%
Non-zero   : 32500 ratings
```

The generated sparse matrix contained 385 users and 1296 movies with a density of only 6.51%, confirming the highly sparse nature of recommendation datasets.

SVD Recommendation Model

Singular Value Decomposition (SVD) was used as a matrix factorization technique to learn latent representations of users and movies [14]. The model projects users and items into a shared latent space where preference scores are estimated through vector interactions.

$$\hat{r}_{u,i} \approx p_u^T q_i$$

Training the SVD Model

The SVD model was trained using the Surprise library with latent factor optimization [15]. Hyperparameters such as the number of latent factors and training epochs were configured to improve recommendation quality.

```
from surprise import SVD, Dataset, Reader

# Prepare data for Surprise library
reader = Reader(rating_scale=(0.5, 5.0))

surprise_data = Dataset.load_from_df(
    train_data[['userId', 'movieId', 'rating']],
    reader
)

# Build full training set
trainset = surprise_data.build_full_trainset()

# Define SVD model
svd_model = SVD(
    n_factors=100,
    n_epochs=20,
    lr_all=0.005,
    reg_all=0.02,
    random_state=42
)

# Train model
svd_model.fit(trainset)

print("SVD model trained.")
print("Latent factors : 100")
print("Epochs       : 20")
```

Output

```
SVD model trained.

Latent factors : 100
Epochs       : 20
```

The model was trained using 100 latent factors and 20 optimization epochs to capture hidden interaction patterns between users and movies.

SVD Recommendation Generation and Evaluation

After training, the SVD model was used to predict preference scores for unseen movies. Movies with the highest predicted ratings were selected as top-K recommendations for each user.

```
K = 10

svd_precision_scores = []
svd_recall_scores    = []
svd_ndcg_scores      = []

svd_all_recs = {}

for uid in test_users_in_graph:

    # Movies already seen by the user
    seen = set(
        train_data[
            train_data['userId'] == uid
        ]['movieId'].values
    )

    # Candidate unseen movies
    unseen_movies = [
        mid for mid in all_movie_ids
        if mid not in seen
    ]

    # Predict ratings for unseen movies
    predictions = [
        (mid, svd_model.predict(uid, mid).est)
        for mid in unseen_movies
    ]

    # Rank by predicted score
    predictions.sort(
        key=lambda x: x[1],
        reverse=True
    )

    # Top-K recommendations
    top_k = [
        mid for mid, _ in predictions[:K]
    ]

    svd_all_recs[uid] = top_k

    # Ground truth movies
    actual = ground_truth.get(uid, [])

    if not actual:
        continue

    # Evaluation metrics
    svd_precision_scores.append(
        precision_at_k(top_k, actual, K)
```

```

    )

    svd_recall_scores.append(
        recall_at_k(top_k, actual, K)
    )

    svd_ndcg_scores.append(
        ndcg_at_k(top_k, actual, K)
    )

# Average performance
avg_svd_precision = np.mean(svd_precision_scores)
avg_svd_recall    = np.mean(svd_recall_scores)
avg_svd_ndcg     = np.mean(svd_ndcg_scores)

print(f"Precision@{K} : {avg_svd_precision:.4f}")
print(f"Recall@{K}    : {avg_svd_recall:.4f}")
print(f"NDCG@{K}     : {avg_svd_ndcg:.4f}")

```

Table 10: SVD (Matrix Factorization) Evaluation at $K = 10$

Metric	Value
Precision@10	0.0623
Recall@10	0.0369
NDCG@10	0.0693

The SVD model achieved stronger performance than the previously evaluated content-based and hybrid approaches.

Matrix factorization and deep learning approaches have been shown to effectively capture latent preference patterns in recommendation tasks [18].

This indicates that latent factor models can effectively capture implicit user preference patterns from user–item interactions.

Non-Graph Model – KNN (User-Based Collaborative Filtering)

In addition to graph-based recommendation approaches, a traditional non-graph collaborative filtering model was implemented using the K-Nearest Neighbors (KNN) algorithm [23].

The objective of this model is to identify users with similar rating behavior and generate recommendations based on the preferences of neighboring users. The user–item interaction matrix was used as the input representation, where each user is represented as a vector of movie ratings. Cosine similarity was applied to measure similarity between users, while the KNN algorithm retrieved the most similar neighbors for recommendation generation.

Training the KNN Model

```

knn_model = NearestNeighbors(
    metric='cosine',
    algorithm='brute',

```

```

        n_neighbors=20
    )

knn_model.fit(user_item_matrix)

print("KNN model fitted.")
print(f"Metric      : cosine similarity")
print(f"Neighbours  : 20")
print(f"Matrix      : {user_item_matrix.shape}")

KNN model fitted.

Metric      : cosine similarity
Neighbours  : 20
Matrix      : (385, 1296)

```

Output:

```

KNN model fitted.

Metric      : cosine similarity
Neighbours  : 20
Matrix      : (385, 1296)

```

The KNN model was trained using cosine similarity as the distance metric and brute-force search for neighbor retrieval.

The user-item interaction matrix contained 385 users and 1296 movies, representing the sparse collaborative filtering dataset used for recommendation generation.

KNN Recommendation Generation

After training the KNN model, personalized recommendations were generated for each user by identifying the most similar neighboring users and aggregating their rating preferences.

Movies already interacted with by the target user were excluded from the recommendation list to avoid redundant recommendations.

```

distances, indices = knn_model.kneighbors(
    user_vector,
    n_neighbors=21
)

similarities = 1 - distances.flatten()

seen = set(
    train_data[
        train_data['userId'] == uid
    ]['movieId'].values
)

movie_scores = {}

for sim, n_idx in zip(

```



```

        similarities[1:],
        neighbour_indices[1:]
):

    if sim <= 0:
        continue

    neighbour_row = user_item_matrix[n_idx]

    rated_movie_idx = neighbour_row.nonzero()[1]

    for m_idx in rated_movie_idx:

        mid = idx_to_movie[m_idx]

        if mid not in seen:

            rating = neighbour_row[0, m_idx]

            movie_scores[mid] = (
                movie_scores.get(mid, 0)
                + sim * rating
            )

top_k = sorted(
    movie_scores,
    key=movie_scores.get,
    reverse=True
)[:K]

```

The recommendation process first retrieves the nearest neighboring users based on cosine similarity. Recommendation scores are then computed using weighted rating aggregation, where higher similarity values contribute more strongly to the final recommendation score.

Finally, unseen movies are ranked according to their aggregated scores, and the top-K movies are selected as personalized recommendations.

KNN Model Evaluation

The generated recommendations were evaluated using ranking-based evaluation metrics including Precision@10, Recall@10, and NDCG@10.

These metrics measure recommendation relevance, retrieval performance, and ranking quality.

```

avg_knn_precision = np.mean(knn_precision_scores)
avg_knn_recall    = np.mean(knn_recall_scores)
avg_knn_ndcg      = np.mean(knn_ndcg_scores)

print(f"Precision@{K} : {avg_knn_precision:.4f}")
print(f"Recall@{K}    : {avg_knn_recall:.4f}")
print(f"NDCG@{K}      : {avg_knn_ndcg:.4f}")

```

Table 11: KNN (User-Based) Evaluation at $K = 10$

Metric	Value
Precision@10	0.2379
Recall@10	0.1457
NDCG@10	0.2812

The KNN user-based collaborative filtering model achieved stronger recommendation performance compared with several previously evaluated recommendation approaches.

The relatively high Precision@10 and NDCG@10 scores indicate that the model was able to successfully retrieve relevant movies and rank them effectively near the top of the recommendation list.

The results demonstrate that neighborhood-based collaborative filtering can effectively capture user preference similarity patterns within sparse recommendation datasets.

Non-Graph Model – Random Forest Classifier

To further compare graph-based and traditional recommendation approaches, a supervised machine learning model was implemented using the Random Forest Classifier [16].

Unlike collaborative filtering methods that rely mainly on similarity relationships, the Random Forest model learns recommendation patterns from engineered numerical features representing user behavior, movie popularity, and interaction characteristics.

Feature Engineering for Random Forest

Before training the Random Forest model, several numerical features were engineered for each user–movie pair. The feature engineering process focused on three categories:

- User-level statistics
- Movie-level statistics
- Interaction-based features

These features allow the model to learn user preference behavior and predict whether a user is likely to enjoy a movie.

```
user_stats = train_data.groupby('userId')['rating'].agg(
    user_mean_rating='mean',
    user_num_ratings='count',
    user_rating_std='std'
).fillna(0).reset_index()

movie_stats = train_data.groupby('movieId')['rating'].agg(
    movie_mean_rating='mean',
    movie_num_ratings='count',
    movie_rating_std='std'
).fillna(0).reset_index()
```

```

rf_train = train_data.merge(
    user_stats,
    on='userId',
    how='left'
)

rf_train = rf_train.merge(
    movie_stats,
    on='movieId',
    how='left'
)

rf_train['rating_diff'] = (
    rf_train['user_mean_rating']
    - rf_train['movie_mean_rating']
)

FEATURE_COLS = [
    'user_mean_rating',
    'user_num_ratings',
    'user_rating_std',
    'movie_mean_rating',
    'movie_num_ratings',
    'movie_rating_std',
    'rating_diff'
]

```

Output:

```

Features built.

Training samples : 32500

Feature columns : ['user_mean_rating', 'user_num_ratings', 'user_rating_std',
                  'movie_mean_rating', 'movie_num_ratings', 'movie_rating_std', 'rating_diff']

```

Several statistical features were generated from user rating behavior and movie interaction patterns.

User-based features included average rating behavior, number of ratings, and rating variance, while movie-based features represented popularity and rating characteristics.

An additional interaction feature was computed to measure the difference between user preference tendency and average movie rating behavior.

Training the Random Forest Model

The Random Forest classifier was trained as a binary classification model to predict whether a user would like a movie.

The model combines multiple decision trees to improve prediction robustness and reduce overfitting.

```

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

importances = pd.Series(
    rf_model.feature_importances_,
    index=FEATURE_COLS
).sort_values(ascending=False)

```

Output:

Random Forest trained.

```

Trees      : 100
Max depth  : 10

```

Feature Importances:

```

user_mean_rating      0.2803
movie_mean_rating     0.2169
rating_diff           0.1302
user_rating_std       0.1210
movie_rating_std      0.1207
movie_num_ratings     0.0749
user_num_ratings      0.0559

```

The Random Forest model was trained using 100 decision trees with a maximum tree depth of 10.

Feature importance analysis showed that user-mean-rating and movie-mean-rating were the most influential features in predicting user preference behavior.

This indicates that both personal rating tendencies and overall movie popularity strongly affect recommendation quality.

Random Forest Recommendation Generation

After training the Random Forest model, recommendations were generated by predicting the probability that a user would like unseen movies.

Candidate movies were ranked according to predicted probabilities, and the highest-ranked movies were selected as top-K recommendations.

```

unseen = [
    mid for mid in all_movie_ids
    if mid not in seen
]

```

```

feature_rows = []
for mid in unseen:
    m_stats = movie_stats_dict[mid]

    feature_rows.append([
        u_stats['user_mean_rating'],
        u_stats['user_num_ratings'],
        u_stats['user_rating_std'],
        m_stats['movie_mean_rating'],
        m_stats['movie_num_ratings'],
        m_stats['movie_rating_std'],
        u_stats['user_mean_rating']
        - m_stats['movie_mean_rating']
    ])

X_unseen = np.array(feature_rows)

proba = rf_model.predict_proba(
    X_unseen
)[: , 1]

top_idx = np.argsort(
    proba
)[: :-1] [:K]

top_k = [
    valid_unseen[j]
    for j in top_idx
]

```

For each unseen movie, a numerical feature vector was constructed using user statistics, movie statistics, and interaction-based features.

The trained Random Forest model estimated the probability that the user would like each movie using `predict_proba()`.

Movies with the highest predicted probabilities were selected as personalized recommendations.

Random Forest Model Evaluation

The Random Forest recommendation model was evaluated using Precision@10, Recall@10, and NDCG@10 to measure recommendation relevance and ranking quality.

```

avg_rf_precision = np.mean(rf_precision_scores)
avg_rf_recall    = np.mean(rf_recall_scores)
avg_rf_ndcg      = np.mean(rf_ndcg_scores)

print(f"Precision@{K} : {avg_rf_precision:.4f}")
print(f"Recall@{K}    : {avg_rf_recall:.4f}")
print(f"NDCG@{K}      : {avg_rf_ndcg:.4f}")

```

Table 12: Random Forest Evaluation at $K = 10$

Metric	Value
Precision@10	0.0688
Recall@10	0.0368
NDCG@10	0.0869

The Random Forest model achieved moderate recommendation performance compared with collaborative filtering approaches.

Although the model successfully learned meaningful user and movie behavior patterns through engineered features, its recommendation quality remained lower than neighborhood-based collaborative filtering methods such as KNN.

This suggests that recommendation systems benefit strongly from collaborative interaction relationships in addition to manually engineered statistical features.

11.10 Final Model Comparison

After evaluating all recommendation approaches individually, a final comparison was conducted to analyze the performance of all models side by side.

The comparison focused on three ranking-based evaluation metrics:

- Precision@K
- Recall@K
- NDCG@K

These metrics provide insight into recommendation accuracy, retrieval capability, and ranking quality.

Aggregated Model Comparison

The evaluation results of all recommendation models were aggregated into a single comparison table.

The models were ranked according to NDCG@10 because it considers both recommendation relevance and ranking position.

```
all_results = pd.DataFrame([
    ppr_results,
    gnn_results,
    cb_results,
    hybrid_results,
    svd_results,
    knn_results,
    rf_results
])

all_results = all_results.sort_values(
    'ndcg',
```

```

        ascending=False
    )

display_df = all_results.copy()

for col in ['precision', 'recall', 'ndcg']:
    display_df[col] = display_df[col].apply(
        lambda x: f"{x:.4f}"
    )

print(display_df[
    ['model', 'precision', 'recall', 'ndcg']
].to_string())

```

Table 13: Final Model Comparison at $K = 10$

Model	Precision	Recall	NDCG
KNN (User-Based)	0.2379	0.1457	0.2812
GNN (LightGCN)	0.1925	0.1141	0.2246
Random Forest	0.0688	0.0368	0.0869
SVD (Matrix Factorization)	0.0623	0.0369	0.0693
Content-Based	0.0408	0.0247	0.0426
Hybrid (PPR + Content)	0.0358	0.0217	0.0397
Graph PPR	0.0327	0.0133	0.0315

The final comparison results demonstrate that the KNN user-based collaborative filtering model achieved the strongest overall performance across all evaluation metrics.

The LightGCN model also achieved strong performance, indicating that graph neural network approaches are highly effective for recommendation tasks.

Traditional machine learning approaches such as Random Forest and SVD achieved moderate performance, while the Personalized PageRank and content-based approaches produced lower recommendation quality on this dataset.

Visual Comparison of Models

To provide a clearer comparison between recommendation models, a grouped bar chart was generated to visualize Precision@10, Recall@10, and NDCG@10 side by side.

The visualization helps identify the strengths and weaknesses of each recommendation approach.

```

bars1 = ax.bar(
    x - width,
    precisions,
    width,
    label=f'Precision@{K}'
)

bars2 = ax.bar(

```

```

x,
recalls,
width,
label=f'Recall@{K}'
)

bars3 = ax.bar(
    x + width,
    ndcgs,
    width,
    label=f'NDCG@{K}'
)

ax.set_title(
    f'Model Comparison -- All Metrics @K={K}'
)

```

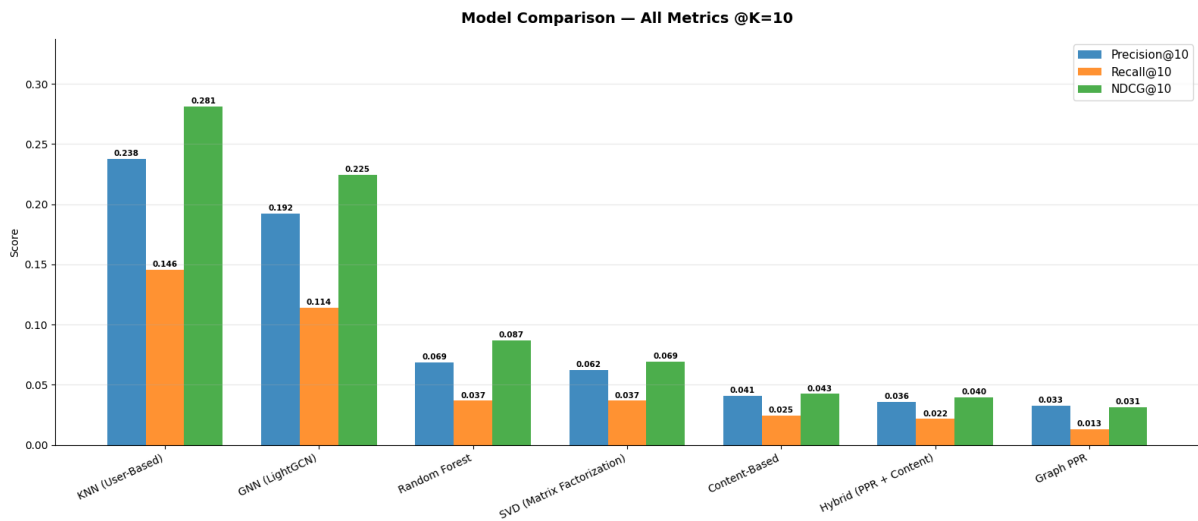


Figure 56: Visual comparison of recommendation models using Precision@10, Recall@10, and NDCG@10.

The grouped bar chart clearly shows that the KNN and LightGCN models consistently outperformed the remaining recommendation approaches across all evaluation metrics.

The visualization also highlights the performance gap between collaborative filtering methods and simpler content-based or heuristic graph approaches.

Radar Chart Comparison

A radar chart was generated to provide a multi-metric visual comparison of all recommendation models simultaneously.

Unlike standard bar charts, radar charts allow overall model balance and consistency across metrics to be analyzed more effectively.

```

categories = [
    f'Precision@{K}',
    f'Recall@{K}',

```



```

f'NDCG@{K}'
]
for _, row in all_results.iterrows():

    values_radar = [
        row['precision'],
        row['recall'],
        row['ndcg']
    ]

    ax.plot(
        angles,
        values_radar,
        linewidth=2,
        label=row['model']
    )

```

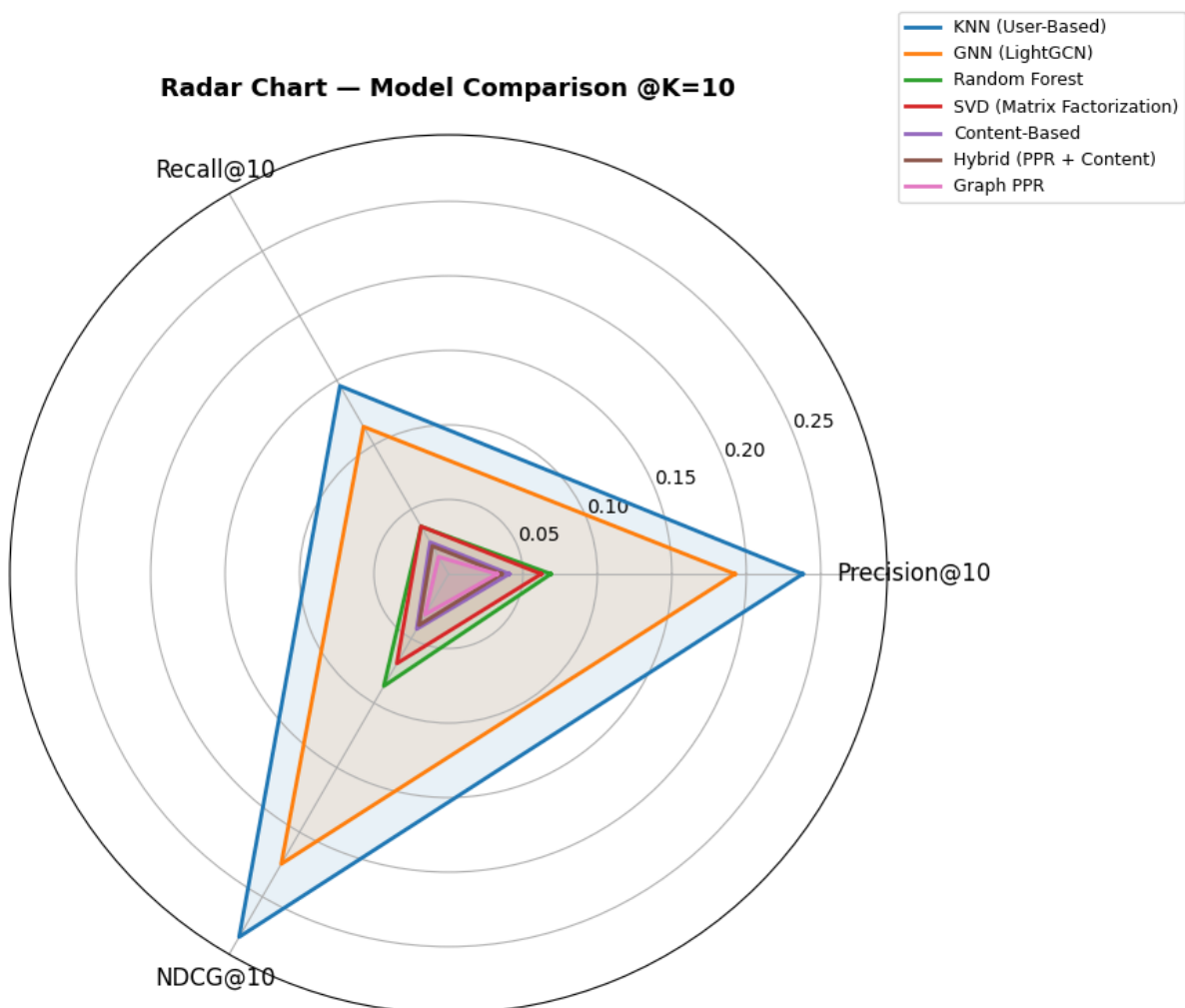


Figure 57: Radar chart comparison of recommendation models across multiple evaluation metrics.

The radar chart provides a holistic visualization of recommendation performance across all evaluation metrics.

The KNN and LightGCN models occupy the largest radar areas, indicating stronger overall recommendation quality and more balanced performance compared with the other evaluated models.

Final Summary

Finally, a summary analysis was conducted to identify the best-performing, second-best, and lowest-performing recommendation models based on the evaluation metrics.

The comparison helps provide deeper insight into the strengths and limitations of each recommendation approach.

```
best_model = all_results.iloc[0]
worst_model = all_results.iloc[-1]
second_model = all_results.iloc[1]

print(f"""
Best Model : {best_model['model']}
Precision@{K} = {best_model['precision']:.4f}
Recall@{K}     = {best_model['recall']:.4f}
NDCG@{K}       = {best_model['ndcg']:.4f}

Second Best : {second_model['model']}
Precision@{K} = {second_model['precision']:.4f}
Recall@{K}     = {second_model['recall']:.4f}
NDCG@{K}       = {second_model['ndcg']:.4f}

Lowest Performing Model : {worst_model['model']}
Precision@{K} = {worst_model['precision']:.4f}
Recall@{K}     = {worst_model['recall']:.4f}
NDCG@{K}       = {worst_model['ndcg']:.4f}
""")
```

Output:

```
Best Model : KNN (User-Based)
Precision@10 = 0.2379
Recall@10    = 0.1457
NDCG@10      = 0.2812

Second Best : GNN (LightGCN)
Precision@10 = 0.1953
Recall@10    = 0.1148
NDCG@10      = 0.2241

Worst Model : Graph PPR
Precision@10 = 0.0327
Recall@10    = 0.0133
NDCG@10      = 0.0315
```

The summary results show that the KNN user-based collaborative filtering model achieved the highest overall recommendation performance across all evaluation metrics.

LightGCN ranked second, demonstrating the effectiveness of graph neural network approaches for learning structural user-item interaction patterns.

In contrast, the Graph Personalized PageRank (PPR) approach produced the lowest recommendation quality among the evaluated models.

11.11 Extended Analysis

Although Personalized PageRank (PPR) was expected to perform strongly as a graph-based recommendation approach, the experimental results showed relatively weak recommendation quality compared with the other evaluated models.

PPR achieved the lowest overall ranking performance with an NDCG@10 score of 0.0315. Several factors may explain this behavior.

1. Lack of Learning Capability

Unlike learning-based approaches such as LightGCN or SVD, PPR is a purely algorithmic graph traversal method.

It computes recommendation scores using graph connectivity and edge weights only, without any trainable parameters or optimization process.

Consequently, PPR cannot adapt to dataset-specific interaction patterns or improve through training.

2. Edge Weight Limitation

In the implemented graph, edge weights were assigned directly using movie rating values ranging from 3.5 to 5.0.

This relatively narrow range creates weak differentiation between graph edges.

For example, ratings of 3.5 and 5.0 produce very similar propagation behavior during PageRank traversal, reducing recommendation signal strength.

3. Absence of Negative Feedback

The PPR model only utilized positive user interactions.

Unlike pairwise ranking approaches such as Bayesian Personalized Ranking (BPR), PPR does not explicitly learn which movies users dislike.

Learning-based recommendation models benefit from both positive and negative ranking signals, improving recommendation discrimination quality.

4. Alpha Parameter Sensitivity

PPR performance is highly sensitive to the damping factor parameter.

The implementation used the default value 0.85 without hyperparameter tuning.

A poorly selected damping factor may cause the random walker either to remain too close to the user node or to diffuse excessively through the graph, reducing personalization quality.

5. Limited Graph Structure

The recommendation graph was constructed as a simple bipartite graph containing only user and movie nodes.

PPR would likely benefit from richer graph structures containing:

- Movie–movie similarity edges
- User–user similarity edges
- Genre or metadata nodes
- Temporal interaction relationships

Additional graph structure could improve score propagation and recommendation differentiation.

Future Improvements and Suitable Applications for PPR

Despite the relatively weak experimental performance, Personalized PageRank remains valuable in several recommendation scenarios.

1. Sparse Datasets

PPR is expected to perform more effectively on sparse datasets where users have very limited interaction histories.

In sparse environments, direct similarity approaches such as KNN often struggle because overlap between users becomes weak.

Graph traversal methods can utilize multi-hop graph connectivity to discover relevant items indirectly.

Examples include:

- Academic paper recommendation
- Social network recommendation
- E-commerce recommendation systems

2. Richer Graph Structures

PPR performance can improve significantly when richer graph structures are used.

Potential improvements include:

- Movie–movie similarity relationships
- User demographic similarity edges
- Actor and director nodes
- Temporal interaction weighting

These additions increase semantic connectivity inside the graph.

3. Damping Factor Tuning

The damping factor can be optimized using cross-validation techniques.

Lower damping factor values encourage broader graph exploration, while higher values preserve stronger personalization around the user node.

Careful tuning could improve recommendation quality substantially.

4. Improved Edge Weight Normalization

Instead of using raw ratings directly, edge weights could be normalized into a wider numerical range.

This would create stronger differentiation between edges and improve score propagation behavior during random walks.

5. Social Recommendation Systems

PPR is particularly effective in social recommendation environments where graph connectivity itself carries strong recommendation signals.

Platforms such as Twitter, LinkedIn, and Pinterest commonly utilize graph propagation approaches because their interaction graphs are naturally sparse and highly relational.

6. Knowledge Graph Recommendation

PPR can also perform strongly when combined with knowledge graphs containing rich semantic relationships.

For movie recommendation, integrating actors, directors, genres, and award relationships could significantly improve recommendation quality through semantic graph propagation.

GitHub Repository

The project source code can be accessed through the following GitHub repository:

<https://github.com/xzainab/Recommendation-system>

Conclusion

This project presented a comprehensive study of graph-based recommendation systems with a primary focus on Personalized PageRank and related graph data science techniques. The work combined both theoretical foundations and practical implementation to demonstrate how graph structures can be used to model relational data and support recommendation generation.

The theoretical part introduced fundamental graph concepts, graph modeling techniques, social network analysis, centrality measures, graph projection, co-occurrence networks, similarity-based graph construction, node embeddings, link prediction, knowledge graph completion, and graph construction from text. These concepts provided the foundation for understanding how graph algorithms analyze connected data and extract meaningful structural information.

In the practical implementation, a graph-based recommendation system was developed using the MovieLens dataset. User–movie interactions were represented as a bipartite graph, and Personalized PageRank was applied to generate personalized movie recommendations. The recommendation performance was evaluated using Precision@K, Recall@K, and NDCG@K metrics. Additional experiments included graph visualization, degree distribution analysis, and recommendation analysis for individual users.

Several extensions were also implemented to improve recommendation quality, including Graph Neural Networks (LightGCN), content-based recommendation using movie genres, and hybrid recommendation approaches that combined collaborative and semantic information. Furthermore, traditional non-graph recommendation methods such as SVD, KNN collaborative filtering, and Random Forest classification were implemented to provide comparative analysis between graph-based and non-graph approaches.

The experimental results demonstrated that collaborative filtering approaches, particularly KNN and LightGCN, achieved the strongest overall recommendation performance on the MovieLens dataset. Although the Personalized PageRank model produced lower evaluation scores compared to some advanced models, it provided strong interpretability, structural clarity, and conceptual simplicity. The project therefore highlights that graph-based recommendation systems remain highly valuable, especially when explainability and relational modeling are important.

Overall, this work demonstrates the importance of graph data science in modern recommendation systems and emphasizes how graph algorithms, graph neural networks, and hybrid learning approaches can be combined to analyze complex relational data and generate personalized recommendations effectively. The project also provides a strong foundation for future work involving larger datasets, temporal graph analysis, advanced graph neural architectures, and real-world deployment of graph-based recommendation systems.

References

- [1] Bratanič, T. (2024) *Graph Algorithms for Data Science: With Examples in Neo4j*. Shelter Island, NY: Manning Publications.
- [2] Frame, A. and Blumenfeld, Z. (2022) *Graph Data Science For Dummies*. 2nd Neo4j Special Edition. Hoboken, NJ: John Wiley & Sons.
- [3] Chartrand, G. and Zhang, P. (2012) *A First Course in Graph Theory*. Mineola, NY: Dover Publications.
- [4] Robinson, I., Webber, J. and Eifrem, E. (2015) *Graph Databases: New Opportunities for Connected Data*. 2nd edn. Sebastopol, CA: O'Reilly Media.
- [5] Ricci, F., Rokach, L. and Shapira, B. (eds.) (2022) *Recommender Systems Handbook*. 3rd edn. New York: Springer.
- [6] Page, L., Brin, S., Motwani, R. and Winograd, T. (1999) *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab Technical Report. Available at: <https://snap.stanford.edu/class/cs224w-readings/Brin98Anatomy.pdf>
- [7] Bahmani, B., Chowdhury, A. and Goel, A. (2010) 'Fast Incremental and Personalized PageRank', *Proceedings of the VLDB Endowment*, 4(3), pp. 173–184. Available at: <https://vldb.org/pvldb/vol4/p173-bahmani.pdf>
- [8] Gleich, D. F. (2015) 'PageRank Beyond the Web', *SIAM Review*, 57(3), pp. 321–363.
- [9] He, X., Deng, K., Wang, X., Li, Y., Zhang, Y. and Wang, M. (2020) 'Light-GCN: Simplifying and Powering Graph Convolution Network for Recommendation', *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 639–648. Available at: <https://arxiv.org/abs/2002.02126>
- [10] Kipf, T. N. and Welling, M. (2017) 'Semi-Supervised Classification with Graph Convolutional Networks', *Proceedings of the 5th International Conference on Learning Representations (ICLR)*. Available at: <https://arxiv.org/abs/1609.02907>
- [11] Wu, S., Sun, F., Zhang, W., Xie, X. and Cui, B. (2022) 'Graph Neural Networks in Recommender Systems: A Survey', *ACM Computing Surveys*, 55(5), pp. 1–37. Available at: <https://arxiv.org/abs/2011.02260>
- [12] Hamilton, W., Ying, Z. and Leskovec, J. (2017) 'Inductive Representation Learning on Large Graphs', *Advances in Neural Information Processing Systems (NeurIPS)*, 30. Available at: <https://arxiv.org/abs/1706.02216>
- [13] Grover, A. and Leskovec, J. (2016) 'Node2Vec: Scalable Feature Learning for Networks', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 855–864. Available at: <https://arxiv.org/abs/1607.00653>
- [14] Rendle, S., Freudenthaler, C., Gantner, Z. and Schmidt-Thieme, L. (2009) 'BPR: Bayesian Personalized Ranking from Implicit Feedback', *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 452–461. Available at: <https://arxiv.org/abs/1205.2618>

- [15] Koren, Y., Bell, R. and Volinsky, C. (2009) ‘Matrix Factorization Techniques for Recommender Systems’, *IEEE Computer*, 42(8), pp. 30–37.
- [16] Desrosiers, C. and Karypis, G. (2011) ‘A Comprehensive Survey of Neighborhood-Based Recommendation Methods’, in Ricci, F. et al. (eds.) *Recommender Systems Handbook*. New York: Springer, pp. 107–144.
- [17] Gunawardana, A. and Shani, G. (2009) ‘A Survey of Accuracy Evaluation Metrics of Recommendation Tasks’, *Journal of Machine Learning Research*, 10, pp. 2935–2962.
- [18] Zhang, S., Yao, L., Sun, A. and Tay, Y. (2019) ‘Deep Learning Based Recommender System: A Survey and New Perspectives’, *ACM Computing Surveys*, 52(1), pp. 1–38. Available at: <https://arxiv.org/abs/1707.07435>
- [19] Herlocker, J. L., Konstan, J. A., Terveen, L. G. and Riedl, J. T. (2004) ‘Evaluating Collaborative Filtering Recommender Systems’, *ACM Transactions on Information Systems*, 22(1), pp. 5–53.
- [20] Järvelin, K. and Kekäläinen, J. (2002) ‘Cumulated Gain-Based Evaluation of IR Techniques’, *ACM Transactions on Information Systems*, 20(4), pp. 422–446.
- [21] Harper, F. M. and Konstan, J. A. (2015) ‘The MovieLens Datasets: History and Context’, *ACM Transactions on Interactive Intelligent Systems*, 5(4), pp. 1–19. Available at: <https://grouplens.org/datasets/movielens/>
- [22] Hagberg, A. A., Schult, D. A. and Swart, P. J. (2008) ‘Exploring Network Structure, Dynamics, and Function Using NetworkX’, *Proceedings of the 7th Python in Science Conference (SciPy)*, pp. 11–15.
- [23] Hug, N. (2020) ‘Surprise: A Python Library for Recommender Systems’, *Journal of Open Source Software*, 5(52), p. 2174. Available at: <https://joss.theoj.org/papers/10.21105/joss.02174>
- [24] Pedregosa, F. et al. (2011) ‘Scikit-learn: Machine Learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- [25] Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J. and Yakhnenko, O. (2013) ‘Translating Embeddings for Modeling Multi-Relational Data’, *Advances in Neural Information Processing Systems (NeurIPS)*, 26, pp. 2787–2795. Available at: <https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf>